

Option Explicit

```

'
' 只在「使用者更動輸入區、且全部必填欄位都有值」時才自動算
'
Private Sub Worksheet_Change(ByVal Target As Range)
    ' 防止自己寫 C13/C14 又觸發自己（無窮迴圈）
    Static busy As Boolean
    If busy Then Exit Sub

    ' 只關心輸入區 C4:C11；其他儲存格動到就忽略
    Dim inputZone As Range
    Set inputZone = Me.Range("C4,C6,C7,C8,C9,C10,C11")
    If Intersect(Target, inputZone) Is Nothing Then Exit Sub

    ' 必填檢查 (Ticker C5 不影響 Greeks, 不列入必填)
    Dim req As Variant, cellRef As Variant
    req = Array("C4", "C6", "C7", "C8", "C9", "C10")
    For Each cellRef In req
        If Trim(CStr(Me.Range(cellRef).Value)) = "" Then
            Me.Range("C13:C14").ClearContents
            Exit Sub
        End If
    Next cellRef

    ' 日期合法性檢查
    If Not IsDate(Me.Range("C4").Value) Or Not IsDate(Me.Range("C8").Value) Then
        Me.Range("C13:C14").ClearContents
        Exit Sub
    End If

    ' 到期必須晚於評價日（但不彈視窗、靜默清空）
    If CDate(Me.Range("C8").Value) <= CDate(Me.Range("C4").Value) Then
        Me.Range("C13:C14").ClearContents
        Exit Sub
    End If

    ' 數值欄位合法性檢查
    Dim numCells As Variant
    numCells = Array("C7", "C9", "C10") ' Strike, Underlying, IV
    For Each cellRef In numCells
        If Not IsNumeric(Me.Range(cellRef).Value) Then
            Me.Range("C13:C14").ClearContents
            Exit Sub
        End If
        If CDbl(Me.Range(cellRef).Value) <= 0 Then
            Me.Range("C13:C14").ClearContents
            Exit Sub
        End If
    Next cellRef

    ' 全部過關，呼叫計算
    busy = True
    On Error Resume Next
    Call CalculateGreeks
    On Error GoTo 0
    busy = False
End Sub

```

Option Explicit

```
Private Const WARRANT_TYPES As String = "Call Warrant,Put Warrant,Bull Cert,Bear Cert"
```

```
Private Sub UserForm_Initialize()
```

```
    cmbAction.List = Split("Buy,Sell,AdjustCost,Expire,Exercise", ",")
```

```
    cmbType.List = Split("Call,Put,Future,Call Warrant,Put Warrant,Bull Cert,Bear Cert,Equity,ETF", ",")
```

```
    cmbStrategy.List = Split("Bull Call,Bear Put,Straddle,Strangle,Iron Condor,Protective,Covered,Calendar,Ratio,Single", ",")
```

```
    cmbBroker.List = Split("Fubon,SinoPac,Cathay,KGI,Yuanta,Capital", ",")
```

```
    txtDate.Value = Format(Date, "yyyy/mm/dd")
```

```
    txtFee.Value = "0"
```

```
    txtTax.Value = "0"
```

```
    txtMultiplier.Value = "1"
```

```
    txtRiskFree.Value = "0.0200" ' 預設 2%，讀 _Config 失敗也有值
```

```
On Error Resume Next
```

```
Dim curR As Variant
```

```
curR = RR5_Core.GetConfigValue("BS_RISK_FREE_RATE")
```

```
If IsNumeric(curR) And CDBl(curR) > 0 Then
```

```
    txtRiskFree.Value = Format(CDBl(curR), "0.0000")
```

```
End If
```

```
On Error GoTo 0
```

```
Me.Caption = "Add Transaction - RR5"
```

```
End Sub
```

```
Private Sub txtShares_Change(): RecalcNet: End Sub
```

```
Private Sub txtPrice_Change(): RecalcNet: End Sub
```

```
Private Sub txtMultiplier_Change(): RecalcNet: End Sub
```

```
Private Sub txtFee_Change(): RecalcNet: End Sub
```

```
Private Sub txtTax_Change(): RecalcNet: End Sub
```

```
Private Sub RecalcNet()
```

```
    Dim s As Double, p As Double, m As Double, f As Double, tx As Double
```

```
    s = val(txtShares.Value)
```

```
    p = val(txtPrice.Value)
```

```
    m = val(txtMultiplier.Value): If m = 0 Then m = 1
```

```
    f = val(txtFee.Value)
```

```
    tx = val(txtTax.Value)
```

```
    txtNetAmount.Value = Format(s * p * m + f + tx, "#,##0.00")
```

```
End Sub
```

```
Private Sub AutoFillEntryGreeks()
```

```
    On Error GoTo SafeExit
```

```
    Dim t As String: t = cmbType.Value
```

```
    If t <> "Call" And t <> "Put" And _
```

```
        t <> "Call Warrant" And t <> "Put Warrant" And _
```

```
        t <> "Bull Cert" And t <> "Bear Cert" Then Exit Sub
```

```
If Not IsNumeric(txtUnderlying.Value) Then Exit Sub
```

```
If Not IsNumeric(txtStrike.Value) Then Exit Sub
```

```
If Not IsNumeric(txtIV.Value) Then Exit Sub
```

```
If Not IsNumeric(txtRiskFree.Value) Then Exit Sub
```

```
Dim s As Double, k As Double, v As Double, r As Double
```

```
s = CDBl(txtUnderlying.Value)
```

```
k = CDBl(txtStrike.Value)
```

```
v = CDBl(txtIV.Value)
```

```
r = CDBl(txtRiskFree.Value)
```

```
If s <= 0 Or k <= 0 Or v <= 0 Or r <= 0 Then Exit Sub
```

```
If Not IsDate(txtDate.Value) Or Not IsDate(txtExpiry.Value) Then Exit Sub
```

```
Dim T_years As Double
```

```
T_years = (CDate(txtExpiry.Value) - CDate(txtDate.Value)) / 365
```

```
If T_years <= 0 Then Exit Sub
```

```
Dim bsType As String
```

```
Select Case t
```

```
    Case "Call", "Call Warrant", "Bull Cert": bsType = "Call"
```

```
    Case Else: bsType = "Put"
```

End Select

```
If Trim(CStr(txtDelta.Value)) = "" Then
    txtDelta.Value = Format(RR5_Core.BS_Delta(s, k, T_years, r, v, bsType), "0.0000")
End If
If Trim(CStr(txtGamma.Value)) = "" Then
    txtGamma.Value = Format(RR5_Core.BS_Gamma(s, k, T_years, r, v), "0.000000")
End If
If Trim(CStr(txtTheta.Value)) = "" Then
    txtTheta.Value = Format(RR5_Core.BS_Theta(s, k, T_years, r, v, bsType), "0.0000")
End If
```

SafeExit:

End Sub

Private Sub btnSave_Click()

On Error GoTo ErrHandler

```
If Trim(txtTicker.Value) = "" Then
    MsgBox "Ticker required.", vbExclamation: txtTicker.SetFocus: Exit Sub
End If
If cmbAction.Value = "" Then
    MsgBox "Action required.", vbExclamation: cmbAction.SetFocus: Exit Sub
End If
If val(txtShares.Value) = 0 Then
    MsgBox "Shares/Contracts must be > 0.", vbExclamation: txtShares.SetFocus: Exit Sub
End If
If val(txtPrice.Value) = 0 Then
    MsgBox "Price must be > 0.", vbExclamation: txtPrice.SetFocus: Exit Sub
End If
```

Dim rfr As Double: rfr = val(txtRiskFree.Value)

If rfr <= 0 Or rfr >= 1 Then

```
    MsgBox "Risk-Free Rate 請輸入小數 (e.g. 0.02 = 2%)", _
        vbExclamation: txtRiskFree.SetFocus: Exit Sub
```

End If

On Error Resume Next

Call RR5_Core.SetConfigValue("BS_RISK_FREE_RATE", rfr)

On Error GoTo ErrHandler

Dim isWarrant As Boolean

isWarrant = (InStr(WARRANT_TYPES, cmbType.Value) > 0 And cmbType.Value <> "")

Dim crVal As Double: crVal = 0

If isWarrant Then

Dim crInput As String

```
crInput = InputBox("Conv Ratio (e.g. 0.1 = 10 warrants per share)", _
    "Conv Ratio - " & Trim(txtTicker.Value), "")
```

If Not IsNumeric(crInput) Or val(crInput) <= 0 Then

```
    MsgBox "Conv Ratio must be > 0.", vbExclamation: Exit Sub
```

End If

crVal = CDBl(crInput)

End If

Call AutoFillEntryGreeks

Dim rowData(1 To 24) As Variant

rowData(1) = RR5_Core.GenerateTxID()

rowData(2) = SafeDate(txtDate.Value)

rowData(3) = Trim(txtTicker.Value)

rowData(4) = cmbAction.Value

rowData(5) = cmbType.Value

rowData(6) = SafeNum(txtStrike.Value)

rowData(7) = SafeDate(txtExpiry.Value)

rowData(8) = val(txtShares.Value)

rowData(9) = IIf(val(txtMultiplier.Value) = 0, 1, val(txtMultiplier.Value))

rowData(10) = val(txtPrice.Value)

rowData(11) = val(txtFee.Value)

rowData(12) = val(txtTax.Value)

rowData(13) = val(Replace(txtNetAmount.Value, ",", ""))

rowData(14) = txtSector.Value

rowData(15) = SafeNum(txtTarget.Value)

rowData(16) = cmbStrategy.Value

```

rowData(17) = SafeNum(txtIV.Value)
rowData(18) = SafeNum(txtDelta.Value)
rowData(19) = SafeNum(txtBeta.Value)
rowData(20) = cmbBroker.Value
rowData(21) = Trim(txtUnderlying.Value)
rowData(22) = SafeNum(txtGamma.Value)
rowData(23) = SafeNum(txtTheta.Value)
rowData(24) = IIf(crVal > 0, crVal, "")

```

```

Call RR5_Core.AppendTransaction(rowData)
MsgBox "Transaction saved: " & rowData(1), vbInformation, "RR5 Transaction"
Call ClearAll
Exit Sub

```

```

ErrorHandler:
MsgBox "Save 錯誤: " & Err.Number & " - " & Err.Description, vbCritical
End Sub

```

```

Private Sub btnClear_Click(): ClearAll: End Sub
Private Sub btnCancel_Click(): Unload Me: End Sub

```

```

Private Function SafeNum(v As Variant) As Variant
If Trim(CStr(v)) = "" Then SafeNum = "" Else SafeNum = val(v)
End Function

```

```

Private Function SafeDate(v As Variant) As Variant
On Error Resume Next
Dim d As Date
If Trim(CStr(v)) = "" Then
SafeDate = ""
Else
d = CDate(v)
If Err.Number = 0 Then SafeDate = d Else SafeDate = v
End If
On Error GoTo 0
End Function

```

```

Private Sub ClearAll()
Dim ctrl As Control
For Each ctrl In Me.Controls
If ctrl.Name = "txtRiskFree" Then GoTo NextCtrl
Select Case TypeName(ctrl)
Case "TextBox": ctrl.Value = ""
Case "ComboBox": ctrl.Value = ""
End Select
NextCtrl:
Next ctrl
txtDate.Value = Format(Date, "yyyy/mm/dd")
txtFee.Value = "0"
txtTax.Value = "0"
txtMultiplier.Value = "1"
On Error Resume Next
txtTicker.SetFocus
End Sub

```

Option Explicit

```

' =====
' COMBINED PERFORMANCE TABLE  v1.1
' RR4 (Stocks) + RR5 (Derivatives) joint summary
'
' Entry points:
'   BuildCombinedPerf  -- full rebuild (format + data)
'   RefreshCombinedPerf -- data refresh only (fast)
'   SyncRR4Data       -- re-read open RR4 workbook
'
' RR5 data: auto-read from this workbook (B1/F2/H2/P1/L1/J2/L2/N2)
' RR4 data: auto-search open workbook with "RR4" in name;
'           orange cells if not found -- type manually then Refresh
' =====

Private Const SH_OUT As String = "CombinedPerf"
Private Const SH_RR5 As String = "RR5"
Private Const SH_LOG As String = "CPLog"

' Column in RR4 Analysis sheet that holds per-ticker market value.
' Run DiagnoseRR4 to inspect the sheet layout, then adjust if needed.
Private Const RR4_PIE_VAL_COL As Long = 7 ' col G = NET EXPOS in RR4 Analysis

' Color palette (dark Bloomberg style)
Private Const LOG_START As Date = #1/1/2026# ' CombinedLog date floor

Private Const CLR_BG_DARK As Long = 1973790 ' #1E1E1E = RGB(30,30,30) -- CPLog sheet bg
Private Const CLR_BG_MID As Long = 1776411 ' #1B1B1B = RGB(27,27,27) -- dark alternating row
Private Const CLR_BG_LIGHT As Long = 1973790 ' #1E1E1E = RGB(30,30,30)
Private Const CLR_SECTION As Long = 1973790 ' #1E1E1E = RGB(30,30,30) -- gray alternating row
Private Const CLR_FG_WHITE As Long = 16777215
Private Const CLR_FG_SILVER As Long = 12895428 ' #C4C4C4
Private Const CLR_FG_GOLD As Long = 55295 ' #FFD700 = RGB(255,215,0) -- actual gold
Private Const CLR_FG_CYAN As Long = 6750105 ' #66CCFF approx
Private Const CLR_FG_GREEN As Long = 6479952 ' #62DC50 approx (RR5 header color)
Private Const CLR_BORDER As Long = 5066061 ' #4D4D4D
Private Const CLR_ORANGE_BG As Long = 4687923 ' manual-input hint

' CPLog group colors: RR4 = yellow/gold, RR5 = blue, Combined = gray.
' Applied to header text, line-chart series, and line-chart titles alike.
Private Const CLR_GRP_RR4 As Long = 55295 ' RGB(255,215,0) -- reuses tested gold/yellow
Private Const CLR_GRP_RR5 As Long = 16754010 ' RGB(90,165,255) -- blue
Private Const CLR_GRP_COMB As Long = 12895428 ' RGB(196,196,196) -- gray (same as CLR_FG_SILVER)

' RR5 sheet cells for IV Avg and TWII Price -- adjust if layout differs
Private Const RR5_IV_CELL As String = "R2" ' avg implied vol of options portfolio
Private Const RR5_TWII_CELL As String = "T2" ' TWII index price

' Row constants
Private Const R_TITLE As Long = 1
Private Const R_UPDATED As Long = 2
Private Const R_HDR As Long = 4
Private Const R_NEXPOS As Long = 5
Private Const R_UNRL_PNL As Long = 6
Private Const R_RL_PNL As Long = 7
Private Const R_COMB_RL As Long = 8 ' Combined RL PNL -- adjacent to Realized PNL
Private Const R_TOT_PNL As Long = 9
Private Const R_UNRL_PCT As Long = 10
Private Const R_POSCOUNT As Long = 11
Private Const R_SEP1 As Long = 12
Private Const R_D_HDR As Long = 13
Private Const R_DELTA As Long = 14
Private Const R_THETA As Long = 15
Private Const R_VEGA As Long = 16
Private Const R_MARGIN As Long = 17
Private Const R_SEP2 As Long = 18
Private Const R_BK_HDR As Long = 19
Private Const R_BK_COL As Long = 20
Private Const R_FUT As Long = 21
Private Const R_OPT As Long = 22

```

```
Private Const R_WRT      As Long = 23
Private Const R_SEP3     As Long = 24
Private Const R_NOTE     As Long = 25
```

```
' Col constants
```

```
Private Const C_LABEL As Long = 1   ' A
Private Const C_RR4   As Long = 2   ' B
Private Const C_RR5   As Long = 3   ' C
Private Const C_TOTAL As Long = 4   ' D
```

```
' =====
' PUBLIC ENTRY POINTS
' =====
```

```
Public Sub BuildCombinedPerf()
```

```
On Error GoTo Fail
Application.ScreenUpdating = False
```

```
Dim wS As Worksheet
Set wS = EnsureSheet()
Call DrawShell(wS)
Call RefreshValues(wS)
```

```
Application.ScreenUpdating = True
Application.StatusBar = "CombinedPerf built OK " & Format(Now, "hh:mm:ss")
wS.Activate
ActiveWindow.DisplayGridlines = False
Exit Sub
```

```
Fail:
```

```
Application.ScreenUpdating = True
MsgBox "BuildCombinedPerf failed: " & Err.Description, vbExclamation, "CombinedPerf"
```

```
End Sub
```

```
Public Sub RefreshCombinedPerf()
```

```
On Error GoTo Fail
Application.ScreenUpdating = False
```

```
Dim wS As Worksheet
On Error Resume Next
Set wS = ThisWorkbook.Sheets(SH_OUT)
On Error GoTo Fail
If wS Is Nothing Then Call BuildCombinedPerf: Exit Sub
```

```
Call RefreshValues(wS)
```

```
Application.ScreenUpdating = True
Application.StatusBar = "CombinedPerf refreshed " & Format(Now, "hh:mm:ss")
Exit Sub
```

```
Fail:
```

```
Application.ScreenUpdating = True
MsgBox "RefreshCombinedPerf failed: " & Err.Description, vbExclamation, "CombinedPerf"
```

```
End Sub
```

```
Public Sub SyncRR4Data()
```

```
On Error GoTo Fail
Application.ScreenUpdating = False
```

```
Dim wS As Worksheet
On Error Resume Next
Set wS = ThisWorkbook.Sheets(SH_OUT)
On Error GoTo Fail
If wS Is Nothing Then Call BuildCombinedPerf: Exit Sub
```

```
Dim found As Boolean
Dim rr4 As Variant
rr4 = GetRR4Vals(found)
Call WriteRR4Cells(wS, rr4, found)
Call RecalcTotals(wS)
wS.Cells(R_UPDATED, C_LABEL).Value = "Updated: " & Format(Now, "yyyy/mm/dd hh:mm:ss")
```

```

Application.ScreenUpdating = True
If found Then
    Application.StatusBar = "RR4 synced OK  " & Format(Now, "hh:mm:ss")
Else
    MsgBox "RR4 workbook not found.  Open it first, or type values in orange cells.", _
        vbInformation, "RR4 not found"
End If
Exit Sub

Fail:
Application.ScreenUpdating = True
MsgBox "SyncRR4Data failed: " & Err.Description, vbExclamation, "CombinedPerf"
End Sub

```

```

' PRIVATE: ensure sheet

```

```

Private Function EnsureSheet() As Worksheet
    Dim wS As Worksheet
    On Error Resume Next
    Set wS = ThisWorkbook.Sheets(SH_OUT)
    On Error GoTo 0
    If wS Is Nothing Then
        Set wS = ThisWorkbook.Sheets.Add(After:=ThisWorkbook.Sheets(ThisWorkbook.Sheets.count))
        wS.Name = SH_OUT
    Else
        wS.Cells.Clear
        wS.Cells.ClearFormats
        On Error Resume Next
        wS.DrawingObjects.Delete
        On Error GoTo 0
    End If
    Set EnsureSheet = wS
End Function

```

```

' PRIVATE: draw static shell (labels, formatting, buttons)

```

```

Private Sub DrawShell(wS As Worksheet)
    wS.Cells.Interior.Color = CLR_BG_LIGHT
    wS.Cells.Font.Color = CLR_FG_SILVER
    ' DisplayGridlines belongs to Window, not Worksheet -- set via ActiveWindow after ws.Activate

    ' column widths
    wS.Columns(C_LABEL).ColumnWidth = 26
    wS.Columns(C_RR4).ColumnWidth = 18
    wS.Columns(C_RR5).ColumnWidth = 18
    wS.Columns(C_TOTAL).ColumnWidth = 18
    wS.Columns(5).ColumnWidth = 14

    ' -- Row 1: title --
    wS.Rows(R_TITLE).RowHeight = 28
    With wS.Range(wS.Cells(R_TITLE, C_LABEL), wS.Cells(R_TITLE, C_TOTAL))
        .Merge
        .Value = "RR4 + RR5  Joint Performance"
        .Font.Bold = True
        .Font.Size = 14
        .Font.Color = CLR_FG_WHITE
        .Interior.Color = CLR_BG_DARK
        .HorizontalAlignment = xlCenter
    End With

    ' -- Row 2: updated timestamp --
    wS.Rows(R_UPDATED).RowHeight = 15
    With wS.Range(wS.Cells(R_UPDATED, C_LABEL), wS.Cells(R_UPDATED, C_TOTAL))
        .Merge
        .Font.Size = 9
        .Font.Color = RGB(130, 130, 130)
        .Interior.Color = CLR_BG_DARK
        .HorizontalAlignment = xlRight
    End With

```

```

End With

' -- Row 3: spacer --
wS.Rows(3).RowHeight = 5

' -- Row 4: column headers --
wS.Rows(R_HDR).RowHeight = 20
Dim hLabels As Variant
hLabels = Array("Metric", "RR4 Stocks", "RR5 Derivatives", "Combined")
Dim hColors As Variant
hColors = Array(CLR_FG_WHITE, CLR_FG_GOLD, CLR_FG_CYAN, RGB(180, 255, 130))
Dim c As Long
For c = C_LABEL To C_TOTAL
    With wS.Cells(R_HDR, c)
        .Value = hLabels(c - 1)
        .Font.Bold = True
        .Font.Color = hColors(c - 1)
        .Interior.Color = CLR_SECTION
        .HorizontalAlignment = xlCenter
    End With
Next c

' -- Rows 5-7: Net Exposure / Unrealized PNL / Realized PNL --
Dim mLabels1 As Variant
mLabels1 = Array("Net Exposure (TWD)", "Unrealized PNL (TWD)", "Realized PNL (TWD)")
Dim r As Long, i As Long
For i = 0 To 2
    r = R_NEXPOS + i
    wS.Rows(r).RowHeight = 19
    With wS.Cells(r, C_LABEL)
        .Value = mLabels1(i)
        .Interior.Color = CLR_BG_MID
    End With
    wS.Cells(r, C_RR4).Interior.Color = CLR_BG_LIGHT
    wS.Cells(r, C_RR5).Interior.Color = CLR_BG_LIGHT
    With wS.Cells(r, C_TOTAL)
        .Interior.Color = CLR_BG_MID
        .Font.Bold = True
        .Font.Color = RGB(180, 255, 130)
    End With
Next i

' -- Row 8: Combined RL PNL (adjacent to Realized PNL, same type) --
wS.Rows(R_COMB_RL).RowHeight = 17
With wS.Cells(R_COMB_RL, C_LABEL)
    .Value = " Combined RL PNL"
    .Interior.Color = CLR_BG_MID
End With
wS.Cells(R_COMB_RL, C_RR4).Interior.Color = CLR_BG_LIGHT
wS.Cells(R_COMB_RL, C_RR5).Interior.Color = CLR_BG_LIGHT
With wS.Cells(R_COMB_RL, C_TOTAL)
    .Interior.Color = CLR_BG_MID
    .Font.Bold = True
    .Font.Color = RGB(180, 255, 130)
End With

' -- Rows 9-11: Total PNL / Realized Return % / Positions --
Dim mLabels2 As Variant
mLabels2 = Array("Total PNL (TWD)", "Realized Return %", "Positions")
Dim mRows2 As Variant
mRows2 = Array(R_TOT_PNL, R_UNRL_PCT, R_POSCOUNT)
For i = 0 To 2
    r = mRows2(i)
    wS.Rows(r).RowHeight = 19
    With wS.Cells(r, C_LABEL)
        .Value = mLabels2(i)
        .Interior.Color = CLR_BG_MID
    End With
    wS.Cells(r, C_RR4).Interior.Color = CLR_BG_LIGHT
    wS.Cells(r, C_RR5).Interior.Color = CLR_BG_LIGHT
    With wS.Cells(r, C_TOTAL)

```



```

        .Interior.Color = CLR_BG_MID
        .Font.Bold = True
        .Font.Color = RGB(180, 255, 130)
    End With
Next i

Call PutSep(wS, R_SEP1)
Call PutSecHdr(wS, R_D_HDR, "-- RR5 Derivatives Detail --")

' -- Rows 13-16: derivatives-only metrics --
Dim dLabels As Variant
dLabels = Array("Net Delta", "Net Theta (TWD/day)", "Net Vega (TWD)", "Margin Requirement (TWD)")
For i = 0 To 3
    r = R_DELTA + i
    wS.Rows(r).RowHeight = 19
    With wS.Cells(r, C_LABEL)
        .Value = dLabels(i)
        .Interior.Color = CLR_BG_MID
    End With
    wS.Cells(r, C_RR4).Interior.Color = RGB(25, 25, 25) ' n/a for stocks
    wS.Cells(r, C_RR5).Interior.Color = CLR_BG_LIGHT
    wS.Cells(r, C_TOTAL).Interior.Color = CLR_BG_LIGHT
Next i

Call PutSep(wS, R_SEP2)
Call PutSecHdr(wS, R_BK_HDR, "-- RR5 Breakdown by Type --")

' -- Row 19: breakdown headers --
wS.Rows(R_BK_COL).RowHeight = 17
Dim bkH As Variant
bkH = Array("Type", "Net Exposure (TWD)", "Unrealized PNL (TWD)", "Net Delta")
For c = C_LABEL To C_TOTAL
    With wS.Cells(R_BK_COL, c)
        .Value = bkH(c - 1)
        .Font.Bold = True
        .Font.Color = RGB(170, 170, 170)
        .Interior.Color = RGB(38, 38, 38)
        .HorizontalAlignment = xlCenter
    End With
Next c

' -- Rows 20-22: breakdown rows --
Dim bkLabels As Variant
bkLabels = Array("Futures", "Options", "Warrants")
For i = 0 To 2
    r = R_FUT + i
    wS.Rows(r).RowHeight = 19
    wS.Cells(r, C_LABEL).Value = bkLabels(i)
    wS.Cells(r, C_LABEL).Interior.Color = CLR_BG_MID
    wS.Cells(r, C_RR4).Interior.Color = CLR_BG_LIGHT
    wS.Cells(r, C_RR5).Interior.Color = CLR_BG_LIGHT
    wS.Cells(r, C_TOTAL).Interior.Color = CLR_BG_LIGHT
Next i

Call PutSep(wS, R_SEP3)

' -- Row 24: note --
wS.Rows(R_NOTE).RowHeight = 28
With wS.Range(wS.Cells(R_NOTE, C_LABEL), wS.Cells(R_NOTE, C_TOTAL))
    .Merge
    .Value = "Note: Orange cells = manual input. Or open RR4 workbook first, then click [Sync RR4]. " & _
        "Realized PNL reads from Realized sheet column I."
    .Font.Size = 8
    .Font.Color = RGB(130, 130, 130)
    .WrapText = True
End With

' -- Buttons (column E area) --
Dim btnTop As Double: btnTop = wS.Rows(R_TITLE).Top + 2
Dim btnH As Double: btnH = wS.Rows(R_TITLE).Height + wS.Rows(R_UPDATED).Height - 4
Dim btnL As Double: btnL = wS.Cells(R_TITLE, 5).Left + 4

```

```

Dim btn As Object
Set btn = wS.Buttons.Add(btnL, btnTop, 88, btnH)
btn.Caption = "Full Rebuild": btn.OnAction = "BuildCombinedPerf"

Set btn = wS.Buttons.Add(btnL + 92, btnTop, 88, btnH)
btn.Caption = "Refresh": btn.OnAction = "RefreshCombinedPerf"

Set btn = wS.Buttons.Add(btnL + 184, btnTop, 88, btnH)
btn.Caption = "Sync RR4": btn.OnAction = "SyncRR4Data"

Set btn = wS.Buttons.Add(btnL + 276, btnTop, 88, btnH)
btn.Caption = "Pie Charts": btn.OnAction = "BuildHoldingsPieCharts"

Set btn = wS.Buttons.Add(btnL + 368, btnTop, 88, btnH)
btn.Caption = "History Log": btn.OnAction = "BuildCombinedHistoryLog"

Set btn = wS.Buttons.Add(btnL + 460, btnTop, 100, btnH)
btn.Caption = "Import RR4 Hist": btn.OnAction = "ImportRR4History"

Set btn = wS.Buttons.Add(btnL + 564, btnTop, 88, btnH)
btn.Caption = "Log Chart": btn.OnAction = "BuildCombinedHistoryLog"

' -- Number formats --
Call ApplyFormats(wS)

' -- Borders --
Call PutBorder(wS, R_HDR, R_POSCOUNT)
Call PutBorder(wS, R_D_HDR, R_MARGIN)
Call PutBorder(wS, R_BK_COL, R_WRT)
End Sub

```

```

' =====
' PRIVATE: refresh all values
' =====
Private Sub RefreshValues(wS As Worksheet)
' Force a full recalc BEFORE reading RR5. RefreshValues can now run from
' RR5_Core.RebuildPositionsFromTransactions while a transaction is still
' being saved -- at that point Application.Calculation is pinned to
' xlCalculationManual by ShowTransactionForm for the whole modal session,
' so RR5's summary formulas (B1/F2/H2/N15/Q27/Q39...) may not yet reflect
' the position data that was just migrated in. Without this, CombinedPerf/
' CPLog capture a stale (pre-transaction) snapshot even though RR5 itself
' reads correctly once Excel later recalculates automatically.
Application.Calculate

Dim rr5 As Variant: rr5 = GetRR5Vals()
Call WriteRR5Cells(wS, rr5)

Dim found As Boolean
Dim rr4 As Variant: rr4 = GetRR4Vals(found)
Call WriteRR4Cells(wS, rr4, found)

Call RecalcTotals(wS)
wS.Cells(R_UPDATED, C_LABEL).Value = "Updated: " & Format(Now, "yyyy/mm/dd hh:mm:ss")

Call AppendLog(wS)
End Sub

```

```

' =====
' PRIVATE: read RR5 values from this workbook
' =====
Private Function GetRR5Vals() As Variant
' idx: 0=NetExpos 1=UnrlPNL 2=R1PNL 3=UnrlPct 4=Positions
'       5=Delta 6=Theta 7=Vega 8=Margin
'       9=FutExpos 10=FutPNL 11=FutDelta
'       12=OptExpos 13=OptPNL 14=OptDelta
'       15=WrtExpos 16=WrtPNL 17=WrtDelta
Dim v(0 To 17) As Variant

```

```
Dim i As Long
For i = 0 To 17: v(i) = 0: Next i
```

```
Dim wsR As Worksheet
On Error Resume Next
Set wsR = ThisWorkbook.Sheets(SH_RR5)
On Error GoTo 0
If wsR Is Nothing Then GetRR5Vals = v: Exit Function
```

```
On Error Resume Next
v(0) = V2D(wsR.Range("B1").Value) ' Total MV
v(1) = V2D(wsR.Range("F2").Value) ' Unrealized PNL
v(2) = V2D(wsR.Range("H2").Value) ' Realized PNL (=SUM Realized!I)
v(3) = V2D(wsR.Range("D2").Value) ' Unrealized PNL %
v(4) = V2D(wsR.Range("P1").Value) ' Positions
v(5) = V2D(wsR.Range("L1").Value) ' Net Delta
v(6) = V2D(wsR.Range("J2").Value) ' Net Theta
v(7) = V2D(wsR.Range("L2").Value) ' Net Vega
v(8) = V2D(wsR.Range("N2").Value) ' Margin

v(9) = V2D(wsR.Range("N15").Value) ' Futures Net Expos
v(10) = V2D(wsR.Range("O15").Value) ' Futures Unrealized PNL
v(11) = V2D(wsR.Range("S15").Value) ' Futures Net Delta

v(12) = V2D(wsR.Range("Q27").Value) ' Options Net Expos
v(13) = V2D(wsR.Range("R27").Value) ' Options Unrealized PNL
v(14) = V2D(wsR.Range("Y27").Value) ' Options Net Delta

v(15) = V2D(wsR.Range("Q39").Value) ' Warrants Net Expos
v(16) = V2D(wsR.Range("R39").Value) ' Warrants Unrealized PNL
v(17) = V2D(wsR.Range("X39").Value) ' Warrants Net Delta (SUMPRODUCT)
On Error GoTo 0
```

```
GetRR5Vals = v
```

```
End Function
```

```
' =====
' PRIVATE: read RR4 values from open workbook
' =====
```

```
Private Function GetRR4Vals(ByRef found As Boolean) As Variant
```

```
    ' idx: 0=NetExpos 1=UnrlPNL 2=R1PNL 3=UnrlPct 4=Positions
    Dim v(0 To 4) As Variant
    Dim i As Long
    For i = 0 To 4: v(i) = 0: Next i
    found = False
```

```
Dim wb As Workbook
Set wb = FindRR4WB()
If wb Is Nothing Then GetRR4Vals = v: Exit Function
found = True
```

```
On Error Resume Next
' Read from RR4 Analysis sheet -- fixed cell layout confirmed by DiagnoseRR4:
```

```
' R5 col B = Total Return %
' R6 col B = Total Cost (cost basis)
' R7 col B = Unrealized PnL
' R8 col B = Realized PnL
' R9 col B = Total PnL
' R31+ col A = Ticker (individual positions; col C = shares numeric)
```

```
Dim wsA As Worksheet
Set wsA = wb.Sheets("Analysis")
```

```
If Not wsA Is Nothing Then
```

```
    Dim totalCost As Double: totalCost = V2D(wsA.Cells(6, 2).Value)
    v(1) = V2D(wsA.Cells(7, 2).Value) ' Unrealized PnL
    v(2) = V2D(wsA.Cells(8, 2).Value) ' Realized PnL
    v(0) = totalCost + v(1) ' Net Exposure = Cost + Unrealized
    v(3) = v(2) / 550000 ' Realized Return % = Realized PNL / 550k
```

```
    ' Count positions: find "Ticker" header in col A, count non-empty rows below
```

```
Dim r As Long, tk As String
```

```

    Dim tickerHdr As Long: tickerHdr = 0
    For r = 1 To 60
        If InStr(1, LCase(Trim(CStr(wsA.Cells(r, 1).Value))), "ticker") > 0 Then
            tickerHdr = r: Exit For
        End If
    Next r
    If tickerHdr = 0 Then tickerHdr = 26 ' fallback: tickers start below row 26
    v(4) = 0
    For r = tickerHdr + 1 To 300
        tk = Trim(CStr(wsA.Cells(r, 1).Value))
        If tk <> "" Then v(4) = v(4) + 1
    Next r
End If
On Error GoTo 0

GetRR4Vals = v
End Function

```

```

Private Function FindRR4WB() As Workbook
    Dim wb As Workbook
    For Each wb In Workbooks
        If wb.Name <> ThisWorkbook.Name Then
            If InStr(UCASE(wb.Name), "RR4") > 0 Then
                Set FindRR4WB = wb
                Exit Function
            End If
        End If
    Next wb
    Set FindRR4WB = Nothing
End Function

```

```

' PRIVATE: write RR5 cells

```

```

Private Sub WriteRR5Cells(ws As Worksheet, v As Variant)
    ws.Cells(R_NEXPOS, C_RR5).Value = v(0)
    ws.Cells(R_UNRL_PNL, C_RR5).Value = v(1)
    ws.Cells(R_RL_PNL, C_RR5).Value = v(2)
    ws.Cells(R_TOT_PNL, C_RR5).Value = v(1) + v(2)
    ws.Cells(R_UNRL_PCT, C_RR5).Value = v(3)
    ws.Cells(R_POSCOUNT, C_RR5).Value = v(4)

    ws.Cells(R_DELTA, C_RR5).Value = v(5)
    ws.Cells(R_THETA, C_RR5).Value = v(6)
    ws.Cells(R_VEGA, C_RR5).Value = v(7)
    ws.Cells(R_MARGIN, C_RR5).Value = v(8)

    ws.Cells(R_DELTA, C_TOTAL).Value = v(5)
    ws.Cells(R_THETA, C_TOTAL).Value = v(6)
    ws.Cells(R_VEGA, C_TOTAL).Value = v(7)
    ws.Cells(R_MARGIN, C_TOTAL).Value = v(8)

    ' Breakdown (use C_RR4 col as Exposure, C_RR5 as PNL, C_TOTAL as Delta)
    ws.Cells(R_FUT, C_RR4).Value = v(9): ws.Cells(R_FUT, C_RR5).Value = v(10): ws.Cells(R_FUT, C_TOTAL).Value = v(11)
    ws.Cells(R_OPT, C_RR4).Value = v(12): ws.Cells(R_OPT, C_RR5).Value = v(13): ws.Cells(R_OPT, C_TOTAL).Value = v(14)
    ws.Cells(R_WRT, C_RR4).Value = v(15): ws.Cells(R_WRT, C_RR5).Value = v(16): ws.Cells(R_WRT, C_TOTAL).Value = v(17)

    ColorPnl ws.Cells(R_UNRL_PNL, C_RR5), v(1)
    ColorPnl ws.Cells(R_RL_PNL, C_RR5), v(2)
    ColorPnl ws.Cells(R_TOT_PNL, C_RR5), v(1) + v(2)
    ColorPnl ws.Cells(R_UNRL_PCT, C_RR5), v(3)
    ColorPnl ws.Cells(R_THETA, C_RR5), v(6)
End Sub

```

```

' PRIVATE: write RR4 cells

```

```

Private Sub WriteRR4Cells(ws As Worksheet, v As Variant, found As Boolean)

```

```

Dim bgClr As Long
If found Then bgClr = CLR_BG_LIGHT Else bgClr = CLR_ORANGE_BG

Dim rows4 As Variant
rows4 = Array(R_NEXPOS, R_UNRL_PNL, R_RL_PNL, R_TOT_PNL, R_UNRL_PCT, R_POSCOUNT)
Dim i As Long
For i = 0 To 5
    With wS.Cells(rows4(i), C_RR4)
        .Interior.Color = bgClr
        If found Then
            Select Case i
                Case 0: .Value = v(0)
                Case 1: .Value = v(1)
                Case 2: .Value = v(2)
                Case 3: .Value = v(1) + v(2)
                Case 4: .Value = v(3)
                Case 5: .Value = v(4)
            End Select
        Else
            If .Value = 0 Or .Value = "" Then .Value = ""
            .Font.Color = RGB(255, 180, 80)
        End If
    End With
Next i

If found Then
    ColorPnl wS.Cells(R_UNRL_PNL, C_RR4), v(1)
    ColorPnl wS.Cells(R_RL_PNL, C_RR4), v(2)
    ColorPnl wS.Cells(R_TOT_PNL, C_RR4), v(1) + v(2)
    ColorPnl wS.Cells(R_UNRL_PCT, C_RR4), v(3)
End If
End Sub

```

```

' =====
' PRIVATE: recalculate totals column
' =====
Private Sub RecalcTotals(wS As Worksheet)
    Dim a As Double, b As Double

    ' Net Exposure
    a = SN(wS.Cells(R_NEXPOS, C_RR4)): b = SN(wS.Cells(R_NEXPOS, C_RR5))
    wS.Cells(R_NEXPOS, C_TOTAL).Value = a + b

    ' Unrealized PNL
    a = SN(wS.Cells(R_UNRL_PNL, C_RR4)): b = SN(wS.Cells(R_UNRL_PNL, C_RR5))
    wS.Cells(R_UNRL_PNL, C_TOTAL).Value = a + b
    ColorPnl wS.Cells(R_UNRL_PNL, C_TOTAL), a + b

    ' Realized PNL
    a = SN(wS.Cells(R_RL_PNL, C_RR4)): b = SN(wS.Cells(R_RL_PNL, C_RR5))
    wS.Cells(R_RL_PNL, C_TOTAL).Value = a + b
    ColorPnl wS.Cells(R_RL_PNL, C_TOTAL), a + b

    ' Total PNL
    a = SN(wS.Cells(R_TOT_PNL, C_RR4)): b = SN(wS.Cells(R_TOT_PNL, C_RR5))
    wS.Cells(R_TOT_PNL, C_TOTAL).Value = a + b
    ColorPnl wS.Cells(R_TOT_PNL, C_TOTAL), a + b

    ' Realized Return % (combined) = total Realized PNL / 550,000
    Dim r14 As Double: r14 = SN(wS.Cells(R_RL_PNL, C_RR4))
    Dim r15 As Double: r15 = SN(wS.Cells(R_RL_PNL, C_RR5))
    Dim combRet As Double: combRet = (r14 + r15) / 550000
    wS.Cells(R_UNRL_PCT, C_TOTAL).Value = combRet
    ColorPnl wS.Cells(R_UNRL_PCT, C_TOTAL), combRet

    ' Positions
    wS.Cells(R_POSCOUNT, C_TOTAL).Value = SN(wS.Cells(R_POSCOUNT, C_RR4)) + _
        SN(wS.Cells(R_POSCOUNT, C_RR5))

    ' Combined RL PNL (adjacent to Realized PNL)

```

```

Dim combRL As Double
combRL = SN(wS.Cells(R_RL_PNL, C_RR4)) + SN(wS.Cells(R_RL_PNL, C_RR5))
wS.Cells(R_COMB_RL, C_TOTAL).Value = combRL
ColorPnl wS.Cells(R_COMB_RL, C_TOTAL), combRL
End Sub

```

```

' PRIVATE: number formats

```

```

Private Sub ApplyFormats(wS As Worksheet)
Dim twdFmt As String: twdFmt = "#,##0;[Red]-#,##0"
Dim pctFmt As String: pctFmt = "0.00%;[Red]-0.00%"
Dim dltFmt As String: dltFmt = "#,##0.0000;[Red]-#,##0.0000"
Dim intFmt As String: intFmt = "0"

```

```

Dim r As Long
For r = R_NEXPOS To R_RL_PNL
    wS.Cells(r, C_RR4).NumberFormat = twdFmt
    wS.Cells(r, C_RR5).NumberFormat = twdFmt
    wS.Cells(r, C_TOTAL).NumberFormat = twdFmt
Next r
wS.Cells(R_COMB_RL, C_TOTAL).NumberFormat = twdFmt
wS.Cells(R_TOT_PNL, C_RR4).NumberFormat = twdFmt
wS.Cells(R_TOT_PNL, C_RR5).NumberFormat = twdFmt
wS.Cells(R_TOT_PNL, C_TOTAL).NumberFormat = twdFmt
wS.Cells(R_UNRL_PCT, C_RR4).NumberFormat = pctFmt
wS.Cells(R_UNRL_PCT, C_RR5).NumberFormat = pctFmt
wS.Cells(R_UNRL_PCT, C_TOTAL).NumberFormat = pctFmt
wS.Cells(R_POSCOUNT, C_RR4).NumberFormat = intFmt
wS.Cells(R_POSCOUNT, C_RR5).NumberFormat = intFmt
wS.Cells(R_POSCOUNT, C_TOTAL).NumberFormat = intFmt

```

```

wS.Cells(R_DELTA, C_RR5).NumberFormat = dltFmt
wS.Cells(R_DELTA, C_TOTAL).NumberFormat = dltFmt
wS.Cells(R_THETA, C_RR5).NumberFormat = twdFmt
wS.Cells(R_THETA, C_TOTAL).NumberFormat = twdFmt
wS.Cells(R_VEGA, C_RR5).NumberFormat = twdFmt
wS.Cells(R_VEGA, C_TOTAL).NumberFormat = twdFmt
wS.Cells(R_MARGIN, C_RR5).NumberFormat = twdFmt
wS.Cells(R_MARGIN, C_TOTAL).NumberFormat = twdFmt

```

```

Dim bkRows As Variant: bkRows = Array(R_FUT, R_OPT, R_WRT)
Dim i As Long
For i = 0 To 2
    wS.Cells(bkRows(i), C_RR4).NumberFormat = twdFmt
    wS.Cells(bkRows(i), C_RR5).NumberFormat = twdFmt
    wS.Cells(bkRows(i), C_TOTAL).NumberFormat = dltFmt
Next i
End Sub

```

```

' PRIVATE: helpers

```

```

Private Sub PutSep(wS As Worksheet, r As Long)
wS.Rows(r).RowHeight = 7
With wS.Range(wS.Cells(r, C_LABEL), wS.Cells(r, C_TOTAL))
    .Interior.Color = CLR_BG_DARK
End With
End Sub

```

```

Private Sub PutSecHdr(wS As Worksheet, r As Long, txt As String)
wS.Rows(r).RowHeight = 17
With wS.Range(wS.Cells(r, C_LABEL), wS.Cells(r, C_TOTAL))
    .Merge
    .Value = txt
    .Interior.Color = CLR_SECTION
    .Font.Color = RGB(160, 210, 255)
    .Font.Bold = True
End With

```

```

        .Font.Size = 10
        .HorizontalAlignment = xlCenter
    End With
End Sub

Private Sub PutBorder(ws As Worksheet, rStart As Long, rEnd As Long)
    With ws.Range(ws.Cells(rStart, C_LABEL), ws.Cells(rEnd, C_TOTAL))
        Dim sides As Variant
        sides = Array(xlEdgeLeft, xlEdgeTop, xlEdgeBottom, xlEdgeRight, _
            xlInsideVertical, xlInsideHorizontal)
        Dim s As Long
        For s = 0 To UBound(sides)
            On Error Resume Next
            .Borders(sides(s)).LineStyle = xlContinuous
            .Borders(sides(s)).Color = CLR_BORDER
            On Error GoTo 0
        Next s
    End With
End Sub

Private Sub ColorPnl(c As Range, ByVal v As Double)
    If v > 0 Then
        c.Font.Color = RGB(80, 220, 100)
    ElseIf v < 0 Then
        c.Font.Color = RGB(255, 80, 80)
    Else
        c.Font.Color = CLR_FG_SILVER
    End If
End Sub

Private Function V2D(v As Variant) As Double
    On Error Resume Next
    If IsNumeric(v) Then V2D = CDbl(v) Else V2D = 0
    On Error GoTo 0
End Function

Private Function SN(c As Range) As Double
    On Error Resume Next
    If IsNumeric(c.Value) Then SN = CDbl(c.Value) Else SN = 0
    On Error GoTo 0
End Function

' =====
' PUBLIC: open history log sheet + rebuild line chart
' =====

Public Sub BuildCombinedHistoryLog()
    Dim wl As Worksheet: Set wl = EnsureLogSheet()
    ' Ensure sheet base background is dark gray (not pure black)
    wl.Cells.Interior.Color = CLR_BG_DARK

    ' --- Remove rows before LOG_START ---
    Dim lr As Long: lr = wl.Cells(wl.Rows.count, 1).End(xlUp).row
    Application.ScreenUpdating = False
    Dim dr As Long
    Dim cellDt As Date
    For dr = lr To 3 Step -1
        If IsDate(wl.Cells(dr, 1).Value) Then
            cellDt = CDate(wl.Cells(dr, 1).Value)
            If cellDt < LOG_START Then wl.Rows(dr).Delete
        End If
    Next dr
    lr = wl.Cells(wl.Rows.count, 1).End(xlUp).row

    ' --- Deduplicate: for same-date rows keep only the LAST (most complete) row ---
    For dr = lr To 4 Step -1 ' scan bottom-up so deleting doesn't shift index
        If wl.Cells(dr, 1).Value = "" Then GoTo NextDR
        Dim dupR As Long: dupR = 0
        Dim sr As Long
        For sr = dr - 1 To 3 Step -1
            If Int(wl.Cells(sr, 1).Value) = Int(wl.Cells(dr, 1).Value) Then

```

```

        dupR = sr: Exit For
    End If
Next sr
If dupR > 0 Then wl.Rows(dupR).Delete ' delete the earlier duplicate
NextDR:
Next dr
Application.ScreenUpdating = True
lr = wl.Cells(wl.Rows.count, 1).End(xlUp).row ' refresh after dedup

' Re-stamp alternating gray/black on every data row (CLR_SECTION / CLR_BG_MID)
Dim rr As Long
Dim bRL As Double, bRet As Double, bComb As Double
For rr = 3 To lr
    wl.Rows(rr).Interior.Color = IIf(rr Mod 2 = 1, CLR_SECTION, CLR_BG_MID)
    ' Backfill col 5 (RR4 RL Ret%) -- derive from col 4 / 550k if empty
    bRL = V2D(wl.Cells(rr, 4).Value)
    If bRL <> 0 And wl.Cells(rr, 5).Value = "" Then
        bRet = bRL / 550000
        wl.Cells(rr, 5).Value = bRet
        wl.Cells(rr, 5).NumberFormat = "0.00%"
        ColorPnl wl.Cells(rr, 5), bRet
    End If
    ' Backfill col 11 (Combined RL PnL) -- col4 + col8 if empty
    If bRL <> 0 And wl.Cells(rr, 11).Value = "" Then
        bComb = bRL + V2D(wl.Cells(rr, 8).Value)
        wl.Cells(rr, 11).Value = bComb
        wl.Cells(rr, 11).NumberFormat = "#,##0;[Red](#,##0)"
        ColorPnl wl.Cells(rr, 11), bComb
    End If
    ' Backfill col 12 (Daily Ret Chg%) -- (this row's Combined PNL - prev row's) / prev row's lCombined MVI
    If rr > 3 And wl.Cells(rr, 12).Value = "" Then
        Dim bPrevMV As Double, bPrevPNL As Double, bDRet As Double
        bPrevMV = V2D(wl.Cells(rr - 1, 9).Value)
        bPrevPNL = V2D(wl.Cells(rr - 1, 10).Value)
        If bPrevMV <> 0 Then
            bDRet = (V2D(wl.Cells(rr, 10).Value) - bPrevPNL) / Abs(bPrevMV)
            wl.Cells(rr, 12).Value = bDRet
            wl.Cells(rr, 12).NumberFormat = "0.00%;[Red](0.00%)"
            ColorPnl wl.Cells(rr, 12), bDRet
        End If
    End If
Next rr
Call BuildRLChart(wl)
Call BuildRiskMetrics(wl, lr)
wl.Activate
On Error Resume Next
ActiveWindow.DisplayGridlines = False
On Error GoTo 0
Application.StatusBar = "CPLog ready " & Format(Now, "hh:mm:ss")
End Sub

```

```

' =====
' PRIVATE: ensure CPLog sheet -- 14-col schema, colored headers
' Row 1 = title Row 2 = headers Row 3+ = data
' =====

```

```

Private Function EnsureLogSheet() As Worksheet
    Dim wl As Worksheet
    Dim shIsNew As Boolean: shIsNew = False
    On Error Resume Next: Set wl = ThisWorkbook.Sheets(SH_LOG): On Error GoTo 0

    If wl Is Nothing Then
        shIsNew = True
        Dim ref As Worksheet
        On Error Resume Next: Set ref = ThisWorkbook.Sheets(SH_OUT): On Error GoTo 0
        If ref Is Nothing Then
            Set wl = ThisWorkbook.Sheets.Add(After:=ThisWorkbook.Sheets(ThisWorkbook.Sheets.count))
        Else
            Set wl = ThisWorkbook.Sheets.Add(After:=ref)
        End If
        wl.Name = SH_LOG
    End If

```



```

    wl.Cells.Interior.Color = CLR_BG_DARK
    wl.Cells.Font.Color = CLR_FG_SILVER
    wl.Cells.Font.Size = 9
End If

' --- ALWAYS: refresh title (row 1) ---
wl.Rows(1).RowHeight = 24
On Error Resume Next: wl.Range(wl.Cells(1, 1), wl.Cells(1, 14)).UnMerge: On Error GoTo 0
With wl.Range(wl.Cells(1, 1), wl.Cells(1, 14))
    .Merge
    .Value = "RR4 + RR5  Combined History Log"
    .Font.Bold = True
    .Font.Size = 13
    .Font.Color = CLR_FG_WHITE
    .Interior.Color = CLR_BG_MID
    .HorizontalAlignment = xlCenter
    .VerticalAlignment = xlCenter
End With

' --- ALWAYS: refresh headers (row 2) ---
wl.Rows(2).RowHeight = 18
Dim hdr As Variant
hdr = Array("Date", "RR4 Net MV", "RR4 Unrl PNL", "RR4 RL PNL", "RR4 RL Ret%", _
    "RR5 Total MV", "RR5 Unrl PNL", "RR5 RL PNL", _
    "Combined MV", "Combined PNL", _
    "Combined RL PnL", "Daily Ret Chg%", "TWII Price", "Notes")
Dim hclr As Variant
hclr = Array(CLR_FG_WHITE, _
    CLR_GRP_RR4, CLR_GRP_RR4, CLR_GRP_RR4, CLR_GRP_RR4, _
    CLR_GRP_RR5, CLR_GRP_RR5, CLR_GRP_RR5, _
    CLR_GRP_COMB, CLR_GRP_COMB, _
    CLR_GRP_COMB, CLR_GRP_COMB, CLR_FG_SILVER, CLR_FG_SILVER)
Dim c As Long
For c = 1 To 14
    With wl.Cells(2, c)
        .Value = hdr(c - 1)
        .Font.Bold = True
        .Font.Color = hclr(c - 1)
        .Interior.Color = CLR_BG_MID
        .HorizontalAlignment = xlCenter
        .VerticalAlignment = xlCenter
    End With
Next c

' --- ALWAYS: column widths ---
wl.Columns(1).ColumnWidth = 11
wl.Columns(2).ColumnWidth = 12
wl.Columns(3).ColumnWidth = 12
wl.Columns(4).ColumnWidth = 12
wl.Columns(5).ColumnWidth = 11
wl.Columns(6).ColumnWidth = 12
wl.Columns(7).ColumnWidth = 12
wl.Columns(8).ColumnWidth = 12
wl.Columns(9).ColumnWidth = 12
wl.Columns(10).ColumnWidth = 13
wl.Columns(11).ColumnWidth = 13
wl.Columns(12).ColumnWidth = 11
wl.Columns(13).ColumnWidth = 12
wl.Columns(14).ColumnWidth = 16

' --- ONLY new sheets: seed row at LOG_START ---
If shIsNew Then
    Dim twdFmt0 As String: twdFmt0 = "#,##0;[Red](#,##0)"
    wl.Cells(3, 1).Value = LOG_START
    wl.Cells(3, 1).NumberFormat = "yyyy/m/d"
    wl.Cells(3, 2).Value = 0: wl.Cells(3, 2).NumberFormat = twdFmt0
    wl.Cells(3, 3).Value = 0: wl.Cells(3, 3).NumberFormat = twdFmt0
    wl.Cells(3, 4).Value = 0: wl.Cells(3, 4).NumberFormat = twdFmt0
    wl.Cells(3, 5).Value = 0: wl.Cells(3, 5).NumberFormat = "0.00%"
    wl.Cells(3, 9).Value = 0: wl.Cells(3, 9).NumberFormat = twdFmt0
    wl.Cells(3, 10).Value = 0: wl.Cells(3, 10).NumberFormat = twdFmt0

```

```

        wl.Cells(3, 11).Value = 0: wl.Cells(3, 11).NumberFormat = twdFmt0
        wl.Rows(3).Interior.Color = CLR_SECTION
    End If

```

```

    Set EnsureLogSheet = wl
End Function

```

```

' PRIVATE: append one log row -- reads live values from CombinedPerf ws

```

```

Private Sub AppendLog(ws As Worksheet)

```

```

    On Error Resume Next

```

```

    Dim wl As Worksheet: Set wl = EnsureLogSheet()

```

```

    If wl Is Nothing Then Exit Sub

```

```

' --- find existing row for today or append new ---

```

```

Dim nr As Long: nr = 0

```

```

Dim lastR As Long: lastR = wl.Cells(wl.Rows.count, 1).End(xlUp).row

```

```

Dim chkR As Long

```

```

For chkR = 3 To lastR

```

```

    If Int(wl.Cells(chkR, 1).Value) = Int(Date) Then

```

```

        nr = chkR

```

```

        Exit For

```

```

    End If

```

```

Next chkR

```

```

If nr = 0 Then

```

```

    nr = lastR + 1

```

```

    If nr < 3 Then nr = 3

```

```

End If

```

```

' --- read from CombinedPerf sheet ---

```

```

Dim rr4MV As Double: rr4MV = SN(ws.Cells(R_NEXPOS, C_RR4))

```

```

Dim rr4Unrl As Double: rr4Unrl = SN(ws.Cells(R_UNRL_PNL, C_RR4))

```

```

Dim rr4RL As Double: rr4RL = SN(ws.Cells(R_RL_PNL, C_RR4))

```

```

Dim rr4Ret As Double: rr4Ret = SN(ws.Cells(R_UNRL_PCT, C_RR4))

```

```

Dim rr5MV As Double: rr5MV = SN(ws.Cells(R_NEXPOS, C_RR5))

```

```

Dim rr5Unrl As Double: rr5Unrl = SN(ws.Cells(R_UNRL_PNL, C_RR5))

```

```

Dim rr5RL As Double: rr5RL = SN(ws.Cells(R_RL_PNL, C_RR5))

```

```

Dim combMV As Double: combMV = SN(ws.Cells(R_NEXPOS, C_TOTAL))

```

```

Dim combPNL As Double: combPNL = SN(ws.Cells(R_TOT_PNL, C_TOTAL))

```

```

Dim combRL As Double: combRL = rr4RL + rr5RL

```

```

' --- read from RR5 sheet (TWII) ---

```

```

Dim wsR As Worksheet

```

```

Set wsR = ThisWorkbook.Sheets(SH_RR5)

```

```

Dim twiiPx As Double: twiiPx = 0

```

```

If Not wsR Is Nothing Then

```

```

    twiiPx = V2D(wsR.Range(RR5_TWII_CELL).Value)

```

```

End If

```

```

Dim twdFmt As String: twdFmt = "#,##0;[Red](#,##0)"

```

```

' col 1: Date

```

```

wl.Cells(nr, 1).Value = Date

```

```

wl.Cells(nr, 1).NumberFormat = "yyyy/m/d"

```

```

wl.Cells(nr, 1).Interior.Color = CLR_BG_MID

```

```

' cols 2-5: RR4 (skip if RR4 not connected)

```

```

If rr4MV <> 0 Then

```

```

    wl.Cells(nr, 2).Value = rr4MV:          wl.Cells(nr, 2).NumberFormat = twdFmt

```

```

    wl.Cells(nr, 3).Value = rr4Unrl: wl.Cells(nr, 3).NumberFormat = twdFmt

```

```

    wl.Cells(nr, 4).Value = rr4RL:          wl.Cells(nr, 4).NumberFormat = twdFmt

```

```

    wl.Cells(nr, 5).Value = rr4Ret:          wl.Cells(nr, 5).NumberFormat = "0.00%"

```

```

    ColorPnl wl.Cells(nr, 3), rr4Unrl

```

```

    ColorPnl wl.Cells(nr, 4), rr4RL

```

```

    ColorPnl wl.Cells(nr, 5), rr4Ret

```

```

End If

```

```

' cols 6-8: RR5 (always write -- RR5 sheet lives in this workbook, unlike

```

```

' RR4 which may genuinely not be open. Net exposure can legitimately be 0
' while Unrl/RL PnL are not, so gating on rr5MV<>0 dropped real data.)
wl.Cells(nr, 6).Value = rr5MV: wl.Cells(nr, 6).NumberFormat = twdFmt
wl.Cells(nr, 7).Value = rr5Unrl: wl.Cells(nr, 7).NumberFormat = twdFmt
wl.Cells(nr, 8).Value = rr5RL: wl.Cells(nr, 8).NumberFormat = twdFmt
ColorPnl wl.Cells(nr, 7), rr5Unrl
ColorPnl wl.Cells(nr, 8), rr5RL

' cols 9-10: Combined (always)
wl.Cells(nr, 9).Value = combMV: wl.Cells(nr, 9).NumberFormat = twdFmt
wl.Cells(nr, 10).Value = combPNL: wl.Cells(nr, 10).NumberFormat = twdFmt
ColorPnl wl.Cells(nr, 10), combPNL

' col 11: Combined RL PnL
wl.Cells(nr, 11).Value = combRL: wl.Cells(nr, 11).NumberFormat = twdFmt: ColorPnl wl.Cells(nr, 11), combRL

' col 12: Daily Ret Chg% = (today's Combined PNL - prev row's) / prev row's |Combined MV|
If nr > 3 Then
    Dim prevMV As Double, prevPNL As Double, dRet As Double
    prevMV = V2D(wl.Cells(nr - 1, 9).Value)
    prevPNL = V2D(wl.Cells(nr - 1, 10).Value)
    If prevMV <> 0 Then
        dRet = (combPNL - prevPNL) / Abs(prevMV)
        wl.Cells(nr, 12).Value = dRet
        wl.Cells(nr, 12).NumberFormat = "0.00%;[Red](0.00%)"
        ColorPnl wl.Cells(nr, 12), dRet
    End If
End If

' col 13: TWII Price
If twiiPx <> 0 Then
    wl.Cells(nr, 13).Value = twiiPx
    wl.Cells(nr, 13).NumberFormat = "#,##0.00"
End If

' alternating gray / dark row colors
wl.Rows(nr).Interior.Color = IIf(nr Mod 2 = 1, CLR_SECTION, CLR_BG_MID)
On Error GoTo 0
End Sub

```

```

' =====
' PUBLIC: seed one historical log row (call from VBA Immediate window)
' e.g.: SeedLogRow #1/1/2026#, 800000, -20000, 350000, 0.636, 0,0,0, 0,0,0
' =====

```

```

Public Sub SeedLogRow(dt As Date, _
    r4MV As Double, r4Up As Double, r4RP As Double, r4Ret As Double, _
    r5MV As Double, r5Up As Double, r5RP As Double, _
    combRL As Double, ivA As Double, twii As Double)
    On Error Resume Next
    Dim wl As Worksheet: Set wl = EnsureLogSheet()
    If wl Is Nothing Then Exit Sub

    Dim nr As Long
    nr = wl.Cells(wl.Rows.count, 1).End(xlUp).row + 1
    If nr < 3 Then nr = 3

    Dim twdFmt As String: twdFmt = "#,##0;[Red](#,##0)"

    wl.Cells(nr, 1).Value = dt
    wl.Cells(nr, 1).NumberFormat = "yyyy/m/d"
    wl.Cells(nr, 1).Interior.Color = CLR_BG_MID

    If r4MV <> 0 Then
        wl.Cells(nr, 2).Value = r4MV: wl.Cells(nr, 2).NumberFormat = twdFmt
        wl.Cells(nr, 3).Value = r4Up: wl.Cells(nr, 3).NumberFormat = twdFmt: ColorPnl wl.Cells(nr, 3), r4Up
        wl.Cells(nr, 4).Value = r4RP: wl.Cells(nr, 4).NumberFormat = twdFmt: ColorPnl wl.Cells(nr, 4), r4RP
        wl.Cells(nr, 5).Value = r4Ret: wl.Cells(nr, 5).NumberFormat = "0.00%": ColorPnl wl.Cells(nr, 5), r4Ret
    End If

    If r5MV <> 0 Then
        wl.Cells(nr, 6).Value = r5MV: wl.Cells(nr, 6).NumberFormat = twdFmt
        wl.Cells(nr, 7).Value = r5Up: wl.Cells(nr, 7).NumberFormat = twdFmt: ColorPnl wl.Cells(nr, 7), r5Up
    End If

```

```

        wl.Cells(nr, 8).Value = r5RP: wl.Cells(nr, 8).NumberFormat = twdFmt: ColorPnl wl.Cells(nr, 8), r5RP
    End If

    Dim combMV As Double: combMV = r4MV + r5MV
    Dim combPNL As Double: combPNL = (r4Up + r4RP) + (r5Up + r5RP)
    wl.Cells(nr, 9).Value = combMV: wl.Cells(nr, 9).NumberFormat = twdFmt
    wl.Cells(nr, 10).Value = combPNL: wl.Cells(nr, 10).NumberFormat = twdFmt: ColorPnl wl.Cells(nr, 10), combPNL

    If combRL <> 0 Then wl.Cells(nr, 11).Value = combRL: wl.Cells(nr, 11).NumberFormat = "#,##0;[Red](#,##0)": ColorPnl wl.Cells(nr, 11), combRL
    If ivA <> 0 Then wl.Cells(nr, 12).Value = ivA: wl.Cells(nr, 12).NumberFormat = "0.00%"
    If twii <> 0 Then wl.Cells(nr, 13).Value = twii: wl.Cells(nr, 13).NumberFormat = "#,##0.00"

    wl.Rows(nr).Interior.Color = IIf(nr Mod 2 = 1, CLR_SECTION, CLR_BG_MID)
    On Error GoTo 0
End Sub

' =====
' PUBLIC: import RR4 HistoryLog -> CPLog (run once, RR4 wb must be open)
' RR4 HistoryLog columns: A=DateTime B=NetMV C=TotalPNL F=TWII G=RealizedPNL
' Derives: Unrl PNL = C - G | Combined MV = B | Combined PNL = C
' Skips rows already having RR4 data. Sorts CPLog by date when done.
' =====

Public Sub ImportRR4History()
    Dim wl As Worksheet: Set wl = EnsureLogSheet()
    If wl Is Nothing Then Exit Sub

    Dim r4wb As Workbook: Set r4wb = FindRR4WB()
    If r4wb Is Nothing Then
        MsgBox "RR4 workbook not open. Open it first.", vbExclamation, "ImportRR4History"
        Exit Sub
    End If

    Dim wsH As Worksheet
    On Error Resume Next: Set wsH = r4wb.Sheets("HistoryLog"): On Error GoTo 0
    If wsH Is Nothing Then
        MsgBox "HistoryLog sheet not found in RR4 workbook.", vbExclamation, "ImportRR4History"
        Exit Sub
    End If

    Dim hLast As Long: hLast = wsH.Cells(wsH.Rows.count, "B").End(xlUp).row
    If hLast < 2 Then MsgBox "RR4 HistoryLog is empty.", vbInformation, "ImportRR4History": Exit Sub

    Dim cpLast As Long: cpLast = wl.Cells(wl.Rows.count, 1).End(xlUp).row
    Dim maxRows As Long: maxRows = hLast + cpLast + 10

    ' existing date -> row map
    Dim exDate() As Date: ReDim exDate(1 To maxRows)
    Dim exRow() As Long: ReDim exRow(1 To maxRows)
    Dim exCnt As Long: exCnt = 0
    Dim cr As Long
    For cr = 3 To cpLast
        Dim cdv As Variant: cdv = wl.Cells(cr, 1).Value
        If IsDate(cdv) Then
            exCnt = exCnt + 1
            exDate(exCnt) = Int(CDate(cdv))
            exRow(exCnt) = cr
        End If
    Next cr

    Dim twdFmt As String: twdFmt = "#,##0;[Red](#,##0)"
    Dim added As Long: added = 0

    Dim hr As Long
    For hr = 2 To hLast
        ' --- read date ---
        Dim rawDt As Variant: rawDt = wsH.Cells(hr, "A").Value
        If Not IsDate(rawDt) Then GoTo SkipHR
        Dim dt As Date: dt = Int(CDate(rawDt))
        If dt < LOG_START Then GoTo SkipHR

```

```

' --- read RR4 values ---
Dim r4MV As Double: r4MV = V2D(wsH.Cells(hr, "B").Value)
Dim r4Tot As Double: r4Tot = V2D(wsH.Cells(hr, "C").Value)
Dim r4RL As Double: r4RL = V2D(wsH.Cells(hr, "G").Value)
Dim r4Unrl As Double: r4Unrl = r4Tot - r4RL
Dim twii As Double: twii = V2D(wsH.Cells(hr, "F").Value)

' --- find or create CLog row ---
Dim wr As Long: wr = 0
Dim di As Long
For di = 1 To exCnt
    If exDate(di) = dt Then wr = exRow(di): Exit For
Next di

If wr > 0 Then
    ' Always update col 11 (Combined RL PnL) even for existing rows
    wl.Cells(wr, 11).Value = r4RL: wl.Cells(wr, 11).NumberFormat = twdFmt: ColorPnl wl.Cells(wr, 11), r4RL
    ' Always update col 5 (RR4 RL Ret% = Realized PNL / 550k)
    wl.Cells(wr, 5).Value = r4RL / 550000: wl.Cells(wr, 5).NumberFormat = "0.00%": ColorPnl wl.Cells(wr, 5), r4RL / 550000
    ' Skip other fields if row already has RR4 data
    If V2D(wl.Cells(wr, 2).Value) <> 0 Then GoTo SkipHR
Else
    ' new row -- append
    cpLast = cpLast + 1
    wr = cpLast
    wl.Rows(wr).Interior.Color = IIf(wr Mod 2 = 1, CLR_SECTION, CLR_BG_MID)
    exCnt = exCnt + 1
    exDate(exCnt) = dt
    exRow(exCnt) = wr
    added = added + 1
End If

' --- write to CLog ---
wl.Cells(wr, 1).Value = dt
wl.Cells(wr, 1).NumberFormat = "yyyy/m/d"
wl.Cells(wr, 1).Interior.Color = CLR_BG_MID

wl.Cells(wr, 2).Value = r4MV: wl.Cells(wr, 2).NumberFormat = twdFmt
wl.Cells(wr, 3).Value = r4Unrl: wl.Cells(wr, 3).NumberFormat = twdFmt: ColorPnl wl.Cells(wr, 3), r4Unrl
wl.Cells(wr, 4).Value = r4RL: wl.Cells(wr, 4).NumberFormat = twdFmt: ColorPnl wl.Cells(wr, 4), r4RL
' col 5: RR4 RL Ret% = Realized PNL / 550k capital base
wl.Cells(wr, 5).Value = r4RL / 550000: wl.Cells(wr, 5).NumberFormat = "0.00%": ColorPnl wl.Cells(wr, 5), r4RL / 550000
' col 11: Combined RL PnL = RR4 RL (RR5 not yet running for historical data)
wl.Cells(wr, 11).Value = r4RL: wl.Cells(wr, 11).NumberFormat = twdFmt: ColorPnl wl.Cells(wr, 11), r4RL

' Combined (RR4-only for history rows where RR5 not yet running)
wl.Cells(wr, 9).Value = r4MV: wl.Cells(wr, 9).NumberFormat = twdFmt
' Combined PNL only if RR5 RL col is blank
If V2D(wl.Cells(wr, 8).Value) = 0 Then
    wl.Cells(wr, 10).Value = r4Tot: wl.Cells(wr, 10).NumberFormat = twdFmt: ColorPnl wl.Cells(wr, 10), r4Tot
End If

If twii <> 0 Then
    wl.Cells(wr, 13).Value = twii
    wl.Cells(wr, 13).NumberFormat = "#,##0.00"
End If

```

SkipHR:

Next hr

```

' sort CLog data rows by date ascending
Dim cpFinal As Long: cpFinal = wl.Cells(wl.Rows.count, 1).End(xlUp).row
If cpFinal > 3 Then
    wl.Range(wl.Cells(3, 1), wl.Cells(cpFinal, 14)).Sort _
        Key1:=wl.Columns(1), Order1:=xlAscending, Header:=xlNo
End If
' Re-stamp alternating row colors after sort scrambles them
Dim srr As Long
For srr = 3 To cpFinal
    wl.Rows(srr).Interior.Color = IIf(srr Mod 2 = 1, CLR_SECTION, CLR_BG_MID)

```

Next srr

MsgBox added & " RR4 history rows added. Press [History Log] to rebuild chart.", _
vbInformation, "ImportRR4History"

End Sub

PRIVATE: build / refresh RL PNL line chart on CLog sheet

Private Sub BuildRLChart(wl As Worksheet)

' --- 1. Delete any existing CLogLine_x charts ---

On Error Resume Next

Dim co As ChartObject

For Each co In wl.ChartObjects

If Left(co.Name, 10) = "CLogLine_" Then co.Delete

Next co

On Error GoTo 0

' --- 2. Determine data extent (col A = dates, rows 3..lastRow) ---

Dim lastRow As Long

lastRow = wl.Cells(wl.Rows.Count, 1).End(xlUp).row

If lastRow < 3 Then Exit Sub

Dim xRng As Range

Set xRng = wl.Range(wl.Cells(3, 1), wl.Cells(lastRow, 1))

' X axis floor = first data date (Excel serial); prevents axis drifting to 2025

Dim xMin As Double

If IsDate(wl.Cells(3, 1).Value) Then

xMin = CDBl(CDate(wl.Cells(3, 1).Value))

Else

xMin = CDBl(CDate(LOG_START))

End If

' --- 3. Series definition: source col, title, line color, Y number format ---

Dim sCols(0 To 7) As Long

Dim sNams(0 To 7) As String

Dim sClrs(0 To 7) As Long

Dim sFmts(0 To 7) As String

sCols(0) = 3: sNams(0) = "RR4 Unrl PNL": sClrs(0) = CLR_GRP_RR4: sFmts(0) = "#,##0"

sCols(1) = 4: sNams(1) = "RR4 RL PNL": sClrs(1) = CLR_GRP_RR4: sFmts(1) = "#,##0"

sCols(2) = 5: sNams(2) = "RR4 RL Ret%": sClrs(2) = CLR_GRP_RR4: sFmts(2) = "0.00%"

sCols(3) = 6: sNams(3) = "RR5 Total MV": sClrs(3) = CLR_GRP_RR5: sFmts(3) = "#,##0"

sCols(4) = 7: sNams(4) = "RR5 Unrl PNL": sClrs(4) = CLR_GRP_RR5: sFmts(4) = "#,##0"

sCols(5) = 8: sNams(5) = "RR5 RL PNL": sClrs(5) = CLR_GRP_RR5: sFmts(5) = "#,##0"

sCols(6) = 9: sNams(6) = "Combined MV": sClrs(6) = CLR_GRP_COMB: sFmts(6) = "#,##0"

sCols(7) = 10: sNams(7) = "Combined PNL": sClrs(7) = CLR_GRP_COMB: sFmts(7) = "#,##0"

Dim bgClr As Long: bgClr = RGB(27, 27, 27) ' #1B1B1B chart background

Dim gridClr As Long: gridClr = RGB(55, 55, 55) ' subtle horizontal gridlines

' --- 4. Build one chart per series (single column, stacked vertically) ---

Dim si As Long

For si = 0 To 7

Dim sRow As Long: sRow = 3 + si * 29 ' row 3 start; 28 rows chart + 1 row gap

Dim sCol As Long: sCol = 15 ' col 0

Dim cLeft As Double: cLeft = wl.Cells(sRow, sCol).Left

Dim cTop As Double: cTop = wl.Cells(sRow, 1).Top

Dim cWidth As Double: cWidth = wl.Cells(sRow, sCol + 14).Left - cLeft

Dim cHeight As Double: cHeight = wl.Cells(sRow + 28, 1).Top - cTop

Dim co2 As ChartObject

Set co2 = wl.ChartObjects.Add(cLeft, cTop, cWidth, cHeight)

co2.Name = "CLogLine_" & si

With co2.Chart

' Line chart: category axis can be a TRUE time scale (calendar months)

.ChartType = 4 ' xlLine

```

' Wipe any auto-created series, then add exactly one
Do While .SeriesCollection.count > 0
    .SeriesCollection(1).Delete
Loop
With .SeriesCollection.NewSeries
    .Values = wl.Range(wl.Cells(3, sCols(si)), wl.Cells(lastRow, sCols(si)))
    .XValues = xRng
    .Name = sNams(si)
    .Border.Color = sClrs(si)
    .Border.Weight = 2 ' xlThin -- clearly visible
    .Smooth = False
End With

' Fonts AFTER ChartType so Excel does not reset them
.ChartArea.Font.Name = "Consolas"
.ChartArea.Font.Color = CLR_FG_WHITE
.ChartArea.Font.Size = 9

' Backgrounds
.PlotArea.Interior.Color = bgClr
.PlotArea.Border.LineStyle = -4142
.ChartArea.Interior.Color = bgClr

' Outer border matches this series' group color (RR4/RR5/Combined)
With .ChartArea.Border
    .LineStyle = 1
    .Color = sClrs(si)
    .Weight = 2
End With

' --- X axis: TIME SCALE = one tick per calendar month ---
On Error Resume Next
With .Axes(1) ' xlCategory
    .CategoryType = 3 ' xlTimeScale -- treat X values as real dates
    .BaseUnit = 0 ' xlDays -- day granularity underneath
    .BaseUnitIsAuto = False
    .MajorUnitScale = 1 ' xlMonths -- MAJOR TICK EVERY CALENDAR MONTH
    .MajorUnit = 1
    .MajorUnitIsAuto = False
    .MinimumScaleIsAuto = False
    .MinimumScale = xMin ' left edge = first data date
    .MaximumScaleIsAuto = True ' right edge follows the latest date
    .TickLabels.NumberFormat = "yyyy/m"
    .TickLabels.Font.Name = "Consolas"
    .TickLabels.Font.Color = CLR_FG_WHITE
    .TickLabels.Font.Size = 8
    .AxisLine.Border.LineStyle = -4142
    .MajorGridlines.Delete
End With
On Error GoTo 0

' --- Y axis ---
On Error Resume Next
With .Axes(2) ' xlValue
    .TickLabels.NumberFormat = sFmts(si)
    .TickLabels.Font.Name = "Consolas"
    .TickLabels.Font.Color = CLR_FG_WHITE
    .TickLabels.Font.Size = 8
    .AxisLine.Border.LineStyle = -4142
    .MajorGridlines.Border.Color = gridClr
    .MajorGridlines.Border.Weight = 1
End With
On Error GoTo 0

' Title
.HasTitle = True
.ChartTitle.Text = sNams(si)
.ChartTitle.Font.Name = "Consolas"
.ChartTitle.Font.Color = CLR_FG_WHITE
.ChartTitle.Font.Bold = False
.ChartTitle.Font.Size = 10

```

```

        .HasLegend = False
    End With
    Set co2 = Nothing
Next si
End Sub

```

```

' =====
' PRIVATE: risk-adjusted performance panel (CPLog cols AD:AF)
' 30D volatility + Sharpe-like ratio (last 30 rows of col 12),
' Max Drawdown (compounds col 12 into a cumulative index),
' Beta / Correlation vs TWII (col 13 daily change vs col 12).
' Placed clear of the 8 stacked charts (which occupy cols O:AB).
' =====

```

```
Private Sub BuildRiskMetrics(wl As Worksheet, lastRow As Long)
```

```

    Dim mc As Long: mc = 30    ' AD -- metric label
    Dim vc As Long: vc = 31    ' AE -- value
    Dim dc As Long: dc = 32    ' AF -- detail

```

```

    wl.Range(wl.Cells(3, mc), wl.Cells(9, dc)).Clear
    wl.Columns(mc).ColumnWidth = 20
    wl.Columns(vc).ColumnWidth = 10
    wl.Columns(dc).ColumnWidth = 24

```

```

    If lastRow < 5 Then Exit Sub    ' need a few days of history before this means anything

```

```

    ' --- 30D volatility + Sharpe-like ratio: last 30 valid col-12 values ---

```

```

    Dim volArr() As Double: ReDim volArr(1 To 30)
    Dim volN As Long: volN = 0
    Dim rr As Long
    For rr = lastRow To 4 Step -1
        If volN >= 30 Then Exit For
        If IsNumeric(wl.Cells(rr, 12).Value) And wl.Cells(rr, 12).Value <> "" Then
            volN = volN + 1
            volArr(volN) = Cdbl(wl.Cells(rr, 12).Value)
        End If
    Next rr

```

```

    Dim dailyVol As Double, annVol As Double, dailyMean As Double, sharpe As Double

```

```

    If volN >= 2 Then
        Dim volSlice() As Double: ReDim volSlice(1 To volN)
        Dim vi As Long
        For vi = 1 To volN: volSlice(vi) = volArr(vi): Next vi
        dailyVol = Application.WorksheetFunction.StDev(volSlice)
        dailyMean = Application.WorksheetFunction.Average(volSlice)
        annVol = dailyVol * Sqr(252)
        If dailyVol <> 0 Then sharpe = (dailyMean / dailyVol) * Sqr(252)
    End If

```

```

    ' --- Max Drawdown: compound col 12 into a cumulative index, track running peak ---

```

```

    Dim cumIdx As Double: cumIdx = 1
    Dim peakIdx As Double: peakIdx = 1
    Dim peakDateRun As Date, ddPeakDate As Date, ddTroughDate As Date
    Dim maxDD As Double: maxDD = 0
    Dim firstDD As Boolean: firstDD = True
    Dim ddRow As Long
    For ddRow = 4 To lastRow
        If IsNumeric(wl.Cells(ddRow, 12).Value) And wl.Cells(ddRow, 12).Value <> "" Then
            cumIdx = cumIdx * (1 + Cdbl(wl.Cells(ddRow, 12).Value))
            If firstDD Then
                peakDateRun = CDate(wl.Cells(ddRow, 1).Value)
                firstDD = False
            End If
            If cumIdx > peakIdx Then
                peakIdx = cumIdx
                peakDateRun = CDate(wl.Cells(ddRow, 1).Value)
            End If
            Dim curDD As Double: curDD = (cumIdx - peakIdx) / peakIdx
            If curDD < maxDD Then
                maxDD = curDD
            End If
        End If
    Next ddRow

```



```

        ddPeakDate = peakDateRun
        ddTroughDate = CDate(wl.Cells(ddRow, 1).Value)
    End If
End If
Next ddRow

' --- Beta / Correlation vs TWII: pair col 12 (port ret) with TWII's own daily change ---
Dim portArr() As Double: ReDim portArr(1 To lastRow)
Dim twiiArr() As Double: ReDim twiiArr(1 To lastRow)
Dim betaN As Long: betaN = 0
Dim br As Long
For br = 4 To lastRow
    Dim tNow As Variant, tPrev As Variant, pRet As Variant
    tNow = wl.Cells(br, 13).Value
    tPrev = wl.Cells(br - 1, 13).Value
    pRet = wl.Cells(br, 12).Value
    If IsNumeric(tNow) And IsNumeric(tPrev) And CDBl(tPrev) <> 0 And IsNumeric(pRet) And pRet <> "" Then
        betaN = betaN + 1
        portArr(betaN) = CDBl(pRet)
        twiiArr(betaN) = (CDBl(tNow) - CDBl(tPrev)) / CDBl(tPrev)
    End If
Next br

Dim beta As Double, corr As Double
Dim haveBeta As Boolean: haveBeta = False
If betaN >= 5 Then
    Dim pSlice() As Double: ReDim pSlice(1 To betaN)
    Dim tSlice() As Double: ReDim tSlice(1 To betaN)
    Dim bi As Long
    For bi = 1 To betaN: pSlice(bi) = portArr(bi): tSlice(bi) = twiiArr(bi): Next bi
    On Error Resume Next
    Err.Clear
    beta = Application.WorksheetFunction.Slope(pSlice, tSlice)
    corr = Application.WorksheetFunction.Correl(pSlice, tSlice)
    If Err.Number = 0 Then haveBeta = True
    On Error GoTo 0
End If

' --- Render: title bar + header row (base styling only, no colors yet) ---
With wl.Range(wl.Cells(3, mc), wl.Cells(3, dc))
    .Merge
    .Value = "RISK-ADJUSTED PERFORMANCE"
    .Font.Name = "Consolas": .Font.Bold = True: .Font.Size = 11
    .Font.Color = CLR_FG_WHITE: .Interior.Color = CLR_BG_MID
    .HorizontalAlignment = xlCenter: .VerticalAlignment = xlCenter
End With
wl.Rows(3).RowHeight = 20

Dim hr As Long: hr = 4
With wl.Range(wl.Cells(hr, mc), wl.Cells(hr, dc))
    .Interior.Color = CLR_BG_MID
    .Font.Name = "Consolas": .Font.Bold = True: .Font.Size = 9
    .HorizontalAlignment = xlCenter
End With
wl.Cells(hr, mc).Value = "Metric"
wl.Cells(hr, vc).Value = "Value"
wl.Cells(hr, dc).Value = "Detail"

Dim pr As Long: pr = 5

wl.Cells(pr, mc).Value = "30D Volatility"
If volN >= 2 Then
    wl.Cells(pr, vc).Value = dailyVol: wl.Cells(pr, dc).NumberFormat = "0.00%"
    wl.Cells(pr, dc).Value = "annualized " & Format(annVol, "0%")
Else
    wl.Cells(pr, vc).Value = "-": wl.Cells(pr, dc).Value = "need more history"
End If
pr = pr + 1

wl.Cells(pr, mc).Value = "Sharpe-like"
If volN >= 2 And dailyVol < 0 Then

```

```

        wl.Cells(pr, vc).Value = sharpe: wl.Cells(pr, vc).NumberFormat = "0.00"
        wl.Cells(pr, dc).Value = "return / volatility"
    Else
        wl.Cells(pr, vc).Value = "-": wl.Cells(pr, dc).Value = "need more history"
    End If
    pr = pr + 1

    wl.Cells(pr, mc).Value = "Max Drawdown"
    If maxDD < 0 Then
        wl.Cells(pr, vc).Value = maxDD: wl.Cells(pr, vc).NumberFormat = "0.00%"
        wl.Cells(pr, dc).Value = Format(ddPeakDate, "yyyy/m/d") & " to " & Format(ddTroughDate, "yyyy/m/d")
    Else
        wl.Cells(pr, vc).Value = "-": wl.Cells(pr, dc).Value = "no drawdown yet"
    End If
    pr = pr + 1

    wl.Cells(pr, mc).Value = "Beta vs TWII"
    If haveBeta Then
        wl.Cells(pr, vc).Value = beta: wl.Cells(pr, vc).NumberFormat = "0.00"
        wl.Cells(pr, dc).Value = "corr " & Format(corr, "0.00")
    Else
        wl.Cells(pr, vc).Value = "-": wl.Cells(pr, dc).Value = "TWII data too sparse"
    End If

    Dim lastPr As Long: lastPr = pr

    ' --- Base styling for the whole data block (BEFORE per-value colors below) ---
    With wl.Range(wl.Cells(5, mc), wl.Cells(lastPr, dc))
        .Font.Name = "Consolas": .Font.Size = 9: .Font.Color = CLR_FG_WHITE
        .Borders.LineStyle = xlContinuous: .Borders.Color = CLR_BORDER
    End With
    Dim rz As Long
    For rz = 5 To lastPr
        wl.Rows(rz).Interior.Color = IIf(rz Mod 2 = 1, CLR_SECTION, CLR_BG_MID)
    Next rz

    ' --- Per-value colors applied LAST so the block styling above doesn't clobber them ---
    If volN >= 2 And dailyVol <> 0 Then ColorPnl wl.Cells(6, vc), sharpe
    If maxDD < 0 Then ColorPnl wl.Cells(7, vc), maxDD
End Sub

```

```

' =====
' PUBLIC: Build holdings pie chart (RR5 breakdown by type)
' Reads the Futures/Options/Warrants net-exposure rows from
' the CombinedPerf sheet and renders a pie chart below the table.
' =====

```

```

Public Sub BuildHoldingsPieCharts()
    ' Builds TWO pie charts on CombinedPerf, stacked vertically under E4:
    ' 1. "MARKET VALUE DISTRIBUTION" -- raw market value, unadjusted
    ' 2. "LEVERAGE-ADJUSTED EXPOSURE" -- weighted by leverage (see below)
    ' Both are kept so raw capital-at-risk and true economic exposure can be
    ' compared side by side -- neither replaces the other.
    ' Data sources (raw vs leveraged):
    ' RR5 Futures rows 7-14 col N (14) = NET EXPOS (already full notional, same in both)
    ' RR5 Options rows 19-26 col Q (17) = NET EXPOS (same in both)
    ' RR5 Warrants rows 31-38 col Q (17) = NET EXPOS; leveraged = NET EXPOS x col AB (28) EFF GEARING
    ' RR4 Analysis col A = Ticker, col G (RR4_PIE_VAL_COL) = NET EXPOS;
    ' leveraged = NET EXPOS x GetLeverageMultiplier (2x for "NNNNL" ETFs, etc.)

    On Error GoTo Fail
    Application.ScreenUpdating = False

    ' --- 1. Collect individual positions into two dicts: raw MV and leveraged MV ---
    Dim dictRaw As Object: Set dictRaw = CreateObject("Scripting.Dictionary")
    Dim dictLev As Object: Set dictLev = CreateObject("Scripting.Dictionary")

    ' RR5 -----
    Dim wsR As Worksheet
    On Error Resume Next
    Set wsR = ThisWorkbook.Sheets("RR5")
    On Error GoTo Fail

```

```

If Not wsR Is Nothing Then
    Dim i As Long, code As String, mv As Double

    ' Futures rows 7-14 col N=14 -- no leverage concept here, same in both dicts
    For i = 7 To 14
        code = Trim(CStr(wsR.Cells(i, 1).Value))
        If code = "" Then GoTo NextFut
        mv = Abs(V2D(wsR.Cells(i, 14).Value))
        If mv > 0 Then
            PieAddDict dictRaw, code, mv
            PieAddDict dictLev, code, mv
        End If
    NextFut:
    Next i

    ' Options rows 19-26 col Q=17 -- same in both dicts
    For i = 19 To 26
        code = Trim(CStr(wsR.Cells(i, 1).Value))
        If code = "" Then GoTo NextOpt
        mv = Abs(V2D(wsR.Cells(i, 17).Value))
        If mv > 0 Then
            PieAddDict dictRaw, code, mv
            PieAddDict dictLev, code, mv
        End If
    NextOpt:
    Next i

    ' Warrants rows 31-38 col Q=17 NET EXPOS (capital deployed)
    ' leveraged = x col AB=28 EFF GEARING -> underlying-equivalent exposure
    For i = 31 To 38
        code = Trim(CStr(wsR.Cells(i, 1).Value))
        If code = "" Then GoTo NextWrt
        mv = Abs(V2D(wsR.Cells(i, 17).Value))
        If mv > 0 Then
            PieAddDict dictRaw, code, mv
            Dim wGear As Double: wGear = V2D(wsR.Cells(i, 28).Value)
            If wGear <= 0 Then wGear = 1 ' EFF GEARING not yet computed -- fall back to raw MV
            PieAddDict dictLev, code, mv * wGear
        End If
    NextWrt:
    Next i
End If

' RR4 -----
Dim rr4wb As Workbook: Set rr4wb = FindRR4WB()
If Not rr4wb Is Nothing Then
    Dim wsA As Worksheet
    ' col G = market value in RR4 sheet (user confirmed)
    On Error Resume Next: Set wsA = rr4wb.Sheets("RR4"): On Error GoTo Fail
    If Not wsA Is Nothing Then
        ' RR4 sheet layout (DrawColumnHeaders):
        ' Row 4 headers: ISIN, NAME, ENTRY DT, DAYS, BROKER, SECTOR,
        ' NET EXPOS(7), SHARES, ENTRY PX, LAST, % CHG,
        ' UNRL PNL, WT%, W.BETA, Beta 180D, P.TARGET, SWING RISK(17)
        ' Data rows start at row 5; broker header / subtotal rows also present.
        Dim r As Long
        Dim valCol As Long: valCol = 7 ' NET EXPOS = col G

        For r = 5 To 500
            Dim tk As String: tk = Trim(CStr(wsA.Cells(r, 1).Value))
            If tk = "" Then GoTo NextRR4
            ' Skip broker header rows ("= BROKER ==") and subtotal rows
            If InStr(tk, "=") > 0 Or InStr(LCase(tk), "subtotal") > 0 Then GoTo NextRR4
            Dim mvRaw As Variant: mvRaw = wsA.Cells(r, valCol).Value
            If Not IsNumeric(mvRaw) Then GoTo NextRR4
            Dim mvAbs As Double: mvAbs = Abs(CDbl(mvRaw))
            If mvAbs > 0 Then
                PieAddDict dictRaw, tk, mvAbs
                PieAddDict dictLev, tk, mvAbs * GetLeverageMultiplier(tk)
            End If
        Next r
    End If
End If

```

```

NextRR4:
    Next r
End If
End If

If dictRaw.count = 0 Then
    MsgBox "No position data found." & vbCrLf & _
        "Refresh RR5 first, and open the RR4 workbook if applicable.", _
        vbInformation, "CombinedPerf"
    GoTo Done
End If

' --- 2. Build both pie charts on CombinedPerf sheet, stacked vertically ---
Dim wS As Worksheet
On Error Resume Next: Set wS = ThisWorkbook.Sheets(SH_OUT): On Error GoTo Fail
If wS Is Nothing Then Set wS = ActiveSheet

Dim anchorLeft As Double: anchorLeft = wS.Cells(4, 5).Left
Dim anchorTop As Double: anchorTop = wS.Cells(4, 5).Top

' Same ticker = same color in both pies, regardless of each pie's own
' value-sorted slice order (alphabetical key -> stable day to day too).
Dim colorMap As Object: Set colorMap = BuildTickerColorMap(dictRaw, dictLev)

Call RenderHoldingsPie(wS, dictRaw, "PieHoldings", _
    "RR4 + RR5 MARKET VALUE DISTRIBUTION", "Market Value", _
    anchorLeft, anchorTop, colorMap)

Call RenderHoldingsPie(wS, dictLev, "PieHoldingsLev", _
    "RR4 + RR5 LEVERAGE-ADJUSTED EXPOSURE", "Leveraged Exposure", _
    anchorLeft, anchorTop + 340, colorMap)

Done:
Application.ScreenUpdating = True
Application.StatusBar = "Holdings pie charts built " & Format(Now, "hh:mm:ss")
If Not wS Is Nothing Then wS.Activate
Exit Sub

Fail:
Application.ScreenUpdating = True
MsgBox "BuildHoldingsPieCharts failed: " & Err.Description, vbExclamation, "CombinedPerf"
End Sub

' Builds a ticker -> RGB color map from the union of both dicts' keys.
' Assignment order is ALPHABETICAL (not by value/rank), so a given ticker's
' color stays fixed day to day even as market values and rankings shift.
Private Function BuildTickerColorMap(dict1 As Object, dict2 As Object) As Object
    Dim allKeys As Object: Set allKeys = CreateObject("Scripting.Dictionary")
    Dim k As Variant
    For Each k In dict1.keys: allKeys(k) = True: Next k
    For Each k In dict2.keys: allKeys(k) = True: Next k

    Dim n As Long: n = allKeys.count
    Dim tk() As String: ReDim tk(1 To n)
    Dim idx As Long: idx = 1
    For Each k In allKeys.keys
        tk(idx) = CStr(k): idx = idx + 1
    Next k

    Dim a As Long, b As Long, tmpS As String
    For a = 1 To n - 1
        For b = a + 1 To n
            If tk(b) < tk(a) Then
                tmpS = tk(a): tk(a) = tk(b): tk(b) = tmpS
            End If
        Next b
    Next a

    Dim pal(0 To 15) As Long
    pal(0) = RGB(91, 155, 213) ' blue
    pal(1) = RGB(192, 80, 77) ' red
    pal(2) = RGB(155, 187, 89) ' green

```

```

pal(3) = RGB(128, 100, 162)    ' purple
pal(4) = RGB(75, 172, 198)    ' teal
pal(5) = RGB(247, 150, 70)    ' orange
pal(6) = RGB(255, 217, 90)    ' gold
pal(7) = RGB(219, 219, 219)    ' light gray
pal(8) = RGB(146, 208, 80)    ' lime
pal(9) = RGB(0, 176, 240)    ' sky blue
pal(10) = RGB(255, 105, 180)    ' pink
pal(11) = RGB(191, 143, 0)    ' brown-gold
pal(12) = RGB(255, 255, 102)    ' yellow
pal(13) = RGB(0, 153, 153)    ' dark teal
pal(14) = RGB(153, 102, 255)    ' violet
pal(15) = RGB(200, 200, 200)    ' silver

```

```
Dim result As Object: Set result = CreateObject("Scripting.Dictionary")
```

```
For idx = 1 To n
```

```
    result(tk(idx)) = pal((idx - 1) Mod 16)
```

```
Next idx
```

```
Set BuildTickerColorMap = result
```

```
End Function
```

```

' Sorts dict by value descending, then draws one pie chart named chartName
' at (leftPos, topPos), 400x320, styled via CombPerfPie.crtx (dark theme,
' Consolas fonts) exactly like the original single-pie implementation.
' colorMap forces each slice's fill by ticker so colors match across pies.

```

```
Private Sub RenderHoldingsPie(wS As Worksheet, dict As Object, chartName As String, _
    titleText As String, seriesName As String, leftPos As Double, topPos As Double, _
    colorMap As Object)
```

```
If dict.count = 0 Then Exit Sub
```

```
' Sort by value descending (insertion sort; small n)
```

```
Dim n As Long: n = dict.count
```

```
Dim tickers() As String: ReDim tickers(1 To n)
```

```
Dim vals() As Double: ReDim vals(1 To n)
```

```
Dim j As Long: j = 1
```

```
Dim k As Variant
```

```
For Each k In dict.keys
```

```
    tickers(j) = CStr(k): vals(j) = CDbl(dict(k)): j = j + 1
```

```
Next k
```

```
Dim a As Long, b As Long, tmpD As Double, tmpS As String
```

```
For a = 1 To n - 1
```

```
    For b = a + 1 To n
```

```
        If vals(b) > vals(a) Then
```

```
            tmpD = vals(a): vals(a) = vals(b): vals(b) = tmpD
```

```
            tmpS = tickers(a): tickers(a) = tickers(b): tickers(b) = tmpS
```

```
        End If
```

```
    Next b
```

```
Next a
```

```
Dim vArr() As Variant: ReDim vArr(1 To n)
```

```
Dim lArr() As Variant: ReDim lArr(1 To n)
```

```
For j = 1 To n: vArr(j) = vals(j): lArr(j) = tickers(j): Next j
```

```
' Remove old chart of the same name
```

```
Dim co As ChartObject
```

```
On Error Resume Next
```

```
For Each co In wS.ChartObjects
```

```
    If co.Name = chartName Then co.Delete
```

```
Next co
```

```
On Error GoTo 0
```

```
Dim pie As ChartObject
```

```
Set pie = wS.ChartObjects.Add(leftPos, topPos, 400, 320)
```

```
pie.Name = chartName
```

```
Dim tplPath As String
```

```
tplPath = Environ("USERPROFILE") & "\CombPerfPie.crtx"
```

```
If Dir(tplPath) = "" Then tplPath = "" ' fallback: style via VBA if file missing
```

```
With pie.Chart
```

```

.ChartType = 5      ' xlPie = 5

' Apply .crtx template: colours, fonts, dark theme all come from file
On Error Resume Next
If Dir(tplPath) <> "" Then .ApplyChartTemplate tplPath
On Error GoTo 0

' Clear any placeholder series the template may have left,
' so our data becomes SeriesCollection(1) (pie only shows series 1)
On Error Resume Next
Dim delSC As Long
For delSC = .SeriesCollection.count To 1 Step -1
    .SeriesCollection(delSC).Delete
Next delSC
On Error GoTo 0

' Add our data as the sole series
Dim sr As Series: Set sr = .SeriesCollection.NewSeries
sr.Values = vArr
sr.XValues = lArr
sr.Name = seriesName

' Force each slice to its mapped color by ticker (not by slice index),
' so the same ticker renders identically in both pies.
On Error Resume Next
Dim pIdx As Long
For pIdx = 1 To n
    If colorMap.Exists(tickers(pIdx)) Then
        sr.Points(pIdx).Interior.Color = colorMap(tickers(pIdx))
    End If
Next pIdx
On Error GoTo 0

.HasTitle = True
.ChartTitle.Text = titleText
.ChartTitle.Font.Bold = True
.ChartTitle.Font.Size = 11
.ChartTitle.Font.Name = "Consolas"
.ChartTitle.Font.Color = CLR_FG_WHITE

.HasLegend = False      ' labels shown directly on each slice

' Data labels: ticker name + percentage on separate lines
On Error Resume Next
With .SeriesCollection(1)
    .ApplyDataLabels
    With .DataLabels
        .ShowCategoryName = True
        .ShowPercentage = True
        .ShowValue = False
        .Separator = Chr(10)
        .NumberFormat = "0%"
        .Font.Bold = True
        .Font.Size = 8
        .Font.Name = "Consolas"
        .Font.Color = CLR_FG_WHITE
    End With
End With
On Error GoTo 0
End With
End Sub

Private Sub PieAddDict(dict As Object, key As String, val As Double)
    If dict.Exists(key) Then
        dict(key) = dict(key) + val
    Else
        dict(key) = val
    End If
End Sub
End Sub

' Leveraged-ETF detector for RR4 holdings.

```

```

' TWSE/TPEX naming convention: a numeric ticker ending in "L" is always a
' 2x leveraged product (e.g. 00631L, 00675L = "positive 2x"); tickers ending
' in "R" are -1x inverse (same magnitude as unleveraged, so no adjustment).
' Extend the Select Case below if a non-Taiwan leveraged ETF (e.g. a US 3x
' product) ever shows up in RR4 and needs a manual multiplier.
Private Function GetLeverageMultiplier(ticker As String) As Double
    Dim tk As String: tk = UCase(Trim(ticker))
    GetLeverageMultiplier = 1
    If Len(tk) < 2 Then Exit Function
    If Right(tk, 1) = "L" And IsNumeric(Left(tk, Len(tk) - 1)) Then
        GetLeverageMultiplier = 2
        Exit Function
    End If
    Select Case tk
        ' Manual overrides for non-Taiwan leveraged ETFs (user's actual US holdings).
        Case "SOXL"                                     ' Direxion Semiconductor Bull 3x (sector index)
            GetLeverageMultiplier = 3
        Case "SNXX", "MRVU", "LABX", "SSO", "MUU"       ' single-stock / S&P500 2x products
            GetLeverageMultiplier = 2
    End Select
End Function

```

```

' =====
' DIAGNOSTIC: scan RR4 Analysis sheet structure
' Run via Alt+F8 -> DiagnoseRR4
' Shows: sheet list, Analysis headers, all-col SUM scan, samples
' =====

```

```

Public Sub DiagnoseRR4()
    Dim wb As Workbook
    Set wb = FindRR4WB()
    If wb Is Nothing Then
        MsgBox "RR4 workbook not found. Open it first.", vbExclamation, "DiagnoseRR4"
        Exit Sub
    End If

    Dim out As String
    out = "==== RR4: " & wb.Name & " ===" & vbCrLf & vbCrLf

    ' -- List all sheet names --
    out = out & "[All Sheets]" & vbCrLf
    Dim wS As Worksheet
    For Each wS In wb.Sheets
        out = out & " " & wS.Name & vbCrLf
    Next wS
    out = out & vbCrLf

    ' -- Analysis sheet --
    Dim wsA As Worksheet
    On Error Resume Next
    Set wsA = wb.Sheets("Analysis")
    On Error GoTo 0

    If wsA Is Nothing Then
        out = out & "[Analysis] sheet NOT FOUND" & vbCrLf
        MsgBox out, vbExclamation, "DiagnoseRR4"
        Exit Sub
    End If

    ' Use UsedRange for true dimensions (avoids col=1 if row1 is a merged title)
    Dim ur As Range: Set ur = wsA.UsedRange
    Dim lastRow As Long: lastRow = ur.Row + ur.Rows.Count - 1
    Dim lastCol As Long: lastCol = ur.Column + ur.Columns.Count - 1
    out = out & "[Analysis] usedRows=2-" & lastRow & " usedCols=1-" & lastCol & vbCrLf & vbCrLf

    ' Dump every non-empty row: show all cols A..lastCol
    out = out & "--- All non-empty rows (full dump) ---" & vbCrLf
    Dim r As Long, c As Long
    For r = 1 To lastRow
        Dim rowStr As String: rowStr = ""
        For c = 1 To lastCol

```

```

        Dim sv As String: sv = Trim(CStr(wsA.Cells(r, c).Value))
        If sv <> "" Then
            rowStr = rowStr & " " & ColLetter(c) & "=" & Left(sv, 16)
        End If
    Next c
    If rowStr <> "" Then
        out = out & "R" & r & ":" & rowStr & vbCrLf
    End If
Next r

' Split output if too long
If Len(out) > 1800 Then
    MsgBox Left(out, 1800), vbInformation, "DiagnoseRR4 (1/2)"
    MsgBox Mid(out, 1801), vbInformation, "DiagnoseRR4 (2/2)"
Else
    MsgBox out, vbInformation, "DiagnoseRR4"
End If
End Sub

Private Function ColLetter(n As Long) As String
    If n <= 26 Then
        ColLetter = Chr(64 + n)
    Else
        ColLetter = Chr(64 + (n - 1) \ 26) & Chr(65 + (n - 1) Mod 26)
    End If
End Function
End Function

```


Option Explicit

' 1. 初始化計算機介面

Sub SetupCalculatorGUI()

Dim wS As Worksheet

Set wS = ActiveSheet

' 清除現有工作表內容與格式

wS.Cells.Clear

' 標題設定

wS.Range("B2").Value = "選擇權單次計算機 (Options Calculator)"

wS.Range("B2").Font.Bold = True

wS.Range("B2").Font.Size = 12

wS.Columns("B").ColumnWidth = 22

wS.Columns("C").ColumnWidth = 15

' --- 建立輸入區標籤 ---

wS.Range("B4").Value = "評價日期 (Date):"

wS.Range("B5").Value = "標的代碼 (Ticker):"

wS.Range("B6").Value = "選擇權類型 (Type):" ' 必須為 Call 或 Put

wS.Range("B7").Value = "履約價 (Strike):"

wS.Range("B8").Value = "到期日 (Expiry):"

wS.Range("B9").Value = "標的現價 (Underlying):" ' 計算 Greeks 必須欄位

wS.Range("B10").Value = "進場 IV (Entry IV):" ' 以小數表示，例如 0.3

' --- 建立輸出區標籤 ---

wS.Range("B12").Value = "[系統計算結果]"

wS.Range("B12").Font.Bold = True

wS.Range("B13").Value = "Gamma (Γ):"wS.Range("B14").Value = "每日 Theta (Θ):"

' --- 視覺化格式設定 ---

' 將輸入區 (C4:C10) 設為淺黃色，加上外框

Dim inputRange As Range

Set inputRange = wS.Range("C4:C10")

inputRange.Interior.Color = RGB(255, 255, 224)

inputRange.Borders.LineStyle = xlContinuous

' 將輸出區 (C13:C14) 設為淺綠色，加上外框

Dim outputRange As Range

Set outputRange = wS.Range("C13:C14")

outputRange.Interior.Color = RGB(224, 255, 224)

outputRange.Borders.LineStyle = xlContinuous

outputRange.NumberFormat = "0.0000" ' 設定計算結果顯示至小數點後四位

' --- 填入預設測試資料 ---

wS.Range("C4").Value = Date

wS.Range("C5").Value = "NVDA"

wS.Range("C6").Value = "Call"

wS.Range("C7").Value = 120

wS.Range("C8").Value = Date + 30 ' 預設為30天後到期

wS.Range("C9").Value = 118.5

wS.Range("C10").Value = 0.45

MsgBox "計算機面板已建置完成。" & vbCrLf & _

"請於黃色儲存格輸入參數後，執行 CalculateGreeks 進行運算。", vbInformation, "初始化成功"

End Sub

Sub CalculateGreeks()

Dim wS As Worksheet

Set wS = ActiveSheet

' 防呆：任何必填欄位空白就靜默退出

Dim req As Variant, cellRef As Variant

req = Array("C4", "C6", "C7", "C8", "C9", "C10")

For Each cellRef In req

If Trim(CStr(wS.Range(cellRef).Value)) = "" Then

wS.Range("C13:C14").ClearContents

Exit Sub

End If

Next cellRef

' 防呆：日期合法、到期 > 評價

```
If Not IsDate(wS.Range("C4").Value) Or Not IsDate(wS.Range("C8").Value) Then
    wS.Range("C13:C14").ClearContents
    Exit Sub
```

End If

Dim t As Double

t = (CDate(wS.Range("C8").Value) - CDate(wS.Range("C4").Value)) / 365

If t <= 0 Then

wS.Range("C13:C14").ClearContents

Exit Sub

End If

' 計算

Dim s As Double, k As Double, v As Double, r As Double, optType As String

s = CDbl(wS.Range("C9").Value)

k = CDbl(wS.Range("C7").Value)

v = CDbl(wS.Range("C10").Value)

optType = CStr(wS.Range("C6").Value)

' IV 格式防護 (使用者填 30 -> 0.3)

If v > 2 Then v = v / 100

' 無風險利率：空 -> 0.02, > 0.5 -> 視為 %

If Trim(CStr(wS.Range("C11").Value)) = "" Then

r = 0.02

Else

r = CDbl(wS.Range("C11").Value)

If r > 0.5 Then r = r / 100

End If

wS.Range("C13").Value = Calc_Gamma(s, k, t, r, v)

wS.Range("C14").Value = Calc_Daily_Theta(s, k, t, r, v, optType)

End Sub

' 核心數學函數：請確保這些程式碼存在於您的模組中

' 1. 計算 dl

Function Calc_dl(s As Double, k As Double, t As Double, r As Double, v As Double) As Double

Calc_dl = (Log(s / k) + (r + v ^ 2 / 2) * t) / (v * Sqr(t))

End Function

' 2. 計算標準常態分配密度函數 N'(x)

Function ND_prime(x As Double) As Double

ND_prime = Exp(-(x ^ 2) / 2) / Sqr(2 * WorksheetFunction.pi())

End Function

' 3. 計算 Gamma

Function Calc_Gamma(s As Double, k As Double, t As Double, r As Double, v As Double) As Double

Dim dl As Double

' 避免到期日為0導致分母為0的錯誤

If t <= 0 Then t = 0.0001

dl = Calc_dl(s, k, t, r, v)

Calc_Gamma = ND_prime(dl) / (s * v * Sqr(t))

End Function

' 4. 計算每日 Theta (時間損耗)

Function Calc_Daily_Theta(s As Double, k As Double, t As Double, r As Double, v As Double, optType As String) As Double

Dim dl As Double, d2 As Double

Dim Nd1p As Double, Nd2 As Double, N_minus_d2 As Double

Dim annualTheta As Double

If t <= 0 Then t = 0.0001

dl = Calc_dl(s, k, t, r, v)

d2 = dl - v * Sqr(t)

Nd1p = ND_prime(dl)

If UCase(optType) = "CALL" Or UCase(optType) = "C" Then

```

Nd2 = WorksheetFunction.Norm_Dist(d2, 0, 1, True)
annualTheta = -(s * v * Nd1p) / (2 * Sqr(t)) - r * k * Exp(-r * t) * Nd2
ElseIf UCase(optType) = "PUT" Or UCase(optType) = "P" Then
    N_minus_d2 = WorksheetFunction.Norm_Dist(-d2, 0, 1, True)
    annualTheta = -(s * v * Nd1p) / (2 * Sqr(t)) + r * k * Exp(-r * t) * N_minus_d2
End If

' 將年化 Theta 轉換為每日時間損耗
Calc_Daily_Theta = annualTheta / 365
End Function

```

```

' 自動抓取 Yahoo Finance 最新股價
Function GetLivePrice(ticker As String) As Double
    Dim req As Object
    Dim url As String
    Dim response As String
    Dim priceStart As Long
    Dim priceEnd As Long
    Dim priceStr As String

    On Error GoTo ErrorHandler

    ' 建立 HTTP 請求
    Set req = CreateObject("MSXML2.XMLHTTP")
    url = "https://query1.finance.yahoo.com/v8/finance/chart/" & ticker

    req.Open "GET", url, False
    req.Send
    response = req.responseText

    ' 簡易 JSON 字串解析 (尋找 "regularMarketPrice")
    priceStart = InStr(response, """"regularMarketPrice"":") + 21
    If priceStart > 21 Then
        priceEnd = InStr(priceStart, response, ",")
        priceStr = Mid(response, priceStart, priceEnd - priceStart)
        GetLivePrice = CDbl(priceStr)
    Else
        GetLivePrice = 0 ' 找不到報價
    End If
Exit Function

```

```

ErrorHandler:
    GetLivePrice = 0
End Function

```

```

Sub GenerateThetaData()
    Dim wS As Worksheet
    Set wS = ActiveSheet

```

```

' 【除錯重點：暫時關閉錯誤隱藏】
' 註解掉 On Error 指令，若有變數或函數錯誤，系統才會明確標示出錯行數
' On Error Resume Next

```

```

' 1. 讀取現有介面參數
Dim s As Double, k As Double, v As Double, r As Double
Dim optType As String
Dim totalDays As Integer

```

```

s = CDbl(wS.Range("C9").Value)
k = CDbl(wS.Range("C7").Value)
v = CDbl(wS.Range("C10").Value)
optType = CStr(wS.Range("C6").Value)

```

```

' 動態讀取無風險利率 (C11)

```

```

If Trim(wS.Range("C11").Value) = "" Then
    r = 0.02
Else
    r = CDbl(wS.Range("C11").Value)
    If r > 0.5 Then r = r / 100
End If

```

```

' 隱含波動率 (IV) 格式防護
If v > 2 Then v = v / 100

' 計算總剩餘天數
totalDays = CInt(CDate(wS.Range("C8").Value) - CDate(wS.Range("C4").Value))
If totalDays <= 0 Then Exit Sub

' 2. 準備資料輸出區 (從 16 列開始)
Dim dataStartRow As Long, i As Long
Dim currentT As Double, dailyTheta As Double
dataStartRow = 16

' 清空舊資料，並預先將整個輸出區塊的字體設為白色
wS.Range("B16:C" & wS.Rows.count).ClearContents
wS.Range("B16:C" & wS.Rows.count).Font.Color = vbWhite

' 寫入標題與格式
wS.Cells(dataStartRow, 2).Value = "Days to Expiration"
wS.Cells(dataStartRow, 3).Value = "Daily Theta"
wS.Cells(dataStartRow, 2).Font.Bold = True
wS.Cells(dataStartRow, 3).Font.Bold = True

' 3. 執行「時間旅行」迴圈計算
Dim rowOffset As Long
rowOffset = 1

For i = totalDays To 1 Step -1
    currentT = i / 365 ' 將天數轉換為年化時間

    ' 呼叫外部核心函數計算單日 Theta
    dailyTheta = Calc_Daily_Theta(s, k, currentT, r, v, optType)

    ' 寫入工作表
    wS.Cells(dataStartRow + rowOffset, 2).Value = i
    wS.Cells(dataStartRow + rowOffset, 3).Value = dailyTheta

    rowOffset = rowOffset + 1
Next i

' 設定小數點格式以利閱讀
wS.Range(wS.Cells(dataStartRow + 1, 3), wS.Cells(dataStartRow + totalDays, 3)).NumberFormat = "0.0000"

MsgBox "Theta 數據已成功產出!", vbInformation
End Sub

```

Option Explicit

' --- Module-level variables (must be at top of module) ---

Private NextRefreshTime As Date

```
'
' =====
' BLACK-SCHOLES FUNCTIONS
' Can be called from worksheet cells: =BS_Delta(S,K,T,r,sigma,"Call")
' =====
```

```
Public Function NormCDF(x As Double) As Double
    ' Standard normal CDF. Previously a broken hybrid of Hart approximation
    ' and Excel's Norm_S_Dist that gave wrong results for x >= 0 (mixed PDF
    ' into a CDF formula). Now uses Excel's built-in CDF directly.
    NormCDF = Application.WorksheetFunction.Norm_S_Dist(x, True)
End Function
```

```
Public Function BS_Price(s As Double, k As Double, t As Double, r As Double, sigma As Double, optType As String) As Double
    If t <= 0 Then
        If UCase(optType) = "CALL" Then
            BS_Price = Application.WorksheetFunction.Max(0, s - k)
        Else
            BS_Price = Application.WorksheetFunction.Max(0, k - s)
        End If
        Exit Function
    End If
    If sigma <= 0 Or s <= 0 Or k <= 0 Then BS_Price = 0: Exit Function
    Dim d1 As Double, d2 As Double
    d1 = (Log(s / k) + (r + 0.5 * sigma ^ 2) * t) / (sigma * Sqr(t))
    d2 = d1 - sigma * Sqr(t)
    If UCase(optType) = "CALL" Then
        BS_Price = s * NormCDF(d1) - k * Exp(-r * t) * NormCDF(d2)
    Else
        BS_Price = k * Exp(-r * t) * NormCDF(-d2) - s * NormCDF(-d1)
    End If
End Function
```

```
Public Function BS_Delta(s As Double, k As Double, t As Double, r As Double, sigma As Double, optType As String) As Double
    If t <= 0 Or sigma <= 0 Or s <= 0 Or k <= 0 Then
        If UCase(optType) = "CALL" Then BS_Delta = IIf(s > k, 1, 0) Else BS_Delta = IIf(s < k, -1, 0)
        Exit Function
    End If
    Dim d1 As Double
    d1 = (Log(s / k) + (r + 0.5 * sigma ^ 2) * t) / (sigma * Sqr(t))
    If UCase(optType) = "CALL" Then
        BS_Delta = NormCDF(d1)
    Else
        BS_Delta = NormCDF(d1) - 1
    End If
End Function
```

```
Public Function BS_Gamma(s As Double, k As Double, t As Double, r As Double, sigma As Double) As Double
    If t <= 0 Or sigma <= 0 Or s <= 0 Or k <= 0 Then BS_Gamma = 0: Exit Function
    Dim d1 As Double
    d1 = (Log(s / k) + (r + 0.5 * sigma ^ 2) * t) / (sigma * Sqr(t))
    BS_Gamma = Exp(-0.5 * d1 ^ 2) / (Sqr(2 * 3.14159265358979) * s * sigma * Sqr(t))
End Function
```

```
Public Function BS_Theta(s As Double, k As Double, t As Double, r As Double, sigma As Double, optType As String) As Double
    If t <= 0 Or sigma <= 0 Or s <= 0 Or k <= 0 Then BS_Theta = 0: Exit Function
    Dim d1 As Double, d2 As Double
    d1 = (Log(s / k) + (r + 0.5 * sigma ^ 2) * t) / (sigma * Sqr(t))
    d2 = d1 - sigma * Sqr(t)
    Dim term1 As Double
    term1 = -s * Exp(-0.5 * d1 ^ 2) / (Sqr(2 * 3.14159265358979) * 2 * Sqr(t)) * sigma
    If UCase(optType) = "CALL" Then
        BS_Theta = (term1 - r * k * Exp(-r * t) * NormCDF(d2)) / 365
    Else
        BS_Theta = (term1 + r * k * Exp(-r * t) * NormCDF(-d2)) / 365
    End If
End Function
```

```

Public Function BS_Vega(s As Double, k As Double, t As Double, r As Double, sigma As Double) As Double
    If t <= 0 Or sigma <= 0 Or s <= 0 Or k <= 0 Then BS_Vega = 0: Exit Function
    Dim d1 As Double
    d1 = (Log(s / k) + (r + 0.5 * sigma ^ 2) * t) / (sigma * Sqr(t))
    BS_Vega = s * Sqr(t) * Exp(-0.5 * d1 ^ 2) / (Sqr(2 * 3.14159265358979) * 100)
End Function

```

```

Public Function BS_Rho(s As Double, k As Double, t As Double, r As Double, sigma As Double, optType As String) As Double
    If t <= 0 Or sigma <= 0 Or s <= 0 Or k <= 0 Then BS_Rho = 0: Exit Function
    Dim d1 As Double, d2 As Double
    d1 = (Log(s / k) + (r + 0.5 * sigma ^ 2) * t) / (sigma * Sqr(t))
    d2 = d1 - sigma * Sqr(t)
    If UCase(optType) = "CALL" Then
        BS_Rho = k * t * Exp(-r * t) * NormCDF(d2) / 100
    Else
        BS_Rho = -k * t * Exp(-r * t) * NormCDF(-d2) / 100
    End If
End Function

```

```

Public Function BS_IV(mktPx As Double, s As Double, k As Double, t As Double, r As Double, optType As String) As Double
    ' Newton-Raphson implied volatility solver (50 iterations).
    ' Bug B5 fix: returns 0 on non-convergence so caller can detect failure
    ' instead of using a noisy final sigma value.
    If t <= 0 Or s <= 0 Or k <= 0 Or mktPx <= 0 Then BS_IV = 0: Exit Function
    Dim sigma As Double, price As Double, vega As Double
    Dim converged As Boolean: converged = False
    sigma = 0.3 ' initial guess
    Dim i As Integer
    For i = 1 To 50
        price = BS_Price(s, k, t, r, sigma, optType)
        vega = BS_Vega(s, k, t, r, sigma) * 100 ' un-scale from /100
        If Abs(vega) < 0.000001 Then Exit For
        sigma = sigma - (price - mktPx) / vega
        If sigma < 0.001 Then sigma = 0.001
        If sigma > 10 Then sigma = 10
        If Abs(price - mktPx) < 0.0001 Then
            converged = True
            Exit For
        End If
    Next i
    ' Final convergence check: tolerate 1c absolute or 0.1% relative error
    If Not converged Then
        Dim finalPx As Double
        finalPx = BS_Price(s, k, t, r, sigma, optType)
        If Abs(finalPx - mktPx) > 0.01 And Abs(finalPx - mktPx) / mktPx > 0.001 Then
            BS_IV = 0
            Exit Function
        End If
    End If
    BS_IV = sigma
End Function

```

```

Public Function Warrant_Delta(s As Double, k As Double, t As Double, r As Double, sigma As Double, convRatio As Double, wType As String) As Double
    ' wType: "Call Warrant" or "Put Warrant" or "Bull Cert" or "Bear Cert"
    Dim optT As String
    If InStr(UCase(wType), "CALL") > 0 Or InStr(UCase(wType), "BULL") > 0 Then
        optT = "Call"
    Else
        optT = "Put"
    End If
    If convRatio = 0 Then Warrant_Delta = 0: Exit Function
    Warrant_Delta = BS_Delta(s, k, t, r, sigma, optT) / convRatio
End Function

```

```

' =====
' OPERATIONAL SUBROUTINES
' =====

```

```

Private Function GetConfig(wsC As Worksheet, key As String) As Variant

```

```

    Dim i As Long
    For i = 1 To 20
        If wsC.Cells(i, 1).Value = key Then
            GetConfig = wsC.Cells(i, 2).Value
            Exit Function
        End If
    Next i
    GetConfig = ""
End Function

' --- Public wrappers so UserForms / other modules can read & update _Config ---
Public Function GetConfigValue(key As String) As Variant
    ' Returns "" if key not found.
    Dim wsC As Worksheet
    Set wsC = ThisWorkbook.Sheets("_Config")
    GetConfigValue = GetConfig(wsC, key)
End Function

Public Sub SetConfigValue(key As String, val As Variant)
    ' Updates a row in _Config (col B) where col A matches key.
    ' Does nothing if key not found.
    Dim wsC As Worksheet
    Set wsC = ThisWorkbook.Sheets("_Config")
    Dim i As Long
    For i = 1 To 20
        If wsC.Cells(i, 1).Value = key Then
            wsC.Cells(i, 2).Value = val
            Exit Sub
        End If
    Next i
End Sub

Private Sub LogDebug(wsD As Worksheet, macroName As String, status As String, msg As String)
    If wsD Is Nothing Then Exit Sub ' guard: DebugLog sheet may not exist
    On Error Resume Next
    Dim nextRow As Long
    nextRow = wsD.Cells(wsD.Rows.count, 1).End(xlUp).row + 1
    wsD.Cells(nextRow, 1).Value = Now()
    wsD.Cells(nextRow, 2).Value = macroName
    wsD.Cells(nextRow, 3).Value = status
    wsD.Cells(nextRow, 4).Value = msg
    On Error GoTo 0
End Sub

Public Sub RefreshGreeks()
    ' Updates Delta/Gamma/Theta/Vega/Rho for all options and warrants.
    ' Called by timer or manually. Reads IV and spot price from input cells.
    '
    ' Fix: _Config OPT_SEC_START=30 points to the header row (row 30).
    ' Auto-advance past the header if the STRIKE cell is non-numeric.
    On Error GoTo ErrHandler
    Dim wsR As Worksheet, wsC As Worksheet, wsD As Worksheet
    Set wsR = ThisWorkbook.Sheets("RR5")
    Set wsC = ThisWorkbook.Sheets("_Config")
    Set wsD = ThisWorkbook.Sheets("DebugLog")

    Dim r As Double
    r = CDbl(GetConfig(wsC, "BS_RISK_FREE_RATE"))
    If r = 0 Then r = 0.02

    Dim optStart As Long, optCount As Long
    optStart = CLng(GetConfig(wsC, "OPT_SEC_START"))
    optCount = CLng(GetConfig(wsC, "OPT_INST_COUNT"))

    ' Auto-advance past header row(s). Bug B6 fix: was single-step (only
    ' advanced once), so misconfigured _Config that pointed > 1 row above
    ' data would silently write Greeks to header cells. Loop until we land
    ' on numeric STRIKE or hit a safety cap.
    Dim hdrGuard As Long: hdrGuard = 0
    Do While Not IsNumeric(wsR.Cells(optStart, 9).Value) And hdrGuard < 5
        optStart = optStart + 1
    
```

```

    hdrGuard = hdrGuard + 1
Loop

' Options section: rows optStart to optStart+optCount-1 (rows 31-70)
Dim i As Long, row As Long
For i = 0 To optCount - 1
    row = optStart + i
    Dim contract As String
    contract = wsR.Cells(row, 1).Value
    If contract = "" Then GoTo NextOpt
    ' Extra guard: skip any non-data row (e.g. if config is off by more)
    If Not IsNumeric(wsR.Cells(row, 9).Value) Then GoTo NextOpt

    Dim s As Double, k As Double, dte As Double, iv As Double
    Dim oType As String
    s = wsR.Cells(row, 11).Value      ' K UNDL LAST
    k = wsR.Cells(row, 9).Value       ' I STRIKE
    dte = wsR.Cells(row, 5).Value      ' E DTE (days)
    iv = wsR.Cells(row, 19).Value     ' S IV (fraction 0-1)
    oType = wsR.Cells(row, 8).Value   ' H TYPE (Call/Put)

    If s <= 0 Or k <= 0 Or dte < 0 Then GoTo NextOpt

    Dim t As Double
    t = dte / 365

    ' Reverse-solve IV from LAST PREM if IV is missing/zero.
    ' Stores the implied vol back into col S so subsequent calls reuse it.
    If iv <= 0 Then
        Dim lastPrem As Double
        lastPrem = wsR.Cells(row, 15).Value      ' O LAST PREM
        If lastPrem > 0 Then
            Dim solvedIV As Double
            solvedIV = BS_IV(lastPrem, s, k, t, r, oType)
            If solvedIV > 0.001 And solvedIV < 5 Then
                iv = solvedIV
                wsR.Cells(row, 19).Value = iv      ' write back to S IV
            End If
        End If
    End If
    If iv <= 0 Then GoTo NextOpt

    wsR.Cells(row, 20).Value = BS_Delta(s, k, t, r, iv, oType) ' T DELTA
    wsR.Cells(row, 21).Value = BS_Gamma(s, k, t, r, iv)        ' U GAMMA
    wsR.Cells(row, 22).Value = BS_Theta(s, k, t, r, iv, oType) ' V THETA/DAY
    wsR.Cells(row, 23).Value = BS_Vega(s, k, t, r, iv)         ' W VEGA
    wsR.Cells(row, 24).Value = BS_Rho(s, k, t, r, iv, oType)   ' X RHO
NextOpt:
Next i

' Warrants section
Dim wrtStart As Long, wrtCount As Long
wrtStart = CLng(GetConfig(wsC, "WRT_SEC_START"))
wrtCount = CLng(GetConfig(wsC, "WRT_INST_COUNT"))

' Auto-advance past warrant header row(s). Same trick as options.
' Warrant STRIKE column is K (col 11). If header text, advance up to 3.
Dim wHg As Long: wHg = 0
Do While Not IsNumeric(wsR.Cells(wrtStart, 11).Value) And wHg < 3
    wrtStart = wrtStart + 1: wHg = wHg + 1
Loop

For i = 0 To wrtCount - 1
    row = wrtStart + i
    Dim wCode As String
    wCode = wsR.Cells(row, 1).Value
    If wCode = "" Then GoTo NextWrt

    Dim wS As Double, wK As Double, wDte As Double, wIV As Double
    Dim convRatio As Double, wType As String
    wS = wsR.Cells(row, 10).Value      ' UNDL LAST

```



```

wK = wsR.Cells(row, 11).Value      ' STRIKE
wDte = wsR.Cells(row, 5).Value      ' DTE
wIV = wsR.Cells(row, 19).Value      ' IV
convRatio = wsR.Cells(row, 12).Value ' CONV RATIO
wType = wsR.Cells(row, 7).Value     ' TYPE

If wS <= 0 Or wK <= 0 Or wIV <= 0 Or convRatio <= 0 Then GoTo NextWrt

Dim wT As Double
wT = wDte / 365

' Normalise warrant type to "Call"/"Put" for BS_Theta sign handling
Dim wOptT As String
If InStr(UCase(wType), "CALL") > 0 Or InStr(UCase(wType), "BULL") > 0 Then
    wOptT = "Call"
Else
    wOptT = "Put"
End If

' All Greeks are scaled by /convRatio to express "per warrant unit"
wsR.Cells(row, 20).Value = Warrant_Delta(wS, wK, wT, r, wIV, convRatio, wType)
wsR.Cells(row, 24).Value = BS_Gamma(wS, wK, wT, r, wIV) / convRatio      ' X GAMMA
wsR.Cells(row, 25).Value = BS_Theta(wS, wK, wT, r, wIV, wOptT) / convRatio ' Y THETA/DAY
wsR.Cells(row, 26).Value = BS_Vega(wS, wK, wT, r, wIV) / convRatio      ' Z VEGA

' Gearing needs LAST PX (col 0) -- guarded separately since a fresh
' position may have S/K/IV/convRatio filled before FetchAllPrices runs.
Dim wLastPx As Double: wLastPx = wsR.Cells(row, 15).Value ' 0 LAST PX
If wLastPx > 0 Then
    Dim wDelta As Double: wDelta = wsR.Cells(row, 20).Value ' T DELTA (just written above)
    wsR.Cells(row, 27).Value = wS / (wLastPx * convRatio) ' AA NOM GEARING = UNDL / (LastPx*ConvRatio)
    wsR.Cells(row, 28).Value = wS * wDelta / wLastPx      ' AB EFF GEARING = UNDL*Delta / LastPx
Else
    wsR.Cells(row, 27).Value = ""
    wsR.Cells(row, 28).Value = ""
End If

NextWrt:
Next i

LogDebug wsD, "RefreshGreeks", "OK", "Greeks updated at " & Now()
Exit Sub

ErrorHandler:
LogDebug wsD, "RefreshGreeks", "ERROR", Err.Description
End Sub

Public Sub DailySnapshot()
' Appends one row to HistoryLog and Greeks_History with today's portfolio state.
'
' HistoryLog columns:
' A=Date B=TotalMarketValue C=TotalCumulativePnL
' D=NetDelta E=NetGamma F=NetTheta G=NetVega
' H=Price(TX) I=Ret%(TX) J=Price(SPY) K=Ret%(SPY)
' L=RealizedPnL M=PortRet% N=PortRetChg% O=IV_Avg P=Summary
Dim wsR As Worksheet, wsH As Worksheet, wsG As Worksheet, wsD As Worksheet
On Error Resume Next
Set wsD = ThisWorkbook.Sheets("DebugLog")
Set wsG = ThisWorkbook.Sheets("Greeks_History") ' optional
On Error GoTo 0
On Error GoTo ErrorHandler
Set wsR = ThisWorkbook.Sheets("RR5")
Set wsH = ThisWorkbook.Sheets("HistoryLog")

' Target row: if today's row already exists, OVERWRITE it (refresh)
' instead of skipping. This keeps one row per day but lets repeated
' saves during the same day update MV / Greeks / PnL in place.
Dim lastRowH As Long
lastRowH = wsH.Cells(wsH.Rows.count, 1).End(xlUp).row

Dim isUpdate As Boolean: isUpdate = False
If lastRowH >= 2 Then
    If IsDate(wsH.Cells(lastRowH, 1).Value) Then

```

```

        If DateValue(wsH.Cells(lastRowH, 1).Value) = Date Then isUpdate = True
    End If
End If

Dim nextH As Long, nextG As Long, prevRow As Long
If isUpdate Then
    nextH = lastRowH      ' overwrite today's row
    prevRow = lastRowH - 1 ' compare against the prior day
Else
    nextH = lastRowH + 1  ' append a new row
    prevRow = lastRowH    ' compare against the last (older) row
End If

If Not wsG Is Nothing Then
    Dim lastG As Long: lastG = wsG.Cells(wsG.Rows.count, 1).End(xlUp).row
    If isUpdate And lastG >= 2 Then nextG = lastG Else nextG = lastG + 1
End If

' --- Read current RR5 summary values ---
' Force a recalc first: ShowTransactionForm runs with Calculation=Manual,
' so the summary formulas (BI MV, LI Delta, ...) would otherwise still
' hold pre-FetchAllPrices values and we'd log a stale / zero snapshot.
On Error Resume Next
wsR.Calculate
On Error GoTo ErrHandler
Dim curMV As Double: curMV = ToDb1(wsR.Range("B1").Value)      ' Total Market Value
Dim netDelta As Double: netDelta = ToDb1(wsR.Range("L1").Value) ' Net Delta
Dim netGamma As Double: netGamma = ToDb1(wsR.Range("N1").Value) ' Net Gamma
Dim netTheta As Double: netTheta = ToDb1(wsR.Range("J2").Value) ' Net Theta/Day
Dim netVega As Double: netVega = ToDb1(wsR.Range("L2").Value)  ' Net Vega
Dim r1PnL As Double: r1PnL = ToDb1(wsR.Range("H2").Value)     ' Realized PnL
Dim unr1PnL As Double: unr1PnL = ToDb1(wsR.Range("F2").Value)  ' Unrealized PnL

' --- Previous row values for % change ---
Dim prevMV As Double: prevMV = 0
Dim prevTX As Double: prevTX = 0
Dim prevSPY As Double: prevSPY = 0
If prevRow >= 2 Then
    If IsNumeric(wsH.Cells(prevRow, 2).Value) Then prevMV = CDb1(wsH.Cells(prevRow, 2).Value)
    If IsNumeric(wsH.Cells(prevRow, 8).Value) Then prevTX = CDb1(wsH.Cells(prevRow, 8).Value)
    If IsNumeric(wsH.Cells(prevRow, 10).Value) Then prevSPY = CDb1(wsH.Cells(prevRow, 10).Value)
End If

' --- Fetch benchmark prices (silently skip on network failure) ---
Dim txPx As Double, spyPx As Double
Application.StatusBar = "DailySnapshot: fetching ^TWII..."
On Error Resume Next
txPx = FetchPrice("^TWII")
spyPx = FetchPrice("SPY")
On Error GoTo ErrHandler
Application.StatusBar = False

' --- IV average across active options + warrants (col S=19) ---
Dim ivSum As Double, ivCnt As Long, i As Long, ivV As Variant
For i = 19 To 26 ' Options
    If Trim(CStr(wsR.Cells(i, 1).Value)) <> "" Then
        ivV = wsR.Cells(i, 19).Value
        If IsNumeric(ivV) And CDb1(ivV) > 0 Then ivSum = ivSum + CDb1(ivV): ivCnt = ivCnt + 1
    End If
Next i
For i = 31 To 38 ' Warrants
    If Trim(CStr(wsR.Cells(i, 1).Value)) <> "" Then
        ivV = wsR.Cells(i, 19).Value
        If IsNumeric(ivV) And CDb1(ivV) > 0 Then ivSum = ivSum + CDb1(ivV): ivCnt = ivCnt + 1
    End If
Next i
Dim ivAvg As Double
If ivCnt > 0 Then ivAvg = ivSum / ivCnt

' --- PortRetChg%: day-over-day MV change ---
Dim portRetChg As Double

```

```

If prevMV > 0 And curMV > 0 Then portRetChg = (curMV - prevMV) / prevMV

' --- PortRet%: cumulative since first snapshot (row 2) ---
Dim portRet As Double
If lastRowH >= 2 And IsNumeric(wsH.Cells(2, 2).Value) Then
    Dim firstMV As Double: firstMV = CDBl(wsH.Cells(2, 2).Value)
    If firstMV > 0 And curMV > 0 Then portRet = (curMV - firstMV) / firstMV
End If

' --- Summary text ---
Dim posCount As Variant: posCount = wsR.Range("P1").Value
Dim summaryTxt As String
summaryTxt = "Pos=" & IIf(IsNumeric(posCount), CStr(CLng(posCount)), "?") & _
    " MV=" & Format(curMV, "#,##0") & _
    " UnrlPnL=" & IIf(unrlPnL >= 0, "+", "") & Format(unrlPnL, "#,##0") & _
    " RlPnL=" & IIf(rlPnL >= 0, "+", "") & Format(rlPnL, "#,##0") & _
    IIf(ivAvg > 0, " IV=" & Format(ivAvg, "0.0%"), "")

' HistoryLog row -- all 16 columns
With wsH
    .Cells(nextH, 1).Value = Now()                ' A Date
    .Cells(nextH, 2).Value = curMV                ' B TotalMarketValue
    .Cells(nextH, 3).Value = unrlPnL + rlPnL       ' C TotalCumulativePnL
    .Cells(nextH, 4).Value = netDelta              ' D NetDelta
    .Cells(nextH, 5).Value = netGamma              ' E NetGamma
    .Cells(nextH, 6).Value = netTheta              ' F NetTheta
    .Cells(nextH, 7).Value = netVega               ' G NetVega
    If txPx > 0 Then .Cells(nextH, 8).Value = txPx  ' H Price(TX)
    If txPx > 0 And prevTX > 0 Then _
        .Cells(nextH, 9).Value = (txPx - prevTX) / prevTX ' I Ret%(TX)
    If spyPx > 0 Then .Cells(nextH, 10).Value = spyPx ' J Price(SPY)
    If spyPx > 0 And prevSPY > 0 Then _
        .Cells(nextH, 11).Value = (spyPx - prevSPY) / prevSPY ' K Ret%(SPY)
    .Cells(nextH, 12).Value = rlPnL                ' L RealizedPnL
    .Cells(nextH, 13).Value = portRet              ' M PortRet%
    .Cells(nextH, 14).Value = portRetChg           ' N PortRetChg%
    If ivAvg > 0 Then .Cells(nextH, 15).Value = ivAvg ' O IV_Avg
    .Cells(nextH, 16).Value = summaryTxt           ' P Summary

    ' Number formats
    .Cells(nextH, 1).NumberFormat = "yyyy/m/d hh:mm"
    .Cells(nextH, 2).NumberFormat = "#,##0"
    .Cells(nextH, 3).NumberFormat = "#,##0;(#,##0)"
    .Cells(nextH, 4).NumberFormat = "+0.0000;-0.0000;0"
    .Cells(nextH, 5).NumberFormat = "0.000000"
    .Cells(nextH, 6).NumberFormat = "0.0000"
    .Cells(nextH, 7).NumberFormat = "0.0000"
    .Cells(nextH, 8).NumberFormat = "#,##0.00"
    .Cells(nextH, 9).NumberFormat = "0.00%"
    .Cells(nextH, 10).NumberFormat = "#,##0.00"
    .Cells(nextH, 11).NumberFormat = "0.00%"
    .Cells(nextH, 12).NumberFormat = "#,##0;(#,##0)"
    .Cells(nextH, 13).NumberFormat = "0.00%"
    .Cells(nextH, 14).NumberFormat = "+0.00%;-0.00%;0.00%"
    .Cells(nextH, 15).NumberFormat = "0.00%"
End With

' Greeks_History row (optional sheet)
If Not wsG Is Nothing Then
    wsG.Cells(nextG, 1).Value = Now()
    wsG.Cells(nextG, 3).Value = netDelta
    wsG.Cells(nextG, 4).Value = netGamma
    wsG.Cells(nextG, 5).Value = netTheta
    wsG.Cells(nextG, 6).Value = netVega
End If

LogDebug wsD, "DailySnapshot", "OK", _
    IIf(isUpdate, "UPDATED row " & nextH, "APPENDED row " & nextH) & _
    " | MV=" & curMV & " TX=" & txPx & " SPY=" & spyPx & _
    " IV_Avg=" & Format(ivAvg, "0.0%")
Exit Sub

```

```

ErrorHandler:
    Application.StatusBar = False
    LogDebug wsD, "DailySnapshot", "ERROR", Err.Description
End Sub

Public Sub ExpiryAlert()
    ' Scans DTE columns across all three sections and alerts if any
    ' <= DTE_WARN_DAYS. Bug B2 fix: row ranges now read from _Config
    ' (FUT/OPT/WRT_SEC_START + INST_COUNT) instead of hardcoded.
    On Error GoTo ErrorHandler
    Dim wsR As Worksheet, wsC As Worksheet, wsD As Worksheet
    Set wsR = ThisWorkbook.Sheets("RR5")
    Set wsC = ThisWorkbook.Sheets("_Config")
    Set wsD = ThisWorkbook.Sheets("DebugLog")

    Dim warnDays As Long
    warnDays = CLng(GetConfig(wsC, "DTE_WARN_DAYS"))
    If warnDays = 0 Then warnDays = 5

    Dim alertMsg As String
    alertMsg = ""

    ' Load section ranges from _Config
    Dim futStart As Long, futCount As Long
    Dim optStart As Long, optCount As Long
    Dim wrtStart As Long, wrtCount As Long
    futStart = CLng(GetConfig(wsC, "FUT_SEC_START")): futCount = CLng(GetConfig(wsC, "FUT_INST_COUNT"))
    optStart = CLng(GetConfig(wsC, "OPT_SEC_START")): optCount = CLng(GetConfig(wsC, "OPT_INST_COUNT"))
    wrtStart = CLng(GetConfig(wsC, "WRT_SEC_START")): wrtCount = CLng(GetConfig(wsC, "WRT_INST_COUNT"))

    ' Advance past header row(s) if the start cell isn't actual data.
    ' STRIKE col differs by section: futures has no strike (use A=contract),
    ' options strike is col 9, warrants strike is col 11.
    Dim hg As Long
    hg = 0: Do While Len(CStr(wsR.Cells(futStart, 1).Value)) = 0 And IsEmpty(wsR.Cells(futStart, 4).Value) And hg < 3
        futStart = futStart + 1: hg = hg + 1
    Loop
    hg = 0: Do While Not IsNumeric(wsR.Cells(optStart, 9).Value) And hg < 5
        optStart = optStart + 1: hg = hg + 1
    Loop
    hg = 0: Do While Not IsNumeric(wsR.Cells(wrtStart, 11).Value) And hg < 5
        wrtStart = wrtStart + 1: hg = hg + 1
    Loop

    Call ScanExpirySection(wsR, futStart, futCount, "FUTURES", warnDays, alertMsg)
    Call ScanExpirySection(wsR, optStart, optCount, "OPTIONS", warnDays, alertMsg)
    Call ScanExpirySection(wsR, wrtStart, wrtCount, "WARRANTS", warnDays, alertMsg)

    If alertMsg <> "" Then
        MsgBox "EXPIRY ALERT - positions expiring within " & warnDays & " days:" & vbCrLf & vbCrLf & alertMsg, _
            vbExclamation, "RR5 Expiry Alert"
    End If
    LogDebug wsD, "ExpiryAlert", "OK", IIf(alertMsg = "", "No alerts", "ALERTED: " & alertMsg)
Exit Sub

ErrorHandler:
    LogDebug wsD, "ExpiryAlert", "ERROR", Err.Description
End Sub

Private Sub ScanExpirySection(wsR As Worksheet, startRow As Long, count As Long, _
    label As String, warnDays As Long, ByRef alertMsg As String)
    Dim i As Long, dte As Variant, contract As String
    For i = startRow To startRow + count - 1
        dte = wsR.Cells(i, 5).Value
        contract = CStr(wsR.Cells(i, 1).Value)
        If contract = "" Then GoTo NextI
        If IsNumeric(dte) Then
            If CLng(dte) > 0 And CLng(dte) <= warnDays Then
                alertMsg = alertMsg & label & ": " & contract & " DTE=" & dte & vbCrLf
            End If
        End If
    NextI:

```

```
Next i
End Sub
```

```
' --- Timer-based auto-refresh (NextRefreshTime declared at top of module) ---
```

```
' =====
' ONE-SHOT: Update _Config to compact 8-rows-per-section layout
' =====
' Futures data rows 7-14 (header 6, subtotal 15)
' Options data rows 19-26 (banner 17, header 18, subtotal 27)
' Warrants data rows 31-38 (banner 29, header 30, subtotal 39)
' GrandTotal row 41
Public Sub Setup_CompactLayout_Config()
    On Error GoTo Fail
    Dim wsC As Worksheet
    Set wsC = ThisWorkbook.Sheets("_Config")

    Call SetConfigValue("FUT_SEC_START", 6) ' header row; ExpiryAlert auto-advances
    Call SetConfigValue("FUT_INST_COUNT", 8)
    Call SetConfigValue("OPT_SEC_START", 18)
    Call SetConfigValue("OPT_INST_COUNT", 8)
    Call SetConfigValue("WRT_SEC_START", 30)
    Call SetConfigValue("WRT_INST_COUNT", 8)

    MsgBox "_Config updated to compact layout:" & vbCrLf & _
        " FUT_SEC_START=6 FUT_INST_COUNT=8 (data 7-14)" & vbCrLf & _
        " OPT_SEC_START=18 OPT_INST_COUNT=8 (data 19-26)" & vbCrLf & _
        " WRT_SEC_START=30 WRT_INST_COUNT=8 (data 31-38)", _
        vbInformation, "Compact Layout Config"
    Exit Sub
```

Fail:

```
MsgBox "Setup_CompactLayout_Config failed: " & Err.Description, vbCritical
End Sub
```

```
Public Sub AutoRefresh_Start()
    Dim wsC As Worksheet
    On Error Resume Next
    Set wsC = ThisWorkbook.Sheets("_Config")
    Dim mins As Long
    mins = CLng(wsC.Cells(7, 2).Value) ' RefreshInterval_Min row
    If mins <= 0 Then mins = 30
    NextRefreshTime = Now() + TimeSerial(0, mins, 0)
    Application.OnTime NextRefreshTime, "RR5_Core.AutoRefresh_Tick"
End Sub
```

```
Public Sub AutoRefresh_Stop()
    On Error Resume Next
    Application.OnTime NextRefreshTime, "RR5_Core.AutoRefresh_Tick", , False
End Sub
```

```
Public Sub AutoRefresh_Tick()
    Call RefreshGreeks
    Call ExpiryAlert
    ' Pull aggregates from Realized / IVTracker / HistoryLog / Greeks_History
    ' into RR5 rows 3-4. Defensive: if RR5_Dashboard module not present
    ' yet, skip silently so timer still runs.
    On Error Resume Next
    Application.Run "RR5_Dashboard.DashboardRefresh"
    On Error GoTo 0
    Call AutoRefresh_Start ' reschedule
End Sub
```

```
' =====
' TRANSACTION ENTRY FORM HELPERS
' =====
```

```
Public Function GenerateTxID() As String
    ' Format: T-YYYYMMDD-HHMMSS
    GenerateTxID = "T-" & Format(Now(), "yyyymmdd-hhnnss")
End Function
```

```

Public Sub AppendTransaction(rowData As Variant)
    ' Appends a variable-length 1-based array to the Transactions sheet,
    ' then auto-chains: RebuildPositionsFromTransactions -> FetchAllPrices
    ' (the latter is invoked transitively by Rebuild's last step).
    ,
    ' Transactions sheet has 24 columns:
    ' 1=ID 2=Date 3=Ticker 4=Action 5=Type 6=Strike 7=Expiry
    ' 8=Qty 9=Mult 10=Price 11=Fee 12=Tax 13=NetAmt
    ' 14=Sector 15=Target 16=Strategy 17=IV 18=Delta 19=Beta 20=Broker
    ' 21=Underlying 22=Gamma 23=Theta 24=Conv_Ratio
    Dim wsT As Worksheet, wsD As Worksheet
    On Error Resume Next
    Set wsD = ThisWorkbook.Sheets("DebugLog") ' optional; LogDebug guards Nothing
    On Error GoTo 0
    On Error GoTo ErrHandler
    Set wsT = ThisWorkbook.Sheets("Transactions")

    Dim nextRow As Long
    nextRow = wsT.Cells(wsT.Rows.count, 1).End(xlUp).row + 1
    If nextRow < 2 Then nextRow = 2

    Dim i As Long
    For i = 1 To UBound(rowData) ' dynamic: handles 20-col or 23-col arrays
        ' Col 3 (Ticker) -- force text format so leading zeros survive
        ' (e.g. warrant code "066669" wouldn't drop to 66669)
        If i = 3 Then wsT.Cells(nextRow, i).NumberFormat = "@"
        wsT.Cells(nextRow, i).Value = rowData(i)
    Next i

    LogDebug wsD, "AppendTransaction", "OK", "Row " & nextRow & " | " & rowData(1)

    ' Chain: rebuild positions -> auto fetch prices -> refresh greeks.
    ' RebuildPositionsFromTransactions itself calls FetchAllPrices(silent:=True)
    ' which calls RefreshGreeks, so one call cascades the whole pipeline.
    Call RebuildPositionsFromTransactions
    Exit Sub
ErrHandler:
    LogDebug wsD, "AppendTransaction", "ERROR", Err.Description
    MsgBox "Error saving transaction: " & Err.Description, vbCritical
End Sub

Public Sub ShowTransactionForm()
    ' Entry point: opens the UserForm. Called by toolbar button or Ctrl+Shift+T.
    ,
    ' We freeze calc + events while the form is open, otherwise volatile
    ' worksheet formulas (option pricing, third-party broker add-ins, etc.)
    ' can fire repeatedly while the user is still mid-entry and spam errors.
    On Error GoTo NoForm

    Dim oldCalc As XlCalculation, oldEvt As Boolean, oldScr As Boolean
    oldCalc = Application.Calculation
    oldEvt = Application.EnableEvents
    oldScr = Application.ScreenUpdating
    Application.Calculation = xlCalculationManual
    Application.EnableEvents = False
    Application.ScreenUpdating = True ' keep UI responsive

    frmTransaction.Show vbModal ' blocks Excel until form closes

    ' Restore state -- always run, even if user cancelled
    Application.Calculation = oldCalc
    Application.EnableEvents = oldEvt
    Application.ScreenUpdating = oldScr
    ' Force one clean recalc now that everything is back to normal
    Application.CalculateFull
    Exit Sub
NoForm:
    Application.Calculation = xlCalculationAutomatic
    Application.EnableEvents = True
    Application.ScreenUpdating = True
    MsgBox "frmTransaction UserForm not found." & vbCrLf & _

```

"Please create it via Alt+F11 -> Insert -> UserForm." & vbCrLf & _
 "See RR5_UserForm_Guide.md for instructions.", vbExclamation

End Sub

```
'
' =====
' PASTE THE FOLLOWING INTO ThisWorkbook MODULE (not here):
'
' Private Sub Workbook_Open()
'     Call RR5_Core.ExpiryAlert
'     Call RR5_Core.RefreshGreeks
'     Call RR5_Core.AutoRefresh_Start
'     Application.OnKey "^+t", "RR5_Core.ShowTransactionForm" ' Ctrl+Shift+T
' End Sub
'
' Private Sub Workbook_BeforeClose(Cancel As Boolean)
'     Call RR5_Core.AutoRefresh_Stop
'     Application.OnKey "^+t" ' release shortcut
' End Sub
'
' =====
```

' REBUILD POSITIONS FROM TRANSACTIONS

```
'
' Reads ALL rows in the Transactions sheet, groups by position key
' (Ticker + Type + Strike + Expiry), nets Buy/Sell quantities, and
' writes active (non-zero) positions to the three RR5 sections:
'
' Future           => Futures  rows 7-26 (cols A/C-D/F/H-K/R)
' Call / Put       => Options  rows 31-70 (cols A/C-D/F/H-N/S-T/AB)
' Call Warrant / Put Warrant
' Bull Cert / Bear Cert => Warrants rows 75-104 (cols A/C-D/F-G/K-N/S-T)
' Equity / ETF      => skipped (no section in RR5)
'
' Action handling:
' Buy              => +qty to net; accumulates weighted-average cost
' Sell             => -qty from net
' AdjustCost       => overrides entry price to transaction price
' Expire /
' Exercise         => zeros out position (closed)
'
' NOTE: After running, manually update in RR5:
' LAST / UNDL LAST (current market prices)
' CONV RATIO       (warrants: conversion ratio)
' MARGIN REQ       (futures: per-contract margin)
' Then re-run RefreshGreeks to update all Greeks.
```

```
Public Sub RebuildPositionsFromTransactions()
    Dim wsT As Worksheet, wsR As Worksheet, wsD As Worksheet
    On Error Resume Next
    Set wsD = ThisWorkbook.Sheets("DebugLog") ' optional; LogDebug guards Nothing
    On Error GoTo 0
    On Error GoTo ErrHandler
    Set wsT = ThisWorkbook.Sheets("Transactions")
    Set wsR = ThisWorkbook.Sheets("RR5")
```

```
' -- Step 1: Read all Transactions -----
' Transactions sheet -- 24 columns:
' 1=Transaction_ID 2=Date      3=Ticker    4=Action  5=Type
' 6=Strike         7=Expiry    8=Shares_Contracts 9=Multiplier
' 10=Price         11=Fee      12=Tax      13=Net_Amount
' 14=Sector        15=Target   16=Strategy_Tag
' 17=IV_At_Entry   18=Delta_At_Entry 19=Beta    20=Broker
' 21=Underlying    22=Gamma    23=Theta    24=Conv_Ratio
```

```
Dim lastRow As Long
lastRow = wsT.Cells(wsT.Rows.count, 1).End(xlUp).row
If lastRow < 2 Then
    MsgBox "Transactions sheet has no data.", vbInformation
    Exit Sub
End If
```

```

' Dictionary: posKey => 18-element Variant array (indices 0-17)
' 0=posKey 1=ticker 2=txType 3=strike 4=expiry
' 5=netQty 6=buyCostSum 7=buyQtySum
' 8=multiplier 9=broker 10=entryDate
' 11=strategy 12=iv 13=delta
' 14=underlying 15=gamma 16=theta 17=convRatio 18=issuer
Dim dict As Object
Set dict = CreateObject("Scripting.Dictionary")

```

```
Dim e(18) As Variant ' reused for each new entry
```

```
Dim i As Long
```

```
For i = 2 To lastRow
```

```
    Dim txDate As Variant
```

```
    Dim ticker As String, action As String, txType As String
```

```
    Dim strike As Variant, expiry As Variant
```

```
    Dim qty As Double, txMult As Double, price As Double
```

```
    Dim strategy As String, iv As Variant, delta As Variant
```

```
    Dim broker As String, underlying As String
```

```
    Dim gamma As Variant, theta As Variant, convRatio As Variant
```

```
    Dim issuer As String
```

```
    txDate = wsT.Cells(i, 2).Value
```

```
    ticker = Trim(CStr(wsT.Cells(i, 3).Value))
```

```
    action = UCase(Trim(CStr(wsT.Cells(i, 4).Value)))
```

```
    txType = Trim(CStr(wsT.Cells(i, 5).Value))
```

```
    strike = wsT.Cells(i, 6).Value
```

```
    expiry = wsT.Cells(i, 7).Value
```

```
    qty = CDBl(wsT.Cells(i, 8).Value)
```

```
    txMult = CDBl(wsT.Cells(i, 9).Value)
```

```
    If txMult = 0 Then txMult = 1
```

```
    price = CDBl(wsT.Cells(i, 10).Value)
```

```
    strategy = Trim(CStr(wsT.Cells(i, 16).Value))
```

```
    iv = wsT.Cells(i, 17).Value
```

```
    delta = wsT.Cells(i, 18).Value
```

```
    broker = Trim(CStr(wsT.Cells(i, 20).Value))
```

```
    underlying = Trim(CStr(wsT.Cells(i, 21).Value)) ' col U
```

```
    gamma = wsT.Cells(i, 22).Value ' col V
```

```
    theta = wsT.Cells(i, 23).Value ' col W
```

```
    convRatio = wsT.Cells(i, 24).Value ' col X Conv_Ratio (warrants)
```

```
    issuer = Trim(CStr(wsT.Cells(i, 25).Value)) ' col Y Issuer (warrants)
```

```
If ticker = "" Then GoTo SkipRow
```

```
' Build position key by instrument type
```

```
Dim posKey As String
```

```
Select Case txType
```

```
    Case "Future"
```

```
        posKey = "FUT|" & ticker & "|" & CStr(expiry)
```

```
    Case "Call", "Put"
```

```
        posKey = "OPT|" & ticker & "|" & txType & "|" & CStr(strike) & "|" & CStr(expiry)
```

```
    Case "Call Warrant", "Put Warrant", "Bull Cert", "Bear Cert"
```

```
        posKey = "WRT|" & ticker
```

```
    Case Else
```

```
        GoTo SkipRow ' Equity / ETF / unknown -- no section in RR5
```

```
End Select
```

```
' Initialise dict entry on first encounter
```

```
If Not dict.Exists(posKey) Then
```

```
    Erase e
```

```
    e(0) = posKey: e(1) = ticker: e(2) = txType
```

```
    e(3) = strike: e(4) = expiry
```

```
    e(5) = 0: e(6) = 0: e(7) = 0
```

```
    e(8) = txMult: e(9) = broker: e(10) = txDate
```

```
    e(11) = strategy: e(12) = iv: e(13) = delta
```

```
    e(14) = underlying: e(15) = gamma: e(16) = theta
```

```
    e(17) = convRatio
```

```
    e(18) = issuer
```

```
    dict(posKey) = e
```

```
End If
```



```

Dim p() As Variant
p = dict(posKey)

' --- Position state update -----
' p(5) = signed netQty (+ long, - short)
' p(6) = avgCost of currently OPEN side (positive, 0 if flat)
' p(7) = 0 (unused after the round-trip bug fix; previously was
'         cumulative buy qty which incorrectly persisted across
'         full close/reopen cycles)
Dim curNet As Double: curNet = Cdbl(p(5))
Dim curAvg As Double: curAvg = Cdbl(p(6))
Dim resQty As Double

Select Case action
Case "BUY"
    If curNet >= 0 Then
        ' Add to (or open) long
        If curNet = 0 Then
            curAvg = price
            p(10) = txDate
        Else
            curAvg = (curNet * curAvg + qty * price) / (curNet + qty)
        End If
        curNet = curNet + qty
    Else
        ' Cover short (curNet < 0)
        If qty >= -curNet Then
            ' Full cover; residual reopens long
            resQty = qty + curNet      ' qty - lshortl
            curNet = 0: curAvg = 0
            p(10) = Empty
            If resQty > 0 Then
                curNet = resQty
                curAvg = price
                p(10) = txDate
            End If
        Else
            ' Partial cover; avg short cost unchanged
            curNet = curNet + qty
        End If
    End If

Case "SELL"
    If curNet > 0 Then
        ' Close long
        If qty >= curNet Then
            ' Full close; residual opens short
            resQty = qty - curNet
            curNet = 0: curAvg = 0
            p(10) = Empty
            If resQty > 0 Then
                curNet = -resQty
                curAvg = price
                p(10) = txDate
            End If
        Else
            ' Partial close; avg long cost unchanged
            curNet = curNet - qty
        End If
    Else
        ' Add to (or open) short
        If curNet = 0 Then
            curAvg = price
            p(10) = txDate
        Else
            curAvg = (-curNet * curAvg + qty * price) / (-curNet + qty)
        End If
        curNet = curNet - qty
    End If

```

```

Case "ADJUSTCOST"
    If curNet <> 0 Then curAvg = price

Case "EXPIRE", "EXERCISE"
    curNet = 0: curAvg = 0
    p(10) = Empty
End Select

p(5) = curNet
p(6) = curAvg
p(7) = 0 ' deprecated

' Keep most-recent non-empty metadata
If strategy <> "" Then p(11) = strategy
If CStr(iv) <> "" And CStr(iv) <> "0" Then p(12) = iv
If CStr(delta) <> "" And CStr(delta) <> "0" Then p(13) = delta
If broker <> "" Then p(9) = broker
If txMult > 0 Then p(8) = txMult
If underlying <> "" Then p(14) = underlying
If CStr(gamma) <> "" And CStr(gamma) <> "0" Then p(15) = gamma
If CStr(theta) <> "" And CStr(theta) <> "0" Then p(16) = theta
' Conv Ratio: keep first non-zero value (fixed per warrant code)
If CStr(convRatio) <> "" And Cdbl(convRatio) > 0 Then
    If CStr(p(17)) = "" Or Cdbl(p(17)) = 0 Then p(17) = convRatio
End If
If issuer <> "" Then p(18) = issuer
dict(posKey) = p

```

SkipRow:

Next i

```

' -- Step 2: Clear input cells in all 3 sections -----
' Only input columns are cleared; formula columns (E,M,N,O,Q,S,V etc.)
' are left untouched so they recalculate automatically.

```

Dim colIdx As Variant

```

' COMPACT LAYOUT (8 rows per section):
' Futures rows 7-14 (header=6, subtotal=15)
' Options rows 19-26 (banner=17, header=18, subtotal=27)
' Warrants rows 31-38 (banner=29, header=30, subtotal=39)
' GrandTotal at row 41

```

' Futures input cols: A B C D F G H I J K L P R T U W

Dim futInp As Variant

futInp = Array(1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 16, 18, 20, 21, 23)

For Each colIdx In futInp

wsR.Range(wsR.Cells(7, CLng(colIdx)), wsR.Cells(14, CLng(colIdx))).ClearContents

Next colIdx

' Options input cols: A B C D F G H I J K L M N O S T U V W X AB

Dim optInp As Variant

optInp = Array(1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 19, 20, 21, 22, 23, 24, 28)

For Each colIdx In optInp

wsR.Range(wsR.Cells(19, CLng(colIdx)), wsR.Cells(26, CLng(colIdx))).ClearContents

Next colIdx

' Warrants input cols: A B C D F G H I J K L M N O S T

Dim wrtInp As Variant

wrtInp = Array(1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 19, 20)

For Each colIdx In wrtInp

wsR.Range(wsR.Cells(31, CLng(colIdx)), wsR.Cells(38, CLng(colIdx))).ClearContents

Next colIdx

' -- Step 3: Write active positions -----

Dim futRow As Long, optRow As Long, wrtRow As Long

futRow = 7: optRow = 19: wrtRow = 31

Dim cntFut As Long, cntOpt As Long, cntWrt As Long

cntFut = 0: cntOpt = 0: cntWrt = 0

' Bug B3 fix: sort dict keys for stable display order across rebuilds.

' FUT section sorted by ticker -> expiry; OPT by underlying -> expiry ->

```

' type (Call before Put) -> strike; WRT by ticker.
Dim sortedKeys() As String
sortedKeys = SortPositionKeys(dict)

Dim k As Variant
Dim ki As Long
For ki = LBound(sortedKeys) To UBound(sortedKeys)
    k = sortedKeys(ki)
    Dim pos() As Variant
    pos = dict(k)

    Dim netQty As Double
    netQty = Cdbl(pos(5))
    If Abs(netQty) < 0.000001 Then GoTo SkipPos ' closed/expired

    ' Avg cost of currently OPEN position (resets on round-trip close).
    ' pos(6) now stores avgCost directly -- not cumulative / cumQty.
    Dim entryPx As Double
    entryPx = Cdbl(pos(6))

    Dim ks As String: ks = CStr(k)

    If Left(ks, 4) = "FUT1" Then
        ' -- Futures -----
        If futRow > 14 Then GoTo SkipPos ' section full (8 rows, 7-14)
        wsR.Cells(futRow, 1).Value = pos(1) ' A CONTRACT
        wsR.Cells(futRow, 3).Value = pos(10) ' C ENTRY DT
        wsR.Cells(futRow, 4).Value = pos(4) ' D EXPIRY
        wsR.Cells(futRow, 6).Value = pos(9) ' F BROKER
        wsR.Cells(futRow, 8).Value = IIf(netQty > 0, "Long", "Short") ' H DIRECTION
        wsR.Cells(futRow, 9).Value = Abs(netQty) ' I CONTRACTS
        wsR.Cells(futRow, 10).Value = Cdbl(pos(8)) ' J MULTIPLIER
        wsR.Cells(futRow, 11).Value = entryPx ' K ENTRY PX
        wsR.Cells(futRow, 18).Value = 1 ' R DELTA = 1
        futRow = futRow + 1: cntFut = cntFut + 1

    ElseIf Left(ks, 4) = "OPT1" Then
        ' -- Options -----
        If optRow > 26 Then GoTo SkipPos ' section full (8 rows, 19-26)
        ' CONTRACT label: e.g. "TXO Call 18000"
        wsR.Cells(optRow, 1).Value = pos(1) & " " & pos(2) & " " & CStr(pos(3))
        wsR.Cells(optRow, 3).Value = pos(10) ' C ENTRY DT
        wsR.Cells(optRow, 4).Value = pos(4) ' D EXPIRY
        wsR.Cells(optRow, 6).Value = pos(9) ' F BROKER
        wsR.Cells(optRow, 8).Value = pos(2) ' H TYPE (Call/Put)
        wsR.Cells(optRow, 9).Value = Cdbl(pos(3)) ' I STRIKE
        wsR.Cells(optRow, 10).Value = pos(14) ' J UNDERLYING (name)
        ' CONTRACTS is signed: positive=long, negative=short
        ' NET DELTA formula =IF(H="Put",-L*M*ABS(T),L*M*T) handles sign correctly
        wsR.Cells(optRow, 12).Value = netQty ' L CONTRACTS (signed)
        wsR.Cells(optRow, 13).Value = Cdbl(pos(8)) ' M MULTIPLIER
        wsR.Cells(optRow, 14).Value = entryPx ' N ENTRY PREM
        wsR.Cells(optRow, 19).Value = pos(12) ' S IV
        wsR.Cells(optRow, 20).Value = pos(13) ' T DELTA
        wsR.Cells(optRow, 21).Value = pos(15) ' U GAMMA
        wsR.Cells(optRow, 22).Value = pos(16) ' V THETA/DAY
        wsR.Cells(optRow, 28).Value = pos(11) ' AB STRATEGY TAG
        optRow = optRow + 1: cntOpt = cntOpt + 1

    ElseIf Left(ks, 4) = "WRT1" Then
        ' -- Warrants -----
        If wrtRow > 38 Then GoTo SkipPos ' section full (8 rows, 31-38)
        ' Pad warrant code to 6 digits (preserves leading zeros lost
        ' when Transactions stored ticker as Number, e.g. 66669 -> 066669)
        Dim wCodeStr As String: wCodeStr = CStr(pos(1))
        If IsNumeric(wCodeStr) And Len(wCodeStr) <= 6 Then
            wCodeStr = Format(val(wCodeStr), "000000")
        End If
        wsR.Cells(wrtRow, 1).NumberFormat = "@"
        wsR.Cells(wrtRow, 1).Value = wCodeStr ' A CODE
        wsR.Cells(wrtRow, 3).Value = pos(10) ' C ENTRY DT

```

```

wsR.Cells(wrtRow, 4).Value = pos(4) ' D EXPIRY
wsR.Cells(wrtRow, 6).Value = pos(9) ' F BROKER
wsR.Cells(wrtRow, 7).Value = pos(2) ' G TYPE
wsR.Cells(wrtRow, 8).Value = CStr(pos(18)) ' H ISSUER
wsR.Cells(wrtRow, 9).Value = pos(14) ' I UNDERLYING (name)
wsR.Cells(wrtRow, 11).Value = CDb1(pos(3)) ' K STRIKE
' L CONV RATIO -- required for Warrant_Delta = BS_Delta / convRatio
If CStr(pos(17)) <> "" And CDb1(pos(17)) > 0 Then
    wsR.Cells(wrtRow, 12).Value = CDb1(pos(17)) ' L CONV RATIO
End If
wsR.Cells(wrtRow, 13).Value = Abs(netQty) ' M SHARES (always +)
wsR.Cells(wrtRow, 14).Value = entryPx ' N ENTRY PX
wsR.Cells(wrtRow, 19).Value = pos(12) ' S IV
wsR.Cells(wrtRow, 20).Value = pos(13) ' T DELTA
wrtRow = wrtRow + 1: cntWrt = cntWrt + 1
End If

SkipPos:
Next ki

' -- Step 4: Fetch live prices + Greeks + Realize -----
' FetchAllPrices internally calls:
' RefreshGreeks (per-leg Greeks from BS)
' RealizeClosedTransactions (derive realized PnL into Realized)
' so one call covers all three downstream steps.
Call FetchAllPrices(silent:=True)

' -- Step 5: Append today's snapshot to HistoryLog -----
' Runs after FetchAllPrices so UNDL LAST / Greeks / RealizedPnL are
' all current before the row is written. DailySnapshot skips if a row
' for today already exists (idempotent).
Call DailySnapshot

' -- Step 6: Refresh combined RR4+RR5 performance sheet + CLog -----
' Runs right after DailySnapshot so it reuses the wsR.Calculate that
' DailySnapshot just forced (RR5 summary cells are guaranteed fresh).
' Defensive: CombinedPerf module may not be imported into every copy
' of this workbook, so a missing module must not abort the rebuild.
On Error Resume Next
Application.Run "CombinedPerf.RefreshCombinedPerf"
On Error GoTo ErrHandler

LogDebug wsD, "RebuildPositionsFromTransactions", "OK", _
    "Fut=" & cntFut & " Opt=" & cntOpt & " Wrt=" & cntWrt

' Count realized rows for the summary box
Dim rlCount As Long
On Error Resume Next
rlCount = ThisWorkbook.Sheets("Realized"). _
    Cells(ThisWorkbook.Sheets("Realized").Rows.count, 1).End(xlUp).row - 1
On Error GoTo ErrHandler
If rlCount < 0 Then rlCount = 0

MsgBox "Rebuild + Fetch + Realize + Snapshot complete!" & vbCrLf & vbCrLf & _
    "Open positions:" & vbCrLf & _
    " Futures : " & cntFut & vbCrLf & _
    " Options : " & cntOpt & vbCrLf & _
    " Warrants: " & cntWrt & vbCrLf & vbCrLf & _
    "Realized (from Transactions): " & rlCount & " row(s)" & vbCrLf & vbCrLf & _
    "Auto-filled:" & vbCrLf & _
    " - UNDL LAST / LAST / LAST PREM (TPEx + TWSE + Yahoo)" & vbCrLf & _
    " - Futures MARGIN REQ (TickerMap)" & vbCrLf & _
    " - Greeks (Delta/Gamma/Theta/Vega/Rho)" & vbCrLf & _
    " - Realized PnL (BUY/SELL/EXPIRE matching)" & vbCrLf & _
    " - HistoryLog snapshot (TX/SPY/IV_Avg/Summary)" & vbCrLf & _
    " - CombinedPerf + CLog row (RR4+RR5 combined)", _
    vbInformation, "RR5 Rebuild"
Exit Sub

ErrHandler:
LogDebug wsD, "RebuildPositionsFromTransactions", "ERROR", _
    "Row " & i & " | " & Err.Description

```

```

    MsgBox "Rebuild failed at row " & i & ": " & Err.Description, vbCritical
End Sub

```

```

' =====
' REALIZE CLOSED TRANSACTIONS
' -----
' Walks Transactions chronologically per position key (Ticker / Type /
' Strike / Expiry). Detects closure events and writes one row per
' realized batch to the Realized worksheet.
'
'   BUY    -> if currently SHORT, COVER (realize short PnL); excess opens long
'   SELL    -> if currently LONG, CLOSE (realize long PnL); excess opens short
'   EXPIRE  -> close ALL remaining at price (price=0 means worthless)
'   EXERCISE -> close ALL remaining at price (price = intrinsic value)
'
' Long-side PnL per unit = (sell_price - avg_long_cost)
' Short-side PnL per unit = (avg_short_cost - cover_price)
' Multiplied by qty * multiplier and converted to TWD (col I) when underlying is USD.
'
' Idempotent: clears Realized rows 2+ before re-deriving from full Transactions.
' =====

```

```

Public Sub RealizeClosedTransactions(Optional silent As Boolean = False)
    On Error GoTo Fail

```

```

    Dim wsT As Worksheet, wsRl As Worksheet, wsR As Worksheet, wsD As Worksheet
    Set wsT = ThisWorkbook.Sheets("Transactions")
    Set wsRl = ThisWorkbook.Sheets("Realized")
    Set wsR = ThisWorkbook.Sheets("RR5")
    Set wsD = ThisWorkbook.Sheets("DebugLog")

```

```

    Dim usdRate As Double
    usdRate = ToDb1(wsR.Range("D1").Value)
    If usdRate <= 0 Then usdRate = 31.6

```

```

    Dim lastRow As Long
    lastRow = wsT.Cells(wsT.Rows.count, 1).End(xlUp).row
    If lastRow < 2 Then
        If Not silent Then MsgBox "Transactions sheet is empty.", vbInformation
        Exit Sub
    End If

```

```

' --- Read all transactions into array ---
    Dim txns As Variant
    txns = wsT.Range("A2:X" & lastRow).Value
    Dim nTx As Long: nTx = UBound(txns, 1)

```

```

' --- Sort by date (col 2). Simple bubble sort -- small N. ---
    Dim ord() As Long: ReDim ord(1 To nTx)
    Dim i As Long, j As Long
    For i = 1 To nTx: ord(i) = i: Next i
    For i = 1 To nTx - 1
        For j = 1 To nTx - i
            Dim dA As Date, dB As Date
            On Error Resume Next
            dA = CDate(txns(ord(j), 2))
            dB = CDate(txns(ord(j + 1), 2))
            On Error GoTo Fail
            If dA > dB Then
                Dim tmp As Long: tmp = ord(j): ord(j) = ord(j + 1): ord(j + 1) = tmp
            End If
        Next j
    Next i

```

```

' --- Per-position state dict ---
' state array layout (Variant array, 0-based):
'   0=netQty (signed: + long, - short)
'   1=avgCost (always positive)
'   2=openDate (Date or Empty)
'   3=entryIV   4=entryDelta
'   5=ticker    6=txType   7=strike   8=expiry   9=broker
'   10=strategy 11=isUSD (Boolean)

```

```

Dim posState As Object: Set posState = CreateObject("Scripting.Dictionary")
Dim outRow As Long: outRow = 2
Dim realizedCount As Long: realizedCount = 0

' --- Clear existing Realized rows 2+ ---
Dim rlLast As Long
rlLast = wsR1.Cells(wsR1.Rows.count, 1).End(xlUp).row
If rlLast >= 2 Then wsR1.Range("A2:T" & rlLast).ClearContents

For i = 1 To nTx
    Dim r As Long: r = ord(i)
    Dim txDate As Date
    On Error Resume Next: txDate = CDate(txns(r, 2)): On Error GoTo Fail
    If txDate = 0 Then GoTo NextTx

    Dim ticker As String: ticker = Trim(CStr(txns(r, 3)))
    If ticker = "" Then GoTo NextTx
    ' Restore leading zeros if numeric warrant code lost them
    If IsNumeric(ticker) And Len(ticker) < 6 Then
        Dim numVal As Long: numVal = CLng(val(ticker))
        If numVal >= 1000 And numVal <= 999999 Then
            ticker = Format(numVal, "000000")
        End If
    End If

    Dim action As String: action = UCase(Trim(CStr(txns(r, 4))))
    Dim txType As String: txType = Trim(CStr(txns(r, 5)))
    Dim strike As Variant: strike = txns(r, 6)
    Dim expiry As Variant: expiry = txns(r, 7)
    Dim qty As Double: qty = ToDbl(txns(r, 8))
    Dim mult As Double: mult = ToDbl(txns(r, 9))
    If mult = 0 Then mult = 1
    Dim price As Double: price = ToDbl(txns(r, 10))
    Dim strategy As String: strategy = Trim(CStr(txns(r, 16)))
    Dim ivAtEntry As Double: ivAtEntry = ToDbl(txns(r, 17))
    Dim deltaAtEntry As Double: deltaAtEntry = ToDbl(txns(r, 18))
    Dim broker As String: broker = Trim(CStr(txns(r, 20)))
    Dim underly As String: underly = Trim(CStr(txns(r, 21)))

    If qty <= 0 Then GoTo NextTx

    ' --- Build position key ---
    Dim posKey As String
    Select Case txType
        Case "Future"
            posKey = "FUT|" & ticker & "|" & CStr(expiry) & "|" & broker
        Case "Call", "Put"
            posKey = "OPT|" & ticker & "|" & txType & "|" & CStr(strike) & "|" & CStr(expiry) & "|" & broker
        Case "Call Warrant", "Put Warrant", "Bull Cert", "Bear Cert"
            posKey = "WRT|" & ticker & "|" & broker
        Case Else
            posKey = "EQTY|" & ticker & "|" & broker
    End Select

    ' --- Determine USD vs TWD ---
    Dim isUSD As Boolean
    isUSD = NotTaiwanUnderlying(underly, ticker)

    ' --- Init state if new key ---
    If Not posState.Exists(posKey) Then
        Dim newS(11) As Variant
        newS(0) = 0#: newS(1) = 0#: newS(2) = Empty
        newS(3) = 0#: newS(4) = 0#
        newS(5) = ticker: newS(6) = txType
        newS(7) = strike: newS(8) = expiry
        newS(9) = broker: newS(10) = strategy
        newS(11) = isUSD
        posState(posKey) = newS
    End If

    Dim st As Variant: st = posState(posKey)

```

```

Dim netQty As Double: netQty = CDb1(st(0))
Dim avgCost As Double: avgCost = CDb1(st(1))

Select Case action
    Case "BUY"
        If netQty < 0 Then
            ' COVER short position
            Dim coverQty As Double
            coverQty = qty
            If coverQty > -netQty Then coverQty = -netQty

            Dim pnlCover As Double
            pnlCover = (avgCost - price) * coverQty * mult
            Call WriteRealizedRow(wsR1, outRow, st, coverQty, _
                                avgCost, price, pnlCover, _
                                txDate, ivAtEntry, deltaAtEntry, _
                                usdRate, strategy, "Cover")

            outRow = outRow + 1
            realizedCount = realizedCount + 1

            netQty = netQty + coverQty
            qty = qty - coverQty
            If Abs(netQty) < 0.000001 Then
                netQty = 0: avgCost = 0
                st(2) = Empty: st(3) = 0#: st(4) = 0#: st(10) = ""
            End If
        End If
        If qty > 0 Then
            ' Adds to (or opens) long
            If netQty = 0 Then
                avgCost = price
                st(2) = txDate: st(3) = ivAtEntry: st(4) = deltaAtEntry
                st(10) = strategy
            Else
                avgCost = (netQty * avgCost + qty * price) / (netQty + qty)
            End If
            netQty = netQty + qty
        End If

    Case "SELL"
        If netQty > 0 Then
            ' CLOSE long position
            Dim closeQty As Double
            closeQty = qty
            If closeQty > netQty Then closeQty = netQty

            Dim pnlClose As Double
            pnlClose = (price - avgCost) * closeQty * mult
            Call WriteRealizedRow(wsR1, outRow, st, closeQty, _
                                avgCost, price, pnlClose, _
                                txDate, ivAtEntry, deltaAtEntry, _
                                usdRate, strategy, "Close")

            outRow = outRow + 1
            realizedCount = realizedCount + 1

            netQty = netQty - closeQty
            qty = qty - closeQty
            If Abs(netQty) < 0.000001 Then
                netQty = 0: avgCost = 0
                st(2) = Empty: st(3) = 0#: st(4) = 0#: st(10) = ""
            End If
        End If
        If qty > 0 Then
            ' Adds to (or opens) short
            If netQty = 0 Then
                avgCost = price
                st(2) = txDate: st(3) = ivAtEntry: st(4) = deltaAtEntry
                st(10) = strategy
            Else
                ' netQty < 0 already
                avgCost = (-netQty * avgCost + qty * price) / (-netQty + qty)
            End If
        End If
    End Select

```

```

        End If
        netQty = netQty - qty
    End If

```

```
Case "EXPIRE", "EXERCISE"
```

```
    Dim closePx As Double: closePx = price ' user supplies 0 for worthless, or intrinsic
```

```
    If netQty > 0 Then
```

```
        Dim pnlExpL As Double
```

```
        pnlExpL = (closePx - avgCost) * netQty * mult
```

```
        Call WriteRealizedRow(wsRl, outRow, st, netQty, _
                               avgCost, closePx, pnlExpL, _
                               txDate, 0#, 0#, _
                               usdRate, strategy, action)
```

```
        outRow = outRow + 1
```

```
        realizedCount = realizedCount + 1
```

```
    ElseIf netQty < 0 Then
```

```
        Dim pnlExpS As Double
```

```
        pnlExpS = (avgCost - closePx) * (-netQty) * mult
```

```
        Call WriteRealizedRow(wsRl, outRow, st, -netQty, _
                               avgCost, closePx, pnlExpS, _
                               txDate, 0#, 0#, _
                               usdRate, strategy, action)
```

```
        outRow = outRow + 1
```

```
        realizedCount = realizedCount + 1
```

```
    End If
```

```
    netQty = 0: avgCost = 0
```

```
    st(2) = Empty: st(3) = 0#: st(4) = 0#: st(10) = ""
```

```
Case "ADJUSTCOST"
```

```
    ' Override avg cost to current price; doesn't realize
```

```
    If netQty <> 0 Then avgCost = price
```

```
End Select
```

```
st(0) = netQty: st(1) = avgCost
```

```
posState(posKey) = st
```

```
NextTx:
```

```
Next i
```

```
LogDebug wsD, "RealizeClosedTransactions", "OK", _
```

```
    "Realized rows written: " & realizedCount
```

```
If Not silent Then
```

```
    MsgBox realizedCount & " realized transaction(s) derived from Transactions.", _
        vbInformation, "Realize Closed Transactions"
```

```
End If
```

```
Exit Sub
```

```
Fail:
```

```
LogDebug wsD, "RealizeClosedTransactions", "ERROR", Err.Description
```

```
If Not silent Then MsgBox "RealizeClosedTransactions failed: " & Err.Description, vbCritical
```

```
End Sub
```

```

' -----
' Helper: write one realized row to Realized worksheet.
' Cols: A TICKER B TYPE C STRATEGY D PNL E SHARES F AVG COST
'       G NET AMT H STRATEGY TAG I PNL(TWD) J DATE K BROKER
'       L STRIKE M EXPIRY N DELTA ENTRY O DELTA EXIT
'       P IV ENTRY Q IV EXIT R THETA COLLECTED S HOLD DAYS T Notes
' -----

```

```
Private Sub WriteRealizedRow(wsRl As Worksheet, rowN As Long, st As Variant, _
                             closeQty As Double, avgCost As Double, _
                             closePx As Double, pnlRaw As Double, _
                             closeDate As Date, ivExit As Double, _
                             deltaExit As Double, usdRate As Double, _
                             strategy As String, noteTag As String)
```

```
    Dim ticker As String: ticker = CStr(st(5))
```

```
    Dim txType As String: txType = CStr(st(6))
```

```
    Dim strike As Variant: strike = st(7)
```

```
    Dim expiry As Variant: expiry = st(8)
```



```

Dim broker As String: broker = CStr(st(9))
Dim stratStored As String: stratStored = CStr(st(10))
Dim openDate As Variant: openDate = st(2)
Dim ivEntry As Double: ivEntry = ToDbl(st(3))
Dim deltaEntry As Double: deltaEntry = ToDbl(st(4))
Dim isUSD As Boolean: isUSD = CBool(st(11))

Dim pnlTWD As Double
If isUSD Then
    pnlTWD = pnlRaw * usdRate
Else
    pnlTWD = pnlRaw
End If

Dim netAmt As Double: netAmt = closeQty * closePx
Dim holdDays As Long
If IsDate(openDate) Then holdDays = closeDate - CDate(openDate)
Dim displayStrat As String
If stratStored <> "" Then displayStrat = stratStored Else displayStrat = strategy

```

```

With wsR1
    ' Force ticker col to text so leading zeros stick (warrant codes)
    .Cells(rowN, 1).NumberFormat = "@"
    .Cells(rowN, 1).Value = ticker
    .Cells(rowN, 2).Value = txType
    .Cells(rowN, 3).Value = displayStrat
    .Cells(rowN, 4).Value = pnlRaw
    .Cells(rowN, 5).Value = closeQty
    .Cells(rowN, 6).Value = avgCost
    .Cells(rowN, 7).Value = netAmt
    .Cells(rowN, 8).Value = displayStrat
    .Cells(rowN, 9).Value = pnlTWD
    .Cells(rowN, 10).Value = closeDate
    .Cells(rowN, 11).Value = broker
    .Cells(rowN, 12).Value = strike
    .Cells(rowN, 13).Value = expiry
    .Cells(rowN, 14).Value = deltaEntry
    .Cells(rowN, 15).Value = deltaExit
    .Cells(rowN, 16).Value = ivEntry
    .Cells(rowN, 17).Value = ivExit
    ' Col 18 R THETA COLLECTED -- estimate omitted (would need full theta trail)
    .Cells(rowN, 19).Value = holdDays
    .Cells(rowN, 20).Value = noteTag & IIf(isUSD, " (USD)", "")

    .Cells(rowN, 4).NumberFormat = "#,##0.00;(#,##0.00)"
    .Cells(rowN, 6).NumberFormat = "#,##0.00"
    .Cells(rowN, 7).NumberFormat = "#,##0.00"
    .Cells(rowN, 9).NumberFormat = "#,##0;(#,##0)"
    .Cells(rowN, 10).NumberFormat = "yyyy/m/d"
    .Cells(rowN, 13).NumberFormat = "yyyy/m/d"
    .Cells(rowN, 14).NumberFormat = "0.0000"
    .Cells(rowN, 15).NumberFormat = "0.0000"
    .Cells(rowN, 16).NumberFormat = "0.00%"
    .Cells(rowN, 17).NumberFormat = "0.00%"

```

```
End With
```

```
End Sub
```

```

Private Function NotTaiwanUnderlying(underly As String, ticker As String) As Boolean
    ' True if the position is denominated in USD (CBOE / US stocks).
    ' Heuristic: Taiwan if underlying is TWII/TAIEX/TXO or numeric 4/6-digit.
    Dim u As String: u = UCase(Trim(underly))
    Dim t As String: t = UCase(Trim(ticker))
    If u = "TWII" Or u = "TAIEX" Or u = "TXO" Or u = "^TWII" Then
        NotTaiwanUnderlying = False
        Exit Function
    End If
    If t = "TXO" Then NotTaiwanUnderlying = False: Exit Function
    ' Strip .TW / .TWO and test numeric
    Dim uStripped As String: uStripped = Replace(Replace(u, ".TWO", ""), ".TW", "")
    If IsNumeric(uStripped) And (Len(uStripped) = 4 Or Len(uStripped) = 6) Then
        NotTaiwanUnderlying = False
    End If

```

```

Exit Function
End If
Dim tStripped As String: tStripped = Replace(Replace(t, ".TWO", ""), ".TW", "")
If IsNumeric(tStripped) And (Len(tStripped) = 4 Or Len(tStripped) = 6) Then
    NotTaiwanUnderlying = False
Exit Function
End If
NotTaiwanUnderlying = True ' default: USD
End Function

Private Function ToDb1(v As Variant) As Double
    If IsNumeric(v) Then ToDb1 = CDb1(v) Else ToDb1 = 0
End Function

' Bug B3 helper: produce a stable sort order for position dict keys.
' Section priority FUT < OPT < WRT; within each section, sort by sort key
' built from (ticker, expiry, type, strike) padded for lexicographic order.
Private Function SortPositionKeys(dict As Object) As String()
    Dim n As Long: n = dict.count
    Dim keys() As String
    Dim sortVals() As String
    ReDim keys(0 To n - 1)
    ReDim sortVals(0 To n - 1)

    Dim i As Long: i = 0
    Dim k As Variant
    For Each k In dict.keys
        keys(i) = CStr(k)
        Dim p() As Variant: p = dict(k)
        Dim sectionRank As String
        If Left(CStr(k), 4) = "FUT" Then
            sectionRank = "1"
        ElseIf Left(CStr(k), 4) = "OPT" Then
            sectionRank = "2"
        ElseIf Left(CStr(k), 4) = "WRT" Then
            sectionRank = "3"
        Else
            sectionRank = "9"
        End If
        Dim ticker As String: ticker = PadRight(CStr(p(1)), 12)
        Dim expStr As String
        If IsDate(p(4)) Then
            expStr = Format(CDate(p(4)), "yyyymmdd")
        Else
            expStr = "99999999"
        End If
        Dim typeRank As String
        Select Case CStr(p(2))
            Case "Call": typeRank = "1"
            Case "Put": typeRank = "2"
            Case "Call Warrant": typeRank = "1"
            Case "Put Warrant": typeRank = "2"
            Case "Bull Cert": typeRank = "3"
            Case "Bear Cert": typeRank = "4"
            Case Else: typeRank = "5"
        End Select
        Dim strikeStr As String
        If IsNumeric(p(3)) Then
            strikeStr = Format(CDb1(p(3)), "0000000.00")
        Else
            strikeStr = "0000000.00"
        End If
        sortVals(i) = sectionRank & "|" & ticker & "|" & expStr & "|" & typeRank & "|" & strikeStr
        i = i + 1
    Next k

    ' Simple insertion sort (n is small -- at most ~40 positions)
    Dim a As Long, b As Long, tmpS As String, tmpK As String
    For a = 0 To n - 2
        For b = a + 1 To n - 1
            If sortVals(a) > sortVals(b) Then

```

```

        tmpS = sortVals(a): sortVals(a) = sortVals(b): sortVals(b) = tmpS
        tmpK = keys(a):      keys(a) = keys(b):      keys(b) = tmpK
    End If
Next b
Next a

SortPositionKeys = keys
End Function

Private Function PadRight(s As String, n As Long) As String
    If Len(s) >= n Then
        PadRight = Left(s, n)
    Else
        PadRight = s & String(n - Len(s), " ")
    End If
End Function

' =====
' LIVE QUOTE FETCH (Yahoo Finance public chart endpoint)
' =====
'
' Workflow:
' 1. AddTransactions -> Transactions sheet
' 2. RebuildPositionsFromTransactions -> RR5 sheet (UNDL LAST blank)
' 3. FetchAllPrices -> auto-fills UNDL LAST / LAST PREM / LAST PX
'    and immediately re-runs RefreshGreeks
' 4. Greeks (Delta/Gamma/Theta/Vega/Rho) appear in RR5 cols T-X
'

Public Function FetchPrice(ticker As String) As Double
    ' Pulls regularMarketPrice from Yahoo Finance chart API.
    ' Returns 0 on any failure (network, ticker not found, parse error).
    ' Bug B7 fix: send User-Agent header so Yahoo doesn't 403 us.
    On Error GoTo Fail
    If Trim(ticker) = "" Then Exit Function

    Dim req As Object, url As String, resp As String
    Set req = CreateObject("MSXML2.XMLHTTP")
    url = "https://query1.finance.yahoo.com/v8/finance/chart/" & ticker
    req.Open "GET", url, False
    req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
    req.Send
    resp = req.responseText

    ' Locate the JSON key "regularMarketPrice":<number>
    Dim p As Long, q As Long, s As String
    p = InStr(resp, """"regularMarketPrice"":")
    If p = 0 Then Exit Function
    p = p + Len("""regularMarketPrice"":")
    q = InStr(p, resp, ",")
    If q = 0 Then q = InStr(p, resp, "}")
    If q <= p Then Exit Function
    s = Mid(resp, p, q - p)
    FetchPrice = val(s)
    Exit Function
Fail:
    FetchPrice = 0
End Function

Private Function YahooOptionTicker(underlying As String, expiry As Date, _
    optType As String, strike As Double) As String
    ' Yahoo Finance option ticker format:
    ' SYMBOL + yymmdd + [CIP] + 8-digit strike * 1000
    ' e.g. AAPL250117C00150000 = AAPL 2025-01-17 Call $150
    ' Works for US options only; Yahoo does NOT carry TAIEX options.
    Dim sym As String: sym = UCase(Trim(underlying))
    Dim dt As String: dt = Format(expiry, "yymmdd")
    Dim cp As String: cp = IIf(UCase(optType) = "CALL", "C", "P")
    Dim sk As String: sk = Format(strike * 1000, "00000000")
    YahooOptionTicker = sym & dt & cp & sk
End Function

```



```

Else
    respText = ""
End If
End If

' Skip "no data" responses (empty data array)
If Len(respText) > 0 And InStr(respText, ""data"":[") = 0 Then
    Dim cl As Double
    cl = ParseFinMindOptionClose(respText, contractCode, strike, cpLower)
    If cl > 0 Then
        FetchTaifexOptionPrice = cl
        Exit Function
    End If
End If

```

```

    tryDt = PreviousBusinessDay(tryDt - 1)
Next attempt

```

```

FetchTaifexOptionPrice = 0
Exit Function

```

Fail:

```

    FetchTaifexOptionPrice = 0
End Function

```

```

Public Function ExpiryToTaifexContractCode(expiry As Date) As String
    ' Returns "YYYYMM" for monthly contract (3rd Wed) or
    ' "YYYYMMW{n}" for weekly. Returns "" if expiry is not a Wednesday.
    If Weekday(expiry, vbMonday) <> 3 Then
        ' Not a Wednesday -- TAIEX options expire on Wed; fall back
        ' to monthly code so caller still has something to query.
        ExpiryToTaifexContractCode = Format(expiry, "yyyymm")
        Exit Function
    End If

```

```

    Dim y As Integer, m As Integer
    y = Year(expiry): m = Month(expiry)

```

```

    ' Index of this Wednesday within the month (0-based)
    Dim wedIdx As Integer: wedIdx = -1
    Dim cur As Date: cur = DateSerial(y, m, 1)
    Dim lastDay As Date: lastDay = DateSerial(y, m + 1, 1) - 1
    Dim count As Integer: count = 0
    Do While cur <= lastDay
        If Weekday(cur, vbMonday) = 3 Then
            If cur = expiry Then wedIdx = count
            count = count + 1
        End If
        cur = cur + 1
    Loop

```

```

    If wedIdx = 2 Then
        ' 3rd Wednesday = monthly contract
        ExpiryToTaifexContractCode = Format(expiry, "yyyymm")
    Else
        ' Weekly: W1/W2/W4/W5 (W3 never exists)
        ExpiryToTaifexContractCode = Format(expiry, "yyyymm") & "W" & (wedIdx + 1)
    End If
End Function

```

```

Private Function ParseFinMindOptionClose(jsonText As String, contractCode As String, _
                                         strike As Double, cpLower As String) As Double
    ' Walks each {...} record block in the 'data' array one at a time,
    ' ensuring contract_date / strike_price / call_put / close /
    ' trading_session are all read from the SAME record.
    ,
    ' Returns the LAST TRADED close price (used for MTM / unrealized P&L,
    ' matching what brokers display as the "deal price" / last traded price).
    ' Settlement_price (the official settlement price) is intentionally NOT
    ' used here -- it's the official daily mark for margin, not the
    ' realizable liquidation price.
    ,

```

```
' Session preference: "position" (daytime regular) > "after_market".
' FinMind labels: "position" = 08:45-13:45, "after_market" = 15:00-04:50.
```

```
Dim startP As Long, endP As Long
startP = InStr(jsonText, ""data"":[")
If startP = 0 Then Exit Function
startP = startP + Len(""data"":[")
endP = InStrRev(jsonText, "]")
If endP <= startP Then Exit Function
Dim body As String: body = Mid(jsonText, startP, endP - startP)
If Len(body) = 0 Then Exit Function

Dim strikeKey As String
If CDBl(CLng(strike)) = strike Then
    strikeKey = ""strike_price"":" & CLng(strike)
Else
    strikeKey = ""strike_price"":" & strike
End If

Dim bestPrice As Double: bestPrice = -1
Dim bestRank As Long: bestRank = -1 ' 2 = close+position, 1 = close+after_market

Dim pos As Long: pos = 1
Do While pos <= Len(body)
    Dim recStart As Long: recStart = InStr(pos, body, "{")
    If recStart = 0 Then Exit Do
    Dim recEnd As Long: recEnd = InStr(recStart, body, "}")
    If recEnd = 0 Then Exit Do
    Dim rec As String: rec = Mid(body, recStart, recEnd - recStart + 1)

    If InStr(rec, ""contract_date"":"" & contractCode & """) = 0 Then GoTo NextRec
    If InStr(rec, ""call_put"":"" & cpLower & """) = 0 Then GoTo NextRec
    Dim hit As Boolean: hit = False
    If InStr(rec, strikeKey & ".0,") > 0 Then hit = True
    If Not hit And InStr(rec, strikeKey & ".0}") > 0 Then hit = True
    If Not hit And InStr(rec, strikeKey & ",") > 0 Then hit = True
    If Not hit And InStr(rec, strikeKey & "}") > 0 Then hit = True
    If Not hit Then GoTo NextRec

    ' --- Session ---
    Dim sess As String: sess = ""
    Dim sp As Long
    sp = InStr(rec, ""trading_session"":""")
    If sp > 0 Then
        sp = sp + Len(""trading_session"":""")
        Dim eq As Long: eq = InStr(sp, rec, """)
        If eq > sp Then sess = Mid(rec, sp, eq - sp)
    End If
    Dim isPosition As Boolean
    isPosition = (sess = "position") Or (sess = "regular")

    ' --- close (last traded price) ---
    Dim cVal As Double: cVal = ExtractJsonNumber(rec, ""close"":")
    If cVal <= 0 Then GoTo NextRec

    Dim thisRank As Long: thisRank = IIf(isPosition, 2, 1)
    If thisRank > bestRank Then
        bestRank = thisRank
        bestPrice = cVal
    End If
NextRec:
    pos = recEnd + 1
Loop

If bestPrice > 0 Then ParseFinMindOptionClose = bestPrice
End Function
```

```
Private Function ExtractJsonNumber(rec As String, key As String) As Double
' Finds key (e.g. ""close"":") and extracts the following numeric value
' until comma or closing brace. Returns 0 if not found or unparseable.
Dim p As Long, q As Long
```

```

    p = InStr(rec, key)
    If p = 0 Then Exit Function
    p = p + Len(key)
    q = InStr(p, rec, ",")
    If q = 0 Then q = InStr(p, rec, "}")
    If q <= p Then Exit Function
    ExtractJsonNumber = val(Mid(rec, p, q - p))
End Function

' =====
' DIAGNOSTIC: Inspect what FinMind returns for the position in O31
' -----
' Run from Alt+F8 or Immediate window. Prints to Debug.Print (Ctrl+G
' Immediate window). Shows contract_code derivation, URL hit, total
' records, how many matched each criterion, sample matched record,
' and final returned price.
' =====
' =====
' DIAGNOSTIC: Inspect why a warrant row didn't fetch / compute
' Pass the actual row number (default 31 in compact layout)
' =====

Public Sub Debug_WarrantFetch(Optional wrtRow As Long = 31)
    Dim wsR As Worksheet: Set wsR = ThisWorkbook.Sheets("RR5")
    Dim wCode As String: wCode = Trim(CStr(wsR.Cells(wrtRow, 1).Value))

    Debug.Print "=== Debug_WarrantFetch (row " & wrtRow & ") ==="
    Debug.Print "Code raw   : '" & wCode & "' (len=" & Len(wCode) & ")"
    If wCode = "" Then
        Debug.Print "EMPTY CODE -- this row has no warrant. Did you re-run RebuildPositionsFromTransactions?"
        Exit Sub
    End If

    Dim wPad As String
    If IsNumeric(wCode) Then
        wPad = Format(val(wCode), "000000")
    Else
        wPad = wCode
    End If
    Debug.Print "Code pad   : '" & wPad & "'"

    Dim px1 As Double, px2 As Double
    px1 = FetchPrice(wPad & ".TW")
    Debug.Print wPad & ".TW -> " & px1
    If px1 <= 0 Then
        px2 = FetchPrice(wPad & ".TWO")
        Debug.Print wPad & ".TWO -> " & px2
    End If

    Debug.Print ""
    Debug.Print "RR5 row " & wrtRow & " current cells:"
    Debug.Print "  A CODE       : " & wsR.Cells(wrtRow, 1).Value
    Debug.Print "  G TYPE       : " & wsR.Cells(wrtRow, 7).Value
    Debug.Print "  I UNDERLYING : " & wsR.Cells(wrtRow, 9).Value
    Debug.Print "  J UNDL LAST  : " & wsR.Cells(wrtRow, 10).Value
    Debug.Print "  K STRIKE     : " & wsR.Cells(wrtRow, 11).Value
    Debug.Print "  L CONV RATIO : " & wsR.Cells(wrtRow, 12).Value
    Debug.Print "  M SHARES     : " & wsR.Cells(wrtRow, 13).Value
    Debug.Print "  N ENTRY PX   : " & wsR.Cells(wrtRow, 14).Value
    Debug.Print "  O LAST PX    : " & wsR.Cells(wrtRow, 15).Value
    Debug.Print "  S IV         : " & wsR.Cells(wrtRow, 19).Value
    Debug.Print "  T DELTA      : " & wsR.Cells(wrtRow, 20).Value
    Debug.Print "  X GAMMA      : " & wsR.Cells(wrtRow, 24).Value
    Debug.Print "  Y THETA/DAY  : " & wsR.Cells(wrtRow, 25).Value
    Debug.Print "  Z VEGA       : " & wsR.Cells(wrtRow, 26).Value
    Debug.Print "  AA NOM GEAR  : " & wsR.Cells(wrtRow, 27).Value
    Debug.Print "  AB EFF GEAR  : " & wsR.Cells(wrtRow, 28).Value

    Debug.Print ""
    Debug.Print "RefreshGreeks needs UNDL LAST + STRIKE + IV + CONV RATIO all > 0."
    Debug.Print "If any is 0/blank, Greeks won't compute."
End Sub

```

```

Public Sub Debug_TaifexFetch(Optional rrRow As Long = 31)
    On Error GoTo Fail
    Dim wsR As Worksheet: Set wsR = ThisWorkbook.Sheets("RR5")

    Dim contract As String, expiry As Variant, strike As Variant
    Dim optType As String, underlying As String
    contract = CStr(wsR.Cells(rrRow, 1).Value)
    expiry = wsR.Cells(rrRow, 4).Value
    strike = wsR.Cells(rrRow, 9).Value
    optType = Trim(CStr(wsR.Cells(rrRow, 8).Value))
    underlying = Trim(CStr(wsR.Cells(rrRow, 10).Value))

    Debug.Print "=== Debug_TaifexFetch (row " & rrRow & ") ==="
    Debug.Print "Contract   : " & contract
    Debug.Print "Underlying: " & underlying
    Debug.Print "Expiry     : " & expiry & " (" & Format(CDate(expiry), "yyyy-mm-dd ddd") & ")"
    Debug.Print "Strike    : " & strike
    Debug.Print "Type      : " & optType

    Dim taifexID As String: taifexID = MapUnderlyingToTaifexOptionID(underlying)
    Dim contractCode As String: contractCode = ExpiryToTaifexContractCode(CDate(expiry))
    Debug.Print "Option ID : " & taifexID
    Debug.Print "Contract$ : " & contractCode

    Dim tradeDt As Date: tradeDt = PreviousBusinessDay(Date)
    Dim dateStr As String: dateStr = Format(tradeDt, "yyyy-mm-dd")
    Dim url As String
    url = "https://api.finmindtrade.com/api/v4/data?dataset=TaiwanOptionDaily" & _
        "&data_id=" & taifexID & "&start_date=" & dateStr & "&end_date=" & dateStr
    Debug.Print "URL      : " & url

    Dim req As Object: Set req = CreateObject("MSXML2.XMLHTTP")
    req.Open "GET", url, False
    req.setRequestHeader "User-Agent", "Mozilla/5.0"
    req.sEnd
    Debug.Print "HTTP      : " & req.status & " (resp len " & Len(req.responseText) & ")"

    Dim resp As String: resp = req.responseText
    Dim startP As Long, endP As Long
    startP = InStr(resp, """"data"":")
    If startP > 0 Then
        startP = startP + Len(""""data"":")
        endP = InStrRev(resp, "]")
        Dim body As String: body = Mid(resp, startP, endP - startP)
        Debug.Print "Body len  : " & Len(body)

        Dim cpLower As String
        If InStr(UCase(optType), "CALL") > 0 Then cpLower = "call" Else cpLower = "put"

        Dim strikeKey As String
        strikeKey = """"strike_price"":"" & CLng(CDbl(strike))

        Dim pos As Long: pos = 1
        Dim totalRecs As Long, cMatch As Long, sMatch As Long, pMatch As Long
        Dim fullMatch As Long, sampleContract As String

        Do While pos <= Len(body)
            Dim rs As Long: rs = InStr(pos, body, "{")
            If rs = 0 Then Exit Do
            Dim re As Long: re = InStr(rs, body, "}")
            If re = 0 Then Exit Do
            Dim rec As String: rec = Mid(body, rs, re - rs + 1)
            totalRecs = totalRecs + 1

            Dim hC As Boolean: hC = InStr(rec, """"contract_date"":"" & contractCode & """"") > 0
            Dim hP As Boolean: hP = InStr(rec, """"call_put"":"" & cpLower & """"") > 0
            Dim hS As Boolean
            hS = (InStr(rec, strikeKey & ".0,") > 0) Or _
                (InStr(rec, strikeKey & ".0}") > 0) Or _
                (InStr(rec, strikeKey & ",") > 0) Or _

```



```
(InStr(rec, strikeKey & "{") > 0)
```

```
If hC Then
    cMatch = cMatch + 1
    If sampleContract = "" Then sampleContract = rec
End If
If hS Then sMatch = sMatch + 1
If hP Then pMatch = pMatch + 1
If hC And hP And hS Then
    fullMatch = fullMatch + 1
    Debug.Print " >>> FULL MATCH #" & fullMatch & ": " & rec
End If
pos = re + 1
Loop
```

```
Debug.Print "Records : " & totalRecs
Debug.Print " contract_date match : " & cMatch
Debug.Print " strike_price match : " & sMatch
Debug.Print " call_put match : " & pMatch
Debug.Print " ALL THREE match : " & fullMatch
If fullMatch = 0 And sampleContract <> "" Then
    Debug.Print "First contract-only match (strike/CP didn't match):"
    Debug.Print " " & Left(sampleContract, 500)
End If
End If
```

```
Dim px As Double
px = FetchTaifexOptionPrice(taifexID, CDate(expiry), Cdbl(strike), optType)
Debug.Print ""
Debug.Print "FetchTaifexOptionPrice -> " & px
Exit Sub
```

```
Fail:
    Debug.Print "ERROR: " & Err.Description
End Sub
```

```
Private Function PreviousBusinessDay(d As Date) As Date
    ' Simple Mon-Fri filter; does not account for TW holidays.
    ' FetchTaifexOptionPrice retries 5 days, so holidays self-correct
    ' as long as a real trading day exists within 5 business days.
    Dim r As Date: r = d
    Do While Weekday(r, vbMonday) > 5
        r = r - 1
    Loop
    PreviousBusinessDay = r
End Function
```

```
Private Function MapUnderlyingToTaifexOptionID(underlying As String) As String
    ' Map RR5 J UNDERLYING value to FinMind option_id.
    ' "TWII" / "TAIEX" / "TXO" -> "TXO" (Taiwan index options)
    ' Individual 4-digit Taiwan codes -> stock-option series mapping
    ' (FinMind uses stock code for individual stock options, e.g. "2330")
    ' Returns "" if no mapping (caller should skip TAIEX path).
    Dim u As String: u = UCase(Trim(underlying))
    Select Case u
        Case "TWII", "TAIEX", "TXO", "^TWII": MapUnderlyingToTaifexOptionID = "TXO"
        Case Else
            If IsNumeric(u) And Len(u) = 4 Then
                MapUnderlyingToTaifexOptionID = u
            Else
                MapUnderlyingToTaifexOptionID = ""
            End If
        End Select
    End Function
```

```
Public Function LookupFuturesMargin(product As String) As Double
    ' Walks TickerMap (col B = Code, col D = Margin_TWD) for a matching product.
    ' Returns 0 if not found.
    On Error Resume Next
    Dim wsM As Worksheet
    Set wsM = ThisWorkbook.Sheets("TickerMap")
    Dim p As String: p = UCase(Trim(product))
```

```

If p = "" Then Exit Function
Dim i As Long
For i = 2 To 20
    If UCase(Trim(CStr(wsM.Cells(i, 2).Value))) = p Then
        If IsNumeric(wsM.Cells(i, 4).Value) Then
            LookupFuturesMargin = CDBl(wsM.Cells(i, 4).Value)
        End If
        Exit Function
    End If
Next i
End Function

Private Function NormalizeTicker(t As String) As String
    ' Map common Taiwan / US tickers to Yahoo Finance format.
    Dim tt As String: tt = UCase(Trim(t))
    Select Case tt
        Case "TWII", "TAIEX": NormalizeTicker = "^TWII"
        Case "TXO":           NormalizeTicker = "^TWII" ' TXO underlying is TAIEX
        Case "SPX":           NormalizeTicker = "^GSPC"
        Case "NDX":           NormalizeTicker = "^NDX"
        Case "DJI":           NormalizeTicker = "^DJI"
        Case "VIX":           NormalizeTicker = "^VIX"
        Case "RUT":           NormalizeTicker = "^RUT"
        Case Else
            ' Taiwan 4-digit stock codes (2330 -> 2330.TW)
            ' Taiwan warrants are typically 6-digit (.TW or .TWO)
            If IsNumeric(tt) And (Len(tt) = 4 Or Len(tt) = 6) Then
                NormalizeTicker = tt & ".TW"
            Else
                NormalizeTicker = tt ' assume US ticker
            End If
        End Select
    End Function

```

```

Public Sub FetchAllPrices(Optional silent As Boolean = False)
    ' Walks all three RR5 sections, fetches Yahoo quotes, fills price cells,
    ' then re-runs RefreshGreeks so Greeks pick up the new UNDL LAST.
    '
    ' silent=True: skip the final MsgBox. Used when called from
    ' RebuildPositionsFromTransactions which has its own summary box.
    On Error GoTo ErrHandler
    Dim wsR As Worksheet, wsD As Worksheet
    Set wsR = ThisWorkbook.Sheets("RR5")
    Set wsD = ThisWorkbook.Sheets("DebugLog")

    Application.StatusBar = "Fetching live prices..."

    Dim okN As Long, failN As Long, i As Long
    Dim tkr As String, px As Double

    ' --- Futures (rows 7-14): A=CONTRACT, G=PRODUCT, L=LAST, P=MARGIN REQ ---
    For i = 7 To 14
        tkr = Trim(CStr(wsR.Cells(i, 1).Value))
        If tkr <> "" Then
            Application.StatusBar = "Fetching Futures " & tkr & "..."
            px = FetchPrice(NormalizeTicker(tkr))
            If px > 0 Then
                wsR.Cells(i, 12).Value = px ' L LAST
                okN = okN + 1
            Else
                failN = failN + 1
            End If
            ' Auto-fill MARGIN REQ from TickerMap via PRODUCT code.
            ' Bug B4 fix: consolidate duplicate If/ElseIf branches that both
            ' did the same lookup. Fill if cell is empty/text OR numeric 0.
            Dim prod As String, mgn As Double
            Dim cellP As Variant
            prod = Trim(CStr(wsR.Cells(i, 7).Value)) ' G PRODUCT
            If prod <> "" Then
                cellP = wsR.Cells(i, 16).Value ' P MARGIN REQ
                If Not IsNumeric(cellP) Or CDBl(cellP) = 0 Then

```

```

        mgn = LookupFuturesMargin(prod)
        If mgn > 0 Then
            wsR.Cells(i, 16).Value = mgn
            okN = okN + 1
        End If
    End If
End If
End If
Next i

' --- Options (rows 19-26): J=UNDERLYING, K=UNDL LAST, O=LAST PREM ---
' UNDL LAST: fetched from underlying ticker (always tries).
' LAST PREM: TAIEX TXO via FinMind, US options via Yahoo.
For i = 19 To 26
    Dim contract As String
    contract = Trim(CStr(wsR.Cells(i, 1).Value))
    If contract <> "" Then
        tkr = Trim(CStr(wsR.Cells(i, 10).Value)) ' J UNDERLYING name
        If tkr <> "" Then
            px = 0
            Dim optUndlC As String
            optUndlC = Replace(Replace(UCase(Trim(tkr)), ".TWO", ""), ".TW", "")
            ' Yahoo Finance first
            Application.StatusBar = "Fetching Opt UNDL " & tkr & " (Yahoo)..."
            px = FetchPrice(NormalizeTicker(tkr))
            ' TPEX -> TWSE as backup for TW numeric codes
            If px <= 0 And IsNumeric(optUndlC) And Len(optUndlC) >= 4 And Len(optUndlC) <= 6 Then
                Application.StatusBar = "Fetching Opt UNDL " & optUndlC & " (TPEX OTC)..."
                px = FetchTPEXStockPrice(optUndlC)
                If px <= 0 Then
                    Application.StatusBar = "Fetching Opt UNDL " & optUndlC & " (TWSE)..."
                    px = FetchTWSEClose(optUndlC)
                End If
            End If
            If px > 0 Then
                wsR.Cells(i, 11).Value = px ' K UNDL LAST
                okN = okN + 1
            Else
                failN = failN + 1
            End If

            ' --- Option premium fetch ---
            ' Route by underlying:
            ' TWII / 4-digit TW code -> FinMind TAIEX (EOD close)
            ' Everything else -> Yahoo Finance US option chain
            Dim oStrike As Variant, oExpiry As Variant, oType As String
            oStrike = wsR.Cells(i, 9).Value ' I STRIKE
            oExpiry = wsR.Cells(i, 4).Value ' D EXPIRY
            oType = Trim(CStr(wsR.Cells(i, 8).Value)) ' H TYPE
            Dim hasOptInputs As Boolean
            hasOptInputs = (IsNumeric(oStrike) And IsDate(oExpiry) And _
                (oType = "Call" Or oType = "Put"))

            Dim taifexID As String
            taifexID = MapUnderlyingToTaifexOptionID(tkr)

            If taifexID <> "" And hasOptInputs Then
                ' TAIEX path via FinMind
                Application.StatusBar = "Fetching TAIEX " & taifexID & _
                    " " & oType & " " & oStrike & " " & Format(oExpiry, "yyyy-mm-dd") & "..."
                Dim taifexPx As Double
                taifexPx = FetchTaifexOptionPrice(taifexID, CDate(oExpiry), _
                    CDBl(oStrike), oType)

                If taifexPx > 0 Then
                    wsR.Cells(i, 15).Value = taifexPx ' O LAST PREM
                    okN = okN + 1
                Else
                    failN = failN + 1
                End If
            ElseIf taifexID = "" And hasOptInputs Then
                ' US option via Yahoo

```

```

        Dim optTkr As String, premPx As Double
        optTkr = YahooOptionTicker(tkr, CDate(oExpiry), _
                                   oType, CDBl(oStrike))
        Application.StatusBar = "Fetching Opt PREM " & optTkr & "..."
        premPx = FetchPrice(optTkr)
        If premPx > 0 Then
            wsR.Cells(i, 15).Value = premPx    ' O LAST PREM
            okN = okN + 1
        End If
    End If
End If
End If
Next i

' --- Warrants (rows 31-38): I=UNDERLYING, J=UNDL LAST; A=CODE, O=LAST PX ---
' Price priority: Yahoo v8 -> TPEx OTC -> TWSE openapi/STOCK_DAY
For i = 31 To 38
    Dim wCode As String
    wCode = Trim(CStr(wsR.Cells(i, 1).Value))
    If wCode <> "" Then
        ' --- Underlying spot price ---
        tkr = Trim(CStr(wsR.Cells(i, 9).Value))    ' I UNDERLYING name/code
        If tkr <> "" Then
            px = 0
            Dim wUndlC As String
            wUndlC = Replace(Replace(UCase(Trim(tkr)), ".TWO", ""), ".TW", "")
            ' Yahoo Finance first
            Application.StatusBar = "Fetching UNDL " & tkr & " (Yahoo)..."
            px = FetchPrice(NormalizeTicker(tkr))
            ' TPEx -> TWSE as backup for TW numeric codes
            If px <= 0 And IsNumeric(wUndlC) And Len(wUndlC) >= 4 And Len(wUndlC) <= 6 Then
                Application.StatusBar = "Fetching UNDL " & wUndlC & " (TPEx OTC)..."
                px = FetchTPExStockPrice(wUndlC)
                If px <= 0 Then
                    Application.StatusBar = "Fetching UNDL " & wUndlC & " (TWSE)..."
                    px = FetchTWSEClose(wUndlC)
                End If
            End If
            If px > 0 Then
                wsR.Cells(i, 10).Value = px    ' J UNDL LAST
                okN = okN + 1
            Else
                failN = failN + 1
            End If
        End If
        ' --- Warrant own market price ---
        ' NOTE: FinMind was evaluated as a potential primary source here but was
        ' skipped. Every FinMind dataset that includes a warrant closing price
        ' (TaiwanStockInfoWithWarrantSummary, TaiwanStockWarrantTradingDailyReport)
        ' requires a paid Sponsor-tier token, and even those either key by the
        ' underlying mother-stock code (not the warrant's own code) or return
        ' per-broker trade prices instead of an official daily close. The one
        ' Free-tier warrant dataset (TaiwanStockInfoWithWarrant) has no price
        ' field at all. Confirmed 2026-07-01 against https://finmind.github.io/llms-full.txt.
        ' Fallback chain left as Yahoo -> OpenAPI (TPEx then TWSE), unchanged in
        ' relative order, just as it was before this review.
        Dim wPad As String
        If IsNumeric(wCode) Then
            wPad = Format(val(wCode), "000000")
        Else
            wPad = wCode
        End If
        px = 0
        Application.StatusBar = "Fetching Wrt " & wPad & ".TW (Yahoo)..."
        px = FetchPrice(wPad & ".TW")
        If px <= 0 Then
            Application.StatusBar = "Fetching Wrt " & wPad & ".TWO (Yahoo)..."
            px = FetchPrice(wPad & ".TWO")
        End If
        If px <= 0 Then
            Application.StatusBar = "Fetching Wrt " & wPad & " (TPEx OTC)..."

```

```

        px = FetchTPExWarrantPrice(wPad)
    End If
    If px <= 0 Then
        Application.StatusBar = "Fetching Wrt " & wPad & " (TWSE)..."
        px = FetchTWSEClose(wPad)
    End If
    If px > 0 Then
        wsR.Cells(i, 15).Value = px      ' 0 LAST PX
        okN = okN + 1
    Else
        failN = failN + 1
    End If
    ' --- Warrant name -> col B (fetch when blank) ---
    If Trim(CStr(wsR.Cells(i, 2).Value)) = "" Then
        Application.StatusBar = "Fetching Wrt name " & wPad & "..."
        Dim wrtName As String
        wrtName = FetchWarrantName(wPad)
        If wrtName <> "" Then wsR.Cells(i, 2).Value = wrtName
    End If
End If
Next i

Application.StatusBar = False

LogDebug wsD, "FetchAllPrices", "OK", _
    "Updated=" & okN & " Failed=" & failN
' Now that UNDL LAST is populated, recalc Greeks
Call RefreshGreeks
' Derive realized PnL from Transactions (silent: aggregate count
' included in our own MsgBox / caller's MsgBox)
Call RealizeClosedTransactions(silent:=True)

If Not silent Then
    ' Count realized rows for summary
    Dim r1C As Long
    On Error Resume Next
    r1C = ThisWorkbook.Sheets("Realized"). _
        Cells(ThisWorkbook.Sheets("Realized").Rows.count, 1).End(xlUp).row - 1
    On Error GoTo ErrHandler
    If r1C < 0 Then r1C = 0

    MsgBox "Quote fetch complete." & vbCrLf & vbCrLf & _
        " Updated: " & okN & " cell(s)" & vbCrLf & _
        " Failed : " & failN & " cell(s)" & vbCrLf & vbCrLf & _
        "Greeks refreshed." & vbCrLf & _
        "Realized rows derived: " & r1C, _
        vbInformation, "RR5 FetchAllPrices"
End If
Exit Sub
ErrHandler:
Application.StatusBar = False
LogDebug wsD, "FetchAllPrices", "ERROR", Err.Description
' Don't show error msgbox in silent mode -- the caller (e.g.
' RebuildPositionsFromTransactions) will report aggregate status
If Not silent Then
    MsgBox "FetchAllPrices failed: " & Err.Description, vbCritical
End If
End Sub

' =====
' ONE-SHOT SETUP: Add GAMMA / THETA / VEGA columns to RR5 Warrants
' =====
'
' Run this ONCE (Alt+F8 -> Setup_WarrantsGreeksColumns -> Run).
' Idempotent -- safe to re-run.
'
' What it does:
' 1. Adds headers in row 74: X=GAMMA Y=THETA/DAY Z=VEGA
' 2. Formats data cells X75:Z104 with 4-decimal precision
' 3. Adds SUMPRODUCT subtotals in row 105 (cols X/Y/Z)
' 4. Patches Summary bar (row 1/2) so Net Gamma/Theta/Vega include warrants

```

```

Public Sub Setup_WarrantsGreeksColumns()
    On Error GoTo ErrHandler
    Dim wsR As Worksheet
    Set wsR = ThisWorkbook.Sheets("RR5")

    ' -- 1. Headers in row 74 (cols X, Y, Z) -----
    Dim hdrSpec As Variant
    hdrSpec = Array(Array(24, "GAMMA"), _
                    Array(25, "THETA/DAY"), _
                    Array(26, "VEGA"))
    Dim h As Variant
    For Each h In hdrSpec
        With wsR.Cells(74, CLng(h(0)))
            .Value = h(1)
            .Font.Name = "Consolas"
            .Font.Size = 9
            .Font.Bold = True
            .Font.Color = RGB(255, 165, 0)    ' orange like other headers
            .Interior.Color = RGB(0, 0, 0)    ' black bg
            .HorizontalAlignment = xlCenter
            .Borders.LineStyle = xlContinuous
            .Borders.Color = RGB(47, 79, 79)
        End With
        wsR.Columns(CLng(h(0))).ColumnWidth = 12
    Next h

    ' -- 2. Format data rows 75-104 (cols X/Y/Z) -----
    With wsR.Range(wsR.Cells(75, 24), wsR.Cells(104, 26))
        .NumberFormat = "0.0000;(0.0000);""-""
        .Font.Name = "Consolas"
        .Font.Size = 10
        .Font.Color = RGB(255, 215, 0)        ' gold (formula style)
        .Interior.Color = RGB(0, 0, 0)
        .Borders.LineStyle = xlContinuous
        .Borders.Color = RGB(47, 79, 79)
    End With

    ' -- 3. Subtotal row 105 -----
    ' Each Greek aggregated as SUMPRODUCT(SHARES * Greek_per_warrant)
    ' to give portfolio-level impact.
    wsR.Cells(105, 24).Formula = _
        "=SUMPRODUCT((A75:A104<>""""")*M75:M104*X75:X104)"
    wsR.Cells(105, 25).Formula = _
        "=SUMPRODUCT((A75:A104<>""""")*M75:M104*Y75:Y104)"
    wsR.Cells(105, 26).Formula = _
        "=SUMPRODUCT((A75:A104<>""""")*M75:M104*Z75:Z104)"
    With wsR.Range(wsR.Cells(105, 24), wsR.Cells(105, 26))
        .Font.Name = "Consolas"
        .Font.Bold = True
        .Font.Color = RGB(255, 255, 255)
        .Interior.Color = RGB(0, 0, 0)
        .NumberFormat = "#,##0.00;(#,##0.00);""-""
        .Borders.LineStyle = xlContinuous
        .Borders.Color = RGB(47, 79, 79)
    End With

    ' -- 4. Patch Summary bar to include warrants -----
    ' L1 NET DELTA: original formula was buggy (used IV*ConvRatio for warrants).
    '           Correct: SHARES * Warrant_Delta-per-warrant
    wsR.Range("L1").Formula = _
        "=IFERROR(SUBTOTAL(9,S7:S26)+SUBTOTAL(9,Y31:Y70)) & _
        "+SUMPRODUCT((A75:A104<>""""")*M75:M104*T75:T104),0)"
    ' N1 NET GAMMA : options (SUMPRODUCT * SHARES * MULT) + warrants X105
    wsR.Range("N1").Formula = _
        "=IFERROR(SUMPRODUCT((H31:H70<>""""")*U31:U70*L31:L70*M31:M70)+X105,0)"
    ' J2 NET THETA/DAY: options SUBTOTAL(Z31:Z70) + warrants Y105
    wsR.Range("J2").Formula = _
        "=IFERROR(SUBTOTAL(9,Z31:Z70)+Y105,0)"
    ' L2 NET VEGA : options SUBTOTAL(AA31:AA70) + warrants Z105
    wsR.Range("L2").Formula = _

```

```
"=IFERROR(SUBTOTAL(9,AA31:AA70)+Z105,0)"
```

```
' -- 5. Done -----
```

```
MsgBox "Warrants Greeks columns installed:" & vbCrLf & vbCrLf & _
" X = GAMMA (per warrant)" & vbCrLf & _
" Y = THETA/DAY (per warrant)" & vbCrLf & _
" Z = VEGA (per warrant)" & vbCrLf & vbCrLf & _
"Row 105: portfolio-level SUMPRODUCT subtotals." & vbCrLf & _
"Summary bar patched:" & vbCrLf & _
" L1 NET DELTA -- bug fixed (was IV*ConvRatio, now SHARES*Delta)" & vbCrLf & _
" N1 NET GAMMA -- now includes warrants" & vbCrLf & _
" J2 NET THETA -- now includes warrants" & vbCrLf & _
" L2 NET VEGA -- now includes warrants" & vbCrLf & vbCrLf & _
"Run RefreshGreeks to populate values.", _
vbInformation, "RR5 Setup"
```

```
Exit Sub
```

```
ErrorHandler:
```

```
MsgBox "Setup failed: " & Err.Description, vbCritical
```

```
End Sub
```

```
' =====
' ONE-SHOT SETUP: Add Gearing (leverage) columns to RR5 Warrants
' =====
'
' Run this ONCE (Alt+F8 -> Setup_WarrantGearingColumns -> Run).
' Idempotent -- safe to re-run.
' Compact layout: warrants header row 30, data rows 31-38.
'
' What it does:
' 1. Adds headers in row 30: AA=NOM GEARING AB=EFF GEARING
' 2. Formats data cells AA31:AB38 (2-decimal)
' 3. Values are computed by RefreshGreeks -- run it after this setup.
'
```

```
Public Sub Setup_WarrantGearingColumns()
```

```
On Error GoTo ErrorHandler
```

```
Dim wsR As Worksheet
```

```
Set wsR = ThisWorkbook.Sheets("RR5")
```

```
Dim hdrSpec As Variant
```

```
hdrSpec = Array(Array(27, "NOM GEARING"), _
Array(28, "EFF GEARING"))
```

```
Dim h As Variant
```

```
For Each h In hdrSpec
```

```
With wsR.Cells(30, CLng(h(0)))
```

```
.Value = h(1)
```

```
.Font.Name = "Consolas"
```

```
.Font.Size = 9
```

```
.Font.Bold = True
```

```
.Font.Color = RGB(255, 165, 0) ' orange like other headers
```

```
.Interior.Color = RGB(0, 0, 0) ' black bg
```

```
.HorizontalAlignment = xlCenter
```

```
.Borders.LineStyle = xlContinuous
```

```
.Borders.Color = RGB(47, 79, 79)
```

```
End With
```

```
wsR.Columns(CLng(h(0))).ColumnWidth = 12
```

```
Next h
```

```
With wsR.Range(wsR.Cells(31, 27), wsR.Cells(38, 28))
```

```
.NumberFormat = "0.00;(0.00);""-"""
```

```
.Font.Name = "Consolas"
```

```
.Font.Size = 10
```

```
.Font.Color = RGB(255, 215, 0) ' gold (formula style)
```

```
.Interior.Color = RGB(0, 0, 0)
```

```
.Borders.LineStyle = xlContinuous
```

```
.Borders.Color = RGB(47, 79, 79)
```

```
End With
```

```
MsgBox "Warrant Gearing columns installed:" & vbCrLf & vbCrLf & _
```

```
" AA = NOM GEARING (UNDL LAST / (LAST PX * CONV RATIO))" & vbCrLf & _
```

```
" AB = EFF GEARING (UNDL LAST * DELTA / LAST PX)" & vbCrLf & vbCrLf & _
```

```
"Run RefreshGreeks to populate values.", _
```

```

        vbInformation, "RR5 Setup"
    Exit Sub
ErrorHandler:
    MsgBox "Setup failed: " & Err.Description, vbCritical
End Sub

' =====
' FINMIND STOCK / WARRANT PRICE (Taiwan EOD close, TWSE + OTC)
' -----
' Uses FinMind TaiwanStockPrice dataset which covers both exchanges with
' no suffix needed. More reliable than Yahoo for warrants because:
'   - Official exchange closing price (not last tick)
'   - Handles OTC-listed warrants Yahoo misidentifies as TWSE
'   - Free, no auth required
' Retries up to 3 previous business days for post-holiday gaps.
' Called as primary source in FetchAllPrices warrant section.
' =====

Public Function FetchFinMindStockPrice(ticker As String) As Double
    On Error GoTo Fail
    If Trim(ticker) = "" Then Exit Function
    Dim startDt As Date: startDt = PreviousBusinessDay(Date)
    Dim attempt As Long
    For attempt = 1 To 3
        Dim dateStr As String: dateStr = Format(startDt, "yyyy-mm-dd")
        Dim url As String
        url = "https://api.finmindtrade.com/api/v4/data" & _
            "?dataset=TaiwanStockPrice" & _
            "&data_id=" & Trim(ticker) & _
            "&start_date=" & dateStr & _
            "&end_date=" & dateStr
        Dim req As Object
        Set req = CreateObject("MSXML2.XMLHTTP")
        req.Open "GET", url, False
        req.setRequestHeader "User-Agent", "Mozilla/5.0"
        req.send
        If req.status = 200 Then
            Dim resp As String: resp = req.responseText
            If InStr(resp, """"data"":[") = 0 And Len(resp) > 20 Then
                Dim cl As Double: cl = ExtractJsonNumber(resp, """"close"":")
                If cl > 0 Then
                    FetchFinMindStockPrice = cl
                    Exit Function
                End If
            End If
            startDt = PreviousBusinessDay(startDt - 1)
        Next attempt
    Fail:
        FetchFinMindStockPrice = 0
    End Function

' =====
' TWSE STOCK / WARRANT CLOSE (openapi.twse.com.tw STOCK_DAY_ALL, cached)
' -----
' Primary: bulk download from openapi.twse.com.tw - covers all TWSE
' listed stocks AND warrants with named JSON fields.
' Fallback: rwd/zh/afterTrading/STOCK_DAY for 6-digit warrant codes
' not present in STOCK_DAY_ALL (parses the closing-price field from data array).
' Cache: one download per calendar day to avoid repeated large fetches.
' =====

Public Function FetchTWSEClose(stockCode As String) As Double
    On Error GoTo Fail
    If Trim(stockCode) = "" Then Exit Function

    Static twseCache As Object
    If twseCache Is Nothing Then Set twseCache = CreateObject("Scripting.Dictionary")
    Dim today As String: today = Format(Date, "yyyymmdd")

    Dim resp As String
    If Not twseCache.Exists(today) Then
        Dim req As Object

```



```

Set req = CreateObject("MSXML2.XMLHTTP")
req.Open "GET", "https://openapi.twse.com.tw/v1/exchangeReport/STOCK_DAY_ALL", False
req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
req.setRequestHeader "Accept", "application/json"
req.sEnd
If req.status = 200 Then
    twseCache(today) = req.responseText
Else
    twseCache(today) = ""
End If
End If
resp = CStr(twseCache(today))

Dim cl As Double
If Len(resp) > 10 Then
    Dim codeKey As String: codeKey = """"Code"":"" & Trim(stockCode) & """"
    Dim idx As Long: idx = InStr(resp, codeKey)
    If idx > 0 Then
        Dim objEnd As Long: objEnd = InStr(idx, resp, "{")
        If objEnd > idx Then
            Dim objStr As String: objStr = Mid(resp, idx, objEnd - idx + 1)
            Dim s As String
            s = JsonStr(objStr, "ClosingPrice")
            If s = "" Then s = JsonStr(objStr, "Close")
            If s <> "" Then
                s = Replace(s, ",", "")
                If IsNumeric(s) Then cl = CDBl(s)
            End If
            If cl <= 0 Then cl = ExtractJsonNumber(objStr, """"ClosingPrice"":")
            If cl <= 0 Then cl = ExtractJsonNumber(objStr, """"Close"":")
        End If
    End If
End If
If cl > 0 Then FetchTWSEClose = cl: Exit Function

' Fallback: rwd STOCK_DAY endpoint (reliable for 6-digit warrant codes)
Dim dt As Date: dt = Date
Dim att As Long
For att = 1 To 2
    Dim dateStr As String
    dateStr = Format(DateSerial(Year(dt), Month(dt), 1), "yyyymmdd")
    Dim req2 As Object
    Set req2 = CreateObject("MSXML2.XMLHTTP")
    req2.Open "GET", "https://www.twse.com.tw/rwd/zh/afterTrading/STOCK_DAY" & _
        "?response=json&date=" & dateStr & "&stockNo=" & Trim(stockCode), False
    req2.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
    req2.setRequestHeader "Referer", "https://www.twse.com.tw/"
    req2.sEnd
    If req2.status = 200 Then
        resp = req2.responseText
        If InStr(resp, """"stat"":""OK""") > 0 Then
            ' data[][6] = closing price; find last inner record before "]"
            Dim dStart As Long: dStart = InStr(resp, """"data"":[")
            If dStart > 0 Then
                dStart = dStart + Len("""data"":[")
                Dim dEnd As Long: dEnd = InStr(dStart, resp, "]]")
                If dEnd > dStart Then
                    Dim lastBrk As Long: lastBrk = InStrRev(resp, "[", dEnd)
                    If lastBrk > dStart Then
                        Dim recStr As String
                        recStr = Mid(resp, lastBrk, dEnd - lastBrk + 1)
                        Dim closeF As String
                        closeF = ParseQuotedField(recStr, 6)
                        closeF = Replace(closeF, ",", "")
                        If IsNumeric(closeF) Then
                            FetchTWSEClose = CDBl(closeF)
                            Exit Function
                        End If
                    End If
                End If
            End If
        End If
    End If
End If
End If

```

```

        End If
    End If
    dt = DateSerial(Year(dt), Month(dt) - 1, 1)
Next att
Fail:
    FetchTWSEClose = 0
End Function

' =====
' TPEX OTC STOCK CLOSE (tpex_mainboard_daily_close_quotes, cached)
' Returns 0 when stock is TWSE-listed (not in OTC list).
' =====

Public Function FetchTPExStockPrice(stockCode As String) As Double
    On Error GoTo Fail
    If Trim(stockCode) = "" Then Exit Function

    Static tpexStkCache As Object
    If tpexStkCache Is Nothing Then Set tpexStkCache = CreateObject("Scripting.Dictionary")
    Dim today As String: today = Format(Date, "yyyymmdd")

    Dim resp As String
    If Not tpexStkCache.Exists(today) Then
        Dim req As Object
        Set req = CreateObject("MSXML2.XMLHTTP")
        req.Open "GET", "https://www.tpex.org.tw/openapi/v1/tpex_mainboard_daily_close_quotes", False
        req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
        req.setRequestHeader "Referer", "https://www.tpex.org.tw/openapi/"
        req.setRequestHeader "Accept", "application/json"
        req.sEnd
        If req.status = 200 Then
            tpexStkCache(today) = req.responseText
        Else
            tpexStkCache(today) = ""
        End If
    End If
    resp = CStr(tpexStkCache(today))
    If Len(resp) < 10 Then Exit Function

    FetchTPExStockPrice = LookupTPExCode(resp, Trim(stockCode))
    Exit Function
Fail:
    FetchTPExStockPrice = 0
End Function

' =====
' TPEX OTC WARRANT CLOSE (tpex_warrant_daily_quts, cached)
' Returns 0 when warrant is TWSE-listed (not in OTC list).
' =====

Public Function FetchTPExWarrantPrice(wCode As String) As Double
    On Error GoTo Fail
    If Trim(wCode) = "" Then Exit Function

    Static tpexWrtCache As Object
    If tpexWrtCache Is Nothing Then Set tpexWrtCache = CreateObject("Scripting.Dictionary")
    Dim today As String: today = Format(Date, "yyyymmdd")

    Dim resp As String
    If Not tpexWrtCache.Exists(today) Then
        Dim req As Object
        Set req = CreateObject("MSXML2.XMLHTTP")
        req.Open "GET", "https://www.tpex.org.tw/openapi/v1/tpex_warrant_daily_quts", False
        req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
        req.setRequestHeader "Referer", "https://www.tpex.org.tw/openapi/"
        req.setRequestHeader "Accept", "application/json"
        req.sEnd
        If req.status = 200 Then
            tpexWrtCache(today) = req.responseText
        Else
            tpexWrtCache(today) = ""
        End If
    End If
    End If

```

```

    resp = CStr(tpexWrtCache(today))
    If Len(resp) < 10 Then Exit Function

```

```

    FetchTPEXWarrantPrice = LookupTPEXCode(resp, Trim(wCode))
    Exit Function

```

Fail:

```

    FetchTPEXWarrantPrice = 0
End Function

```

```

' -----
' LookupTPEXCode: shared helper for TPEX JSON bulk responses.
' Tries "Code" and "SecuritiesCompanyCode" field names; extracts
' ClosingPrice (quoted string or raw number, comma-thousand handled).
' -----

```

```

Private Function LookupTPEXCode(resp As String, code As String) As Double
    On Error Resume Next
    Dim codeKey As String: codeKey = """"Code"":"""" & code & """"
    Dim idx As Long: idx = InStr(resp, codeKey)
    If idx = 0 Then
        codeKey = """"SecuritiesCompanyCode"":"""" & code & """"
        idx = InStr(resp, codeKey)
    End If
    If idx = 0 Then Exit Function

    Dim objStart As Long: objStart = InStrRev(resp, "{", idx)
    If objStart = 0 Then objStart = idx
    Dim objEnd As Long: objEnd = InStr(idx, resp, "{")
    If objEnd = 0 Then Exit Function
    Dim obj As String: obj = Mid(resp, objStart, objEnd - objStart + 1)

    Dim s As String
    s = JsonStr(obj, "ClosingPrice")
    If s = "" Then s = JsonStr(obj, "Close")
    If s = "" Then s = JsonStr(obj, "ClosePrice")
    If s <> "" Then
        s = Replace(s, ",", "")
        If IsNumeric(s) Then LookupTPEXCode = CDBl(s): Exit Function
    End If
    Dim cl As Double
    cl = ExtractJsonNumber(obj, """"ClosingPrice"":")
    If cl <= 0 Then cl = ExtractJsonNumber(obj, """"Close"":")
    LookupTPEXCode = cl
End Function

```

```

' -----
' ParseQuotedField: extract the nth (0-based) quoted value from a
' comma-separated list where values may be quoted strings containing
' commas (e.g. "36,000" in TWSE STOCK_DAY data arrays).
' -----

```

```

Private Function ParseQuotedField(s As String, fieldIdx As Long) As String
    Dim count As Long: count = 0
    Dim p As Long: p = 1
    Do While p <= Len(s)
        If Mid(s, p, 1) = """" Then
            Dim q As Long: q = InStr(p + 1, s, """"")
            If q = 0 Then Exit Do
            If count = fieldIdx Then
                ParseQuotedField = Mid(s, p + 1, q - p - 1)
                Exit Function
            End If
            count = count + 1
            p = q + 1
        Else
            p = p + 1
        End If
    Loop
End Function

```

```

' -----
' Debug_TPEXPrices: verify TPEX API field names and coverage.
' Run from Alt+F8. Prints HTTP status + first matched object for each

```

```

' endpoint so you can confirm ClosingPrice / Code field names.
' -----
Public Sub Debug_TPEXPrices(Optional stockCode As String = "2454", _
                          Optional wCode As String = "")
    Dim req As Object, resp As String, idx As Long
    Debug.Print "=== Debug_TPEXPrices ==="

    ' --- 1. OTC stock price ---
    Set req = CreateObject("MSXML2.XMLHTTP")
    req.Open "GET", "https://www.tpex.org.tw/openapi/v1/tpex_mainboard_daily_close_quotes", False
    req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
    req.setRequestHeader "Accept", "application/json"
    req.sEnd
    Debug.Print "[1] tpex_mainboard_daily_close_quotes HTTP=" & req.status & " len=" & Len(req.responseText)
    resp = req.responseText
    idx = InStr(resp, """"Code"":"""" & stockCode & """"")
    If idx = 0 Then idx = InStr(resp, """"SecuritiesCompanyCode"":"""" & stockCode & """"")
    If idx > 0 Then
        Debug.Print "    " & stockCode & " found: " & Mid(resp, InStrRev(resp, "{", idx), InStr(idx, resp, "}")) - InStrRev(resp, "{", idx) + 1)
    Else
        Debug.Print "    " & stockCode & " NOT found in OTC list (may be TWSE-listed)"
    End If
    Debug.Print "    FetchTPEXStockPrice(" & stockCode & ") = " & FetchTPEXStockPrice(stockCode)

    ' --- 2. OTC warrant price ---
    Set req = CreateObject("MSXML2.XMLHTTP")
    req.Open "GET", "https://www.tpex.org.tw/openapi/v1/tpex_warrant_daily_quts", False
    req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
    req.setRequestHeader "Accept", "application/json"
    req.sEnd
    Debug.Print "[2] tpex_warrant_daily_quts HTTP=" & req.status & " len=" & Len(req.responseText)
    resp = req.responseText
    ' Show first object to reveal field names
    Dim firstObj As Long: firstObj = InStr(resp, "{")
    Dim firstObjEnd As Long: firstObjEnd = InStr(firstObj, resp, ",}")
    If firstObj > 0 And firstObjEnd > firstObj Then
        Debug.Print "    First object: " & Mid(resp, firstObj, firstObjEnd - firstObj + 1)
    End If
    If wCode <> "" Then
        idx = InStr(resp, """"Code"":"""" & wCode & """"")
        If idx > 0 Then
            Debug.Print "    " & wCode & " found: " & Mid(resp, InStrRev(resp, "{", idx), InStr(idx, resp, "}")) - InStrRev(resp, "{", idx) + 1)
        Else
            Debug.Print "    " & wCode & " NOT found (likely TWSE-listed)"
        End If
        Debug.Print "    FetchTPEXWarrantPrice(" & wCode & ") = " & FetchTPEXWarrantPrice(wCode)
    End If

    ' --- 3. TWSE bulk ---
    Set req = CreateObject("MSXML2.XMLHTTP")
    req.Open "GET", "https://openapi.twse.com.tw/v1/exchangeReport/STOCK_DAY_ALL", False
    req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
    req.setRequestHeader "Accept", "application/json"
    req.sEnd
    Debug.Print "[3] TWSE STOCK_DAY_ALL HTTP=" & req.status & " len=" & Len(req.responseText)
    resp = req.responseText
    firstObj = InStr(resp, "{")
    firstObjEnd = InStr(firstObj, resp, ",}")
    If firstObj > 0 And firstObjEnd > firstObj Then
        Debug.Print "    First object: " & Mid(resp, firstObj, firstObjEnd - firstObj + 1)
    End If
    If wCode <> "" Then
        idx = InStr(resp, """"Code"":"""" & wCode & """"")
        If idx > 0 Then
            Debug.Print "    " & wCode & " TWSE found, FetchTWSEClose=" & FetchTWSEClose(wCode)
        Else
            Debug.Print "    " & wCode & " not in STOCK_DAY_ALL -> will use STOCK_DAY rwd fallback"
            Debug.Print "    FetchTWSEClose(" & wCode & ") = " & FetchTWSEClose(wCode)
        End If
    End If

```

```
End If
End Sub
```

```
'
=====
' WARRANT NAME FETCH
' -----
' Primary: Yahoo Finance chart API -> meta.longName / meta.shortName
'         Tries .TW (TWSE) then .TWO (OTC/TPEX).
' Fallback: FinMind TaiwanStockInfo -> stock_name field
' Returns "" on any failure so caller can skip writing.
'
=====
' WARRANT NAME FETCH (3 sources, in order)
' 1. Yahoo Finance v7/quote -> shortName (best for TW warrants)
' 2. FinMind TaiwanStockInfo -> stock_name (needs date param)
' 3. TWSE searchSecurities -> name (TWSE-listed only)
'
=====
' WARRANT NAME FETCH (2 sources confirmed working)
'
' Method 1: TWSE afterTrading/STOCK_DAY [TWSE-listed warrants]
' URL: /rwd/zh/afterTrading/STOCK_DAY?response=json&date=YYYYMMDD&stockNo=CODE
' Title format: "115/06 051812 TSMC-YuantaSecurities-5A-Call04 daily trading info"
' Parse: token immediately after the code in the title string.
' Tries current month first, then previous month as fallback.
'
' Method 2: TPEX tpex_warrant_issue [OTC-listed warrants, bulk JSON]
' Searches the bulk list for "Code": "wCode" then extracts Name field.
'
=====
Public Function FetchWarrantName(wCode As String) As String
    On Error GoTo Fail
    If Trim(wCode) = "" Then Exit Function

    Dim req As Object, resp As String, nm As String

    ' --- Method 1: TWSE STOCK_DAY (listed warrants) ---
    Dim attempt As Long
    Dim dt As Date: dt = Date
    For attempt = 1 To 2
        Dim dateStr As String
        dateStr = Format(DateSerial(Year(dt), Month(dt), 1), "yyyymmdd")
        Set req = CreateObject("MSXML2.XMLHTTP")
        req.Open "GET", "https://www.twse.com.tw/rwd/zh/afterTrading/STOCK_DAY" & _
            "?response=json&date=" & dateStr & "&stockNo=" & wCode, False
        req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
        req.setRequestHeader "Referer", "https://www.twse.com.tw/"
        req.sEnd
        If req.status = 200 Then
            resp = req.responseText
            If InStr(resp, """"stat"":""OK""") > 0 Then
                nm = JsonStr(resp, "title")
                If nm <> "" Then
                    Dim cTag As String: cTag = wCode & " "
                    Dim cPos As Long: cPos = InStr(nm, cTag)
                    If cPos > 0 Then
                        Dim afterCode As String
                        afterCode = Mid(nm, cPos + Len(cTag))
                        Dim spPos As Long: spPos = InStr(afterCode, " ")
                        If spPos > 1 Then
                            FetchWarrantName = Trim(Left(afterCode, spPos - 1))
                            Exit Function
                        End If
                    End If
                End If
            End If
        End If
        ' Retry with previous month
        dt = DateSerial(Year(dt), Month(dt) - 1, 1)
    Next attempt

    ' --- Method 2: TPEX tpex_warrant_issue (OTC-listed warrants, bulk JSON) ---
```

```

Set req = CreateObject("MSXML2.XMLHTTP")
req.Open "GET", "https://www.tpex.org.tw/openapi/v1/tpex_warrant_issue", False
req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
req.setRequestHeader "Referer", "https://www.tpex.org.tw/openapi/"
req.setRequestHeader "Accept", "application/json"
req.sEnd
If req.status = 200 Then
    resp = req.responseText
    Dim codeKey As String: codeKey = """"Code"":"""" & wCode & """"
    Dim codeIdx As Long: codeIdx = InStr(resp, codeKey)
    If codeIdx > 0 Then
        ' Extract Name from the same JSON object (comes after Code in schema)
        Dim objEnd As Long: objEnd = InStr(codeIdx, resp, "{")
        Dim objStr As String
        objStr = Mid(resp, codeIdx, objEnd - codeIdx + 1)
        nm = JsonStr(objStr, "Name")
        If nm <> "" Then
            FetchWarrantName = nm
            Exit Function
        End If
    End If
End If
Exit Function

```

Fail:

```

    FetchWarrantName = ""
End Function

```

```

' -----
' JsonStr: extract string value for a given field name from raw JSON.
' Handles both "key": "value" and "key": "value" (space after colon).
' Returns "" if field not found or value is empty.
' -----

```

```

Private Function JsonStr(json As String, fieldName As String) As String
    Dim needle As String: needle = """" & fieldName & """"
    Dim p As Long: p = InStr(json, needle)
    If p = 0 Then Exit Function
    p = p + Len(needle)
    ' skip : and optional whitespace
    Do While p <= Len(json)
        Dim ch As String: ch = Mid(json, p, 1)
        If ch = ":" Or ch = " " Or ch = Chr(9) Then
            p = p + 1
        Else
            Exit Do
        End If
    Loop
    ' expect opening quote
    If Mid(json, p, 1) <> """" Then Exit Function
    p = p + 1
    ' find closing quote (not escaped)
    Dim q As Long: q = p
    Do While q <= Len(json)
        If Mid(json, q, 1) = """" And (q = 1 Or Mid(json, q - 1, 1) <> "\") Then
            Exit Do
        End If
        q = q + 1
    Loop
    If q > p Then JsonStr = Mid(json, p, q - p)
End Function

```

```

' -----
' Debug_WarrantName: run from Alt+F8 to see exactly what each API returns.
' Change wCode to the warrant code you want to test.
' -----
' Debug_WarrantName: run from Alt+F8 to verify name fetch for any code.
' -----

```

```

Public Sub Debug_WarrantName(Optional wCode As String = "051812")
    Dim req As Object, resp As String
    Debug.Print "=== Debug_WarrantName: " & wCode & " ==="

```

```

' --- Test 1: TWSE STOCK_DAY (primary, confirmed working) ---
Dim dateStr As String: dateStr = Format(DateSerial(Year(Date), Month(Date), 1), "yyyymmdd")
Set req = CreateObject("MSXML2.XMLHTTP")
req.Open "GET", "https://www.twse.com.tw/rwd/zh/afterTrading/STOCK_DAY" & _
    "?response=json&date=" & dateStr & "&stockNo=" & wCode, False
req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
req.setRequestHeader "Referer", "https://www.twse.com.tw/"
req.sEnd
Debug.Print "[1] TWSE STOCK_DAY HTTP=" & req.status & " date=" & dateStr
resp = req.responseText
Dim nm As String: nm = JsonStr(resp, "title")
Debug.Print "    title -> [" & nm & "]"
Debug.Print "    FetchWarrantName result -> [" & FetchWarrantName(wCode) & "]"

' --- Test 2: TPEx tpex_warrant_issue (OTC warrants, bulk search) ---
Set req = CreateObject("MSXML2.XMLHTTP")
req.Open "GET", "https://www.tpex.org.tw/openapi/v1/tpex_warrant_issue", False
req.setRequestHeader "User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
req.setRequestHeader "Referer", "https://www.tpex.org.tw/openapi/"
req.setRequestHeader "Accept", "application/json"
req.sEnd
Debug.Print "[2] TPEx tpex_warrant_issue HTTP=" & req.status & " len=" & Len(req.responseText)
Dim codeKey As String: codeKey = """"Code"":"""" & wCode & """"
Dim codeIdx As Long: codeIdx = InStr(req.responseText, codeKey)
If codeIdx > 0 Then
    Debug.Print "    FOUND at " & codeIdx & ": " & Mid(req.responseText, codeIdx, 150)
Else
    Debug.Print "    Code " & wCode & " NOT found (likely TWSE-listed, not OTC)"
End If
End Sub

```

Option Explicit

```

=====
' RR5 DASHBOARD AGGREGATIONS
'-----
' One-shot setup:
'   FixWorksheetFormulas    -- patches 4 bugs in B1 / D2 / F2 / H2
'
' Main routine:
'   DashboardRefresh        -- pulls Realized / IVTracker / HistoryLog /
'                             Greeks_History aggregates, writes rows 3-4
'
' Helpers (private): WriteRealizedMetrics / WriteIVMetrics /
'                     WriteHistoryLogMetrics / WriteGreeksHistoryMetrics /
'                     WritePositionMetrics / EnsureDashboardLabels
=====

```

```

=====
' ONE-SHOT: Archive GreeksView (RR5 now replaces it)
'-----
' Renames "GreeksView" -> "_GreeksView_Archive" and hides it via
' xlSheetVeryHidden (only re-shown by editing VBA properties).
' Non-destructive: undo by running RestoreGreeksView.
=====

```

```

Public Sub ArchiveGreeksView()
    Dim wS As Worksheet
    On Error Resume Next
    Set wS = ThisWorkbook.Sheets("GreeksView")
    On Error GoTo 0
    If wS Is Nothing Then
        MsgBox "GreeksView already archived or never existed.", vbInformation
        Exit Sub
    End If

    On Error GoTo Fail
    wS.Name = "_GreeksView_Archive"
    wS.Visible = xlSheetVeryHidden    ' xlSheetVeryHidden = 2

    MsgBox "GreeksView archived as '_GreeksView_Archive' (very hidden)." & vbCrLf & _
        "Run RestoreGreeksView to undo.", vbInformation
    Exit Sub
Fail:
    MsgBox "ArchiveGreeksView failed: " & Err.Description, vbCritical
End Sub

```

```

Public Sub RestoreGreeksView()
    Dim wS As Worksheet
    On Error Resume Next
    Set wS = ThisWorkbook.Sheets("_GreeksView_Archive")
    On Error GoTo 0
    If wS Is Nothing Then
        MsgBox "No archived GreeksView found.", vbInformation
        Exit Sub
    End If

    On Error GoTo Fail
    wS.Visible = xlSheetVisible
    wS.Name = "GreeksView"
    MsgBox "GreeksView restored.", vbInformation
    Exit Sub
Fail:
    MsgBox "RestoreGreeksView failed: " & Err.Description, vbCritical
End Sub

```

```

=====
' ONE-SHOT: Rewrite summary-bar formulas to COMPACT layout (8 rows/sec)
'-----
' Futures  data rows 7-14  (subtotal 15)
' Options  data rows 19-26 (subtotal 27)
' Warrants data rows 31-38 (subtotal 39)
' GrandTotal row 41

```



```

Public Sub FixWorksheetFormulas()
    On Error GoTo Fail
    Dim wsR As Worksheet
    Set wsR = ThisWorkbook.Sheets("RR5")

    ' --- Row 1 (label / Total MV / Net Delta / Net Gamma / Positions) ---
    wsR.Range("B1").Formula = _
        "=IFERROR(SUBTOTAL(9,N7:N14)+SUBTOTAL(9,Q19:Q26)+SUBTOTAL(9,Q31:Q38),0)"
    wsR.Range("L1").Formula = _
        "=IFERROR(SUBTOTAL(9,S7:S14)+SUBTOTAL(9,Y19:Y26)" & _
        "+SUMPRODUCT((A31:A38<>""""))*M31:M38*T31:T38),0)"
    wsR.Range("N1").Formula = _
        "=IFERROR(SUMPRODUCT((H19:H26<>""""))*U19:U26*L19:L26*M19:M26)+X39,0)"
    wsR.Range("P1").Formula = "=COUNTA(A7:A14)+COUNTA(A19:A26)+COUNTA(A31:A38)"

    ' --- Row 2 (UNRL PNL% / UNRL PNL / RL PNL / NET THETA / NET VEGA / MARGIN) ---
    wsR.Range("D2").Formula = _
        "=IFERROR((SUBTOTAL(9,O7:O14)+SUBTOTAL(9,R19:R26)+SUBTOTAL(9,R31:R38))/B1,0)"
    wsR.Range("F2").Formula = _
        "=IFERROR(SUBTOTAL(9,O7:O14)+SUBTOTAL(9,R19:R26)+SUBTOTAL(9,R31:R38),0)"
    wsR.Range("H2").Formula = "=IFERROR(SUM(Realized!I:I),0)"
    wsR.Range("J2").Formula = "=IFERROR(SUBTOTAL(9,Z19:Z26)+Y39,0)"
    wsR.Range("L2").Formula = "=IFERROR(SUBTOTAL(9,AA19:AA26)+Z39,0)"
    wsR.Range("N2").Formula = "=IFERROR(SUBTOTAL(9,Q7:Q14),0)"

    MsgBox "Summary bar formulas rewritten for compact layout:" & vbCrLf & _
        " Futures N7:N14 (NET EXPOS), O7:O14 (UNRL PNL)" & vbCrLf & _
        " Options Q19:Q26 (NET EXPOS), R19:R26 (UNRL PNL)" & vbCrLf & _
        " Warrants Q31:Q38 (NET EXPOS), R31:R38 (UNRL PNL)" & vbCrLf & vbCrLf & _
        "Don't forget to also update:" & vbCrLf & _
        " - row 15 Subtotal Futures formulas" & vbCrLf & _
        " - row 27 Subtotal Options formulas" & vbCrLf & _
        " - row 39 Subtotal Warrants formulas (X/Y/Z need SUMPRODUCT)" & vbCrLf & _
        " - row 41 Grand Total formulas", _
        vbInformation, "Worksheet Patch OK"

    Exit Sub
Fail:
    MsgBox "FixWorksheetFormulas failed: " & Err.Description, vbCritical
End Sub

```

```

' =====
' ONE-SHOT: Write all PER-ROW formulas at compact layout rows.
' -----
' Restores DTE / % CHG / NET EXPOS / UNRL PNL / NET DELTA / WT% / etc.
' formulas at the new positions (Fut 7-14, Opt 19-26, Wrt 31-38).
' Excel auto-adjusts row references when Formula is set on a range,
' so each cell ends up with the correct row number.
' =====

```

```

Public Sub Setup_CompactRowFormulas()
    On Error GoTo Fail
    Dim wsR As Worksheet: Set wsR = ThisWorkbook.Sheets("RR5")

    ' ----- Futures rows 7-14 -----
    wsR.Range("E7:E14").Formula = _
        "=IF(D7=""", """, MAX(0,D7-TODAY()))"
    wsR.Range("M7:M14").Formula = _
        "=IF(OR(K7=""", L7=""", """, IFERROR((L7-K7)/K7, """)))"
    wsR.Range("N7:N14").Formula = _
        "=IF(OR(I7=""", J7=""", L7=""", """, IF(H7=""Long"", I7*J7*L7,-I7*J7*L7)))"
    wsR.Range("O7:O14").Formula = _
        "=IF(OR(I7=""", J7=""", K7=""", L7=""", """, IF(H7=""Long"", I7*J7*(L7-K7),-I7*J7*(L7-K7))))"
    wsR.Range("Q7:Q14").Formula = _
        "=IF(OR(I7=""", P7=""", """, I7*P7))"
    wsR.Range("S7:S14").Formula = _
        "=IF(OR(I7=""", J7=""", R7=""", """, IF(H7=""Long"", I7*J7*R7,-I7*J7*R7)))"
    wsR.Range("V7:V14").Formula = _
        "=IF(OR(N7=""", 'RR5'!$B$1=0), """, N7/'RR5'!$B$1)"

    ' ----- Options rows 19-26 -----
    wsR.Range("E19:E26").Formula = _

```

```

"=IF(D19="","",MAX(0,D19-TODAY()))"
wsR.Range("P19:P26").Formula = _
"=IF(OR(N19="","",O19=""),"",IFERROR((O19-N19)/N19,""))"
wsR.Range("Q19:Q26").Formula = _
"=IF(OR(L19="","",M19="","",O19=""),"",&_
"IF(G19="CBOE",L19*M19*O19*'RR5'!$D$1,L19*M19*O19))"
wsR.Range("R19:R26").Formula = _
"=IF(OR(L19="","",M19="","",N19="","",O19=""),"",&_
"IF(G19="CBOE",L19*M19*(O19-N19)*'RR5'!$D$1,L19*M19*(O19-N19)))"
wsR.Range("Y19:Y26").Formula = _
"=IF(OR(L19="","",M19="","",T19=""),"",&_
"IF(H19="Put",-L19*M19*ABS(T19),L19*M19*T19))"
wsR.Range("Z19:Z26").Formula = _
"=IF(OR(L19="","",M19="","",V19=""),"",L19*M19*V19)"
wsR.Range("AA19:AA26").Formula = _
"=IF(OR(L19="","",M19="","",W19=""),"",L19*M19*W19)"
wsR.Range("AC19:AC26").Formula = _
"=IF(OR(Q19="","",RR5'!$B$1=0),"",Q19/'RR5'!$B$1)"

' ----- Warrants rows 31-38 -----
wsR.Range("E31:E38").Formula = _
"=IF(D31="","",MAX(0,D31-TODAY()))"
wsR.Range("P31:P38").Formula = _
"=IF(OR(N31="","",O31=""),"",IFERROR((O31-N31)/N31,""))"
wsR.Range("Q31:Q38").Formula = _
"=IF(OR(M31="","",O31=""),"",M31*O31)"
wsR.Range("R31:R38").Formula = _
"=IF(OR(M31="","",N31="","",O31=""),"",M31*(O31-N31))"
wsR.Range("U31:U38").Formula = _
"=IF(OR(O31="","",J31="","",K31="","",L31="","",O31=0),"",&_
"IFERROR((O31-MAX(0,(J31-K31)/L31))/O31,""))"
wsR.Range("V31:V38").Formula = "E31"
wsR.Range("W31:W38").Formula = _
"=IF(OR(Q31="","",RR5'!$B$1=0),"",Q31/'RR5'!$B$1)"

MsgBox "Per-row formulas restored:" & vbCrLf & _
" Futures rows 7-14 (E/M/N/O/Q/S/V)" & vbCrLf & _
" Options rows 19-26 (E/P/Q/R/Y/Z/AA/AC)" & vbCrLf & _
" Warrants rows 31-38 (E/P/Q/R/U/V/W)" & vbCrLf & vbCrLf & _
"Now DTE/% CHG/NET EXPOS/UNRL PNL/WT% etc. auto-compute.", _
vbInformation, "Compact Row Formulas OK"
Exit Sub
Fail:
MsgBox "Setup_CompactRowFormulas failed: " & Err.Description, vbCritical
End Sub

' =====
' ONE-SHOT: Write subtotal + grand-total formulas at the COMPACT layout
' rows (15, 27, 39, 41). Run AFTER FixWorksheetFormulas.
' =====

Public Sub Setup_CompactSubtotals()
On Error GoTo Fail
Dim wsR As Worksheet: Set wsR = ThisWorkbook.Sheets("RR5")

' --- Row 15: Futures subtotal (N=NET EXPOS, O=UNRL PNL, Q=MARGIN, S=NET DELTA, V=WT%) ---
wsR.Range("A15").Value = " Subtotal Futures"
wsR.Range("N15").Formula = "=SUBTOTAL(9,N7:N14)"
wsR.Range("O15").Formula = "=SUBTOTAL(9,O7:O14)"
wsR.Range("Q15").Formula = "=SUBTOTAL(9,Q7:Q14)"
wsR.Range("S15").Formula = "=SUBTOTAL(9,S7:S14)"
wsR.Range("V15").Formula = "=SUBTOTAL(9,V7:V14)"

' --- Row 27: Options subtotal (Q=NET EXPOS, R=UNRL PNL, Y=NET DELTA, Z=NET THETA, AA=NET VEGA, AC=WT%) ---
wsR.Range("A27").Value = " Subtotal Options"
wsR.Range("Q27").Formula = "=SUBTOTAL(9,Q19:Q26)"
wsR.Range("R27").Formula = "=SUBTOTAL(9,R19:R26)"
wsR.Range("Y27").Formula = "=SUBTOTAL(9,Y19:Y26)"
wsR.Range("Z27").Formula = "=SUBTOTAL(9,Z19:Z26)"
wsR.Range("AA27").Formula = "=SUBTOTAL(9,AA19:AA26)"
wsR.Range("AC27").Formula = "=SUBTOTAL(9,AC19:AC26)"

```

```

' --- Row 39: Warrants subtotal (Q=NET EXPOS, R=UNRL PNL, W=WT%, X/Y/Z=SUMPRODUCT Greeks) ---
wsR.Range("A39").Value = " Subtotal Warrants"
wsR.Range("Q39").Formula = "=SUBTOTAL(9,Q31:Q38)"
wsR.Range("R39").Formula = "=SUBTOTAL(9,R31:R38)"
wsR.Range("W39").Formula = "=SUBTOTAL(9,W31:W38)"
' SUMPRODUCT for warrant Greeks (SHARES ? per-warrant Greek)
wsR.Range("X39").Formula = "=SUMPRODUCT((A31:A38<>\"\")*M31:M38*X31:X38)"
wsR.Range("Y39").Formula = "=SUMPRODUCT((A31:A38<>\"\")*M31:M38*Y31:Y38)"
wsR.Range("Z39").Formula = "=SUMPRODUCT((A31:A38<>\"\")*M31:M38*Z31:Z38)"

' --- Row 41: Grand Total (sum of the three subtotals) ---
wsR.Range("A41").Value = " GRAND TOTAL"
wsR.Range("N41").Formula = "=N15+Q27+Q39" ' Total NET EXPOS TWD
wsR.Range("O41").Formula = "=O15+R27+R39" ' Total UNRL PNL TWD

MsgBox "Compact subtotals written:" & vbCrLf & _
    " Row 15: Subtotal Futures" & vbCrLf & _
    " Row 27: Subtotal Options" & vbCrLf & _
    " Row 39: Subtotal Warrants (+ SUMPRODUCT for X/Y/Z Greeks)" & vbCrLf & _
    " Row 41: Grand Total" & vbCrLf & vbCrLf & _
    "(You can still style row heights / banners visually as you like.)", _
    vbInformation, "Compact Subtotals OK"

Exit Sub
Fail:
MsgBox "Setup_CompactSubtotals failed: " & Err.Description, vbCritical
End Sub

```

```

' =====
' MAIN: DashboardRefresh
' =====

```

```

Public Sub DashboardRefresh()
    On Error GoTo Fail
    Application.ScreenUpdating = False

    Dim wsR As Worksheet
    Set wsR = ThisWorkbook.Sheets("RR5")

    Call EnsureDashboardLabels(wsR)
    Call WriteRealizedMetrics(wsR)
    Call WriteIVMetrics(wsR)
    Call WriteHistoryLogMetrics(wsR)
    Call WriteGreeksHistoryMetrics(wsR)
    Call WritePositionMetrics(wsR)
    Call RefreshAnalysis ' Per-position + DTE buckets + Strategy aggregation
    Call BuildScenarioAnalysis ' F1: spot/IV scenario P&L grid
    Call BuildEquityDrawdownCharts ' F4: equity curve + drawdown

    Application.ScreenUpdating = True
    Application.StatusBar = "DashboardRefresh OK " & Format(Now, "hh:mm:ss")
    Exit Sub

```

```

Fail:
    Application.ScreenUpdating = True
    Application.StatusBar = False
    MsgBox "DashboardRefresh failed: " & Err.Description, vbExclamation
End Sub

```

```

' =====
' RefreshAnalysis -- writes Analysis sheet per-position table + aggregates
' -----
' Reads RR5 sections (Futures 7-14 / Options 19-26 / Warrants 31-38),
' writes one row per active position into Analysis starting at row 15
' (table headers are at row 14), then appends derived sections:
'   - DTE Bucket (rows 40-44): count + Idelta + theta exposure
'   - Strategy (rows 46-55): per strategy tag count + net delta + theta
'   - Underlying (rows 57-63): per underlying notional + delta exposure
' =====

```

```

Public Sub RefreshAnalysis()
    On Error GoTo Fail
    Dim wsA As Worksheet, wsR As Worksheet
    Set wsA = ThisWorkbook.Sheets("Analysis")
    Set wsR = ThisWorkbook.Sheets("RR5")

```

```

' Clear old data values only (do NOT touch user's formatting)
' D2:Q13 = new top-area summary sections (DTE / Strategy / Underlying)
' A15:Q70 = per-position table + any legacy bottom sections
wsA.Range("D2:Q13").ClearContents
wsA.Range("A15:Q70").ClearContents

Dim r As Long: r = 15

' --- Aggregation buckets while we walk positions ---
Dim dteCnt(0 To 3) As Long, dteAbsDelta(0 To 3) As Double, dteTheta(0 To 3) As Double
Dim stratDict As Object: Set stratDict = CreateObject("Scripting.Dictionary")
Dim undlDict As Object: Set undlDict = CreateObject("Scripting.Dictionary")

' --- Walk 3 sections ---
Dim sections As Variant
sections = Array(Array(7, 14, "fut"), Array(19, 26, "opt"), Array(31, 38, "wrt"))
Dim sec As Variant, i As Long

For Each sec In sections
    Dim startR As Long: startR = CLng(sec(0))
    Dim endR As Long: endR = CLng(sec(1))
    Dim kind As String: kind = CStr(sec(2))

    For i = startR To endR
        Dim code As String
        code = Trim(CStr(wsR.Cells(i, 1).Value))
        If code = "" Then GoTo nextRow

        ' Common fields
        wsA.Cells(r, 1).Value = code
        wsA.Cells(r, 2).Value = wsR.Cells(i, 2).Value

        Dim entryDt As Variant: entryDt = wsR.Cells(i, 3).Value
        If IsDate(entryDt) Then wsA.Cells(r, 4).Value = Date - CDate(entryDt)

        wsA.Cells(r, 8).Value = wsR.Cells(i, 5).Value    ' H DTE

        Dim typStr As String, undl As Variant, undlPx As Variant
        Dim strike As Variant, chgPct As Variant
        Dim delta As Variant, gamma As Variant, theta As Variant, vega As Variant
        Dim strat As String

        Select Case kind
            Case "fut"
                typStr = "Future"
                undl = wsR.Cells(i, 7).Value                ' G PRODUCT
                undlPx = wsR.Cells(i, 12).Value              ' L LAST
                strike = ""
                chgPct = wsR.Cells(i, 13).Value              ' M % CHG
                delta = wsR.Cells(i, 18).Value               ' R DELTA
                gamma = "": theta = "": vega = ""
                strat = "Futures"
            Case "opt"
                typStr = CStr(wsR.Cells(i, 8).Value)        ' H TYPE
                undl = wsR.Cells(i, 10).Value                ' J UNDERLYING
                undlPx = wsR.Cells(i, 11).Value              ' K UNDL LAST
                strike = wsR.Cells(i, 9).Value               ' I STRIKE
                chgPct = wsR.Cells(i, 16).Value              ' P % CHG
                delta = wsR.Cells(i, 20).Value               ' T DELTA
                gamma = wsR.Cells(i, 21).Value               ' U GAMMA
                theta = wsR.Cells(i, 22).Value               ' V THETA
                vega = wsR.Cells(i, 23).Value                ' W VEGA
                strat = CStr(wsR.Cells(i, 28).Value)         ' AB STRATEGY TAG
            Case "wrt"
                typStr = CStr(wsR.Cells(i, 7).Value)        ' G TYPE
                undl = wsR.Cells(i, 9).Value                 ' I UNDERLYING
                undlPx = wsR.Cells(i, 10).Value              ' J UNDL LAST
                strike = wsR.Cells(i, 11).Value              ' K STRIKE
                chgPct = wsR.Cells(i, 16).Value              ' P % CHG
                delta = wsR.Cells(i, 20).Value               ' T DELTA

```

```

        gamma = wsR.Cells(i, 24).Value      ' X GAMMA
        theta = wsR.Cells(i, 25).Value      ' Y THETA
        vega = wsR.Cells(i, 26).Value       ' Z VEGA
        strat = "Warrant"

```

```
End Select
```

```

wsA.Cells(r, 3).Value = typStr
wsA.Cells(r, 5).Value = undl
wsA.Cells(r, 6).Value = undlPx
wsA.Cells(r, 7).Value = strike
wsA.Cells(r, 9).Value = chgPct
wsA.Cells(r, 11).Value = delta
wsA.Cells(r, 12).Value = gamma
wsA.Cells(r, 13).Value = theta
wsA.Cells(r, 14).Value = vega
wsA.Cells(r, 15).Value = DeriveTrend(typStr, undlPx, strike, delta)
wsA.Cells(r, 16).Value = strat
wsA.Cells(r, 17).Value = DeriveNote(wsR.Cells(i, 5).Value, delta, chgPct)

```

```

' Number formats
wsA.Cells(r, 6).NumberFormat = "#,##0.00"
wsA.Cells(r, 7).NumberFormat = "#,##0.00"
wsA.Cells(r, 9).NumberFormat = "0.00%"
wsA.Cells(r, 11).NumberFormat = "0.0000"
wsA.Cells(r, 12).NumberFormat = "0.000000"
wsA.Cells(r, 13).NumberFormat = "0.0000"
wsA.Cells(r, 14).NumberFormat = "0.0000"

```

```
' --- Bucket: DTE ---
```

```

Dim dteBucket As Long: dteBucket = -1
If IsNumeric(wsR.Cells(i, 5).Value) Then
    Dim dteVal As Long: dteVal = CLng(wsR.Cells(i, 5).Value)
    Select Case True
        Case dteVal <= 7: dteBucket = 0
        Case dteVal <= 30: dteBucket = 1
        Case dteVal <= 60: dteBucket = 2
        Case Else: dteBucket = 3
    End Select
    dteCnt(dteBucket) = dteCnt(dteBucket) + 1
    If IsNumeric(delta) Then dteAbsDelta(dteBucket) = dteAbsDelta(dteBucket) + Abs(CDbl(delta))
    If IsNumeric(theta) Then dteTheta(dteBucket) = dteTheta(dteBucket) + CDbl(theta)
End If

```

```
' --- Bucket: Strategy ---
```

```

If strat <> "" Then
    If Not stratDict.Exists(strat) Then stratDict(strat) = Array(0&, 0#, 0#)
    Dim sArr As Variant: sArr = stratDict(strat)
    sArr(0) = sArr(0) + 1
    If IsNumeric(delta) Then sArr(1) = sArr(1) + CDbl(delta)
    If IsNumeric(theta) Then sArr(2) = sArr(2) + CDbl(theta)
    stratDict(strat) = sArr
End If

```

```
' --- Bucket: Underlying ---
```

```

Dim uKey As String: uKey = Trim(CStr(undl))
If uKey <> "" Then
    If Not undlDict.Exists(uKey) Then undlDict(uKey) = Array(0&, 0#, 0#)
    Dim uArr As Variant: uArr = undlDict(uKey)
    uArr(0) = uArr(0) + 1
    If IsNumeric(delta) Then uArr(1) = uArr(1) + CDbl(delta)
    If IsNumeric(undlPx) Then uArr(2) = uArr(2) + CDbl(undlPx)
    undlDict(uKey) = uArr
End If

```

```
r = r + 1
```

```
nextRow:
```

```
Next i
```

```
Next sec
```

```

' =====
' Top-area summary sections (col D / I / N) alongside KPI bar (col A-B)

```

```

' =====
' --- DTE Bucket (cols D-G, rows 2-7) ---
wsA.Cells(2, 4).Value = "DTE BUCKET"
wsA.Cells(3, 4).Value = "Bucket"
wsA.Cells(3, 5).Value = "Count"
wsA.Cells(3, 6).Value = "Sum |Delta|"
wsA.Cells(3, 7).Value = "Sum Theta/day"
Dim labels As Variant: labels = Array("<=7d", "8-30d", "31-60d", ">60d")
Dim b As Long
For b = 0 To 3
    wsA.Cells(4 + b, 4).Value = labels(b)
    wsA.Cells(4 + b, 5).Value = dteCnt(b)
    wsA.Cells(4 + b, 6).Value = dteAbsDelta(b): wsA.Cells(4 + b, 6).NumberFormat = "0.0000"
    wsA.Cells(4 + b, 7).Value = dteTheta(b): wsA.Cells(4 + b, 7).NumberFormat = "0.0000"
Next b

' --- Strategy (cols I-L, rows 2 onwards) ---
wsA.Cells(2, 9).Value = "STRATEGY"
wsA.Cells(3, 9).Value = "Strategy"
wsA.Cells(3, 10).Value = "Count"
wsA.Cells(3, 11).Value = "Sum Delta"
wsA.Cells(3, 12).Value = "Sum Theta/day"
Dim sr As Long: sr = 4
Dim sKey As Variant
For Each sKey In stratDict.keys
    Dim sv As Variant: sv = stratDict(sKey)
    wsA.Cells(sr, 9).Value = sKey
    wsA.Cells(sr, 10).Value = sv(0)
    wsA.Cells(sr, 11).Value = sv(1): wsA.Cells(sr, 11).NumberFormat = "0.0000"
    wsA.Cells(sr, 12).Value = sv(2): wsA.Cells(sr, 12).NumberFormat = "0.0000"
    sr = sr + 1
    If sr > 13 Then Exit For ' max 10 strategies (rows 4-13)
Next sKey

' --- Underlying (cols N-Q, rows 2 onwards) ---
wsA.Cells(2, 14).Value = "UNDERLYING"
wsA.Cells(3, 14).Value = "Underlying"
wsA.Cells(3, 15).Value = "Pos Count"
wsA.Cells(3, 16).Value = "Sum Delta"
wsA.Cells(3, 17).Value = "Avg Spot"
Dim ur As Long: ur = 4
Dim uKeyV As Variant
For Each uKeyV In undlDict.keys
    Dim uV As Variant: uV = undlDict(uKeyV)
    wsA.Cells(ur, 14).Value = uKeyV
    wsA.Cells(ur, 15).Value = uV(0)
    wsA.Cells(ur, 16).Value = uV(1): wsA.Cells(ur, 16).NumberFormat = "0.0000"
    If uV(0) > 0 Then
        wsA.Cells(ur, 17).Value = uV(2) / uV(0)
        wsA.Cells(ur, 17).NumberFormat = "#,##0.00"
    End If
    ur = ur + 1
    If ur > 10 Then Exit For ' max 7 underlyings (rows 4-10)
Next uKeyV

Exit Sub
Fail:
    MsgBox "RefreshAnalysis failed: " & Err.Description, vbExclamation
End Sub

```

```

' =====
' SCENARIO ANALYSIS (F1)
' -----
' Re-evaluates every option / warrant leg under bumped spot / IV scenarios
' using Black-Scholes. Aggregates portfolio-level P&L impact and writes
' a small grid to Analysis worksheet rows 40-46.
'
' Scenarios:
' Spot bumps: -5% / -3% / -1% / 0 / +1% / +3% / +5%
' IV bumps: -5pp / 0 / +5pp (absolute, not relative; e.g. 0.20 -> 0.25)

```

```

' Output layout (Analysis sheet):
'   Row 40 "SCENARIO ANALYSIS (Portfolio P&L, TWD)"
'   Row 41 Header: "Spot Bump" | "-5%" | "-3%" | "-1%" | "0" | "+1%" | "+3%" | "+5%" | (sep) | "IV-5pp" | "IV+5pp"
'   Row 42 PnL row: <name> | val | val | val | val | val | val | val | (sep) | val | val
'
' Futures contribute via direct spot delta (linear, 1:1 P&L per point per contract * mult).
' Equity/unknown types skipped.
'
' Errors at each stage are caught and reported with stage tag for
' rapid diagnosis (e.g. "F1 Stage 2 read positions: ...").

```

```

Public Sub BuildScenarioAnalysis()
    Dim stage As String: stage = "init"
    On Error GoTo Fail

    stage = "open sheets"
    Dim wsA As Worksheet, wsR As Worksheet, wsC As Worksheet
    Set wsA = ThisWorkbook.Sheets("Analysis")
    Set wsR = ThisWorkbook.Sheets("RR5")
    Set wsC = ThisWorkbook.Sheets("_Config")

    stage = "read risk-free rate"
    Dim rRate As Double
    rRate = ToDblD(GetConfigVal(wsC, "BS_RISK_FREE_RATE"))
    If rRate <= 0 Then rRate = 0.02

    stage = "define scenarios"
    Dim spotBumps As Variant: spotBumps = Array(-0.05, -0.03, -0.01, 0#, 0.01, 0.03, 0.05)
    Dim ivBumps As Variant: ivBumps = Array(-0.05, 0.05) ' absolute pp

    stage = "clear old output rows 40-50"
    wsA.Range("A40:Q50").ClearContents

    stage = "write headers"
    wsA.Cells(40, 1).Value = "SCENARIO ANALYSIS (Portfolio P&L, TWD)"
    wsA.Cells(41, 1).Value = "Spot Bump"
    Dim h As Long
    For h = 0 To UBound(spotBumps)
        wsA.Cells(41, 2 + h).Value = ScenarioLabel(CDbl(spotBumps(h)), "pct")
    Next h
    wsA.Cells(41, 2 + UBound(spotBumps) + 1).Value = "I"
    For h = 0 To UBound(ivBumps)
        wsA.Cells(41, 2 + UBound(spotBumps) + 2 + h).Value = ScenarioLabel(CDbl(ivBumps(h)), "ivpp")
    Next h

    ' Initialize accumulator arrays
    Dim nSpot As Long: nSpot = UBound(spotBumps) + 1
    Dim nIv As Long: nIv = UBound(ivBumps) + 1
    Dim pnlSpot() As Double: ReDim pnlSpot(0 To nSpot - 1)
    Dim pnlIV() As Double: ReDim pnlIV(0 To nIv - 1)

    stage = "scan Options rows 19-26"
    Call ScanScenarioOptionsSection(wsR, 19, 26, rRate, spotBumps, ivBumps, pnlSpot, pnlIV)

    stage = "scan Warrants rows 31-38"
    Call ScanScenarioWarrantsSection(wsR, 31, 38, rRate, spotBumps, ivBumps, pnlSpot, pnlIV)

    stage = "scan Futures rows 7-14 (linear delta only)"
    Call ScanScenarioFuturesSection(wsR, 7, 14, spotBumps, pnlSpot)

    stage = "write portfolio P&L row"
    wsA.Cells(42, 1).Value = "Portfolio P&L"
    Dim sc As Long
    For sc = 0 To nSpot - 1
        wsA.Cells(42, 2 + sc).Value = pnlSpot(sc)
        wsA.Cells(42, 2 + sc).NumberFormat = "#,##0;[Red](#,##0)"
    Next sc
    For sc = 0 To nIv - 1
        wsA.Cells(42, 2 + nSpot + 1 + sc).Value = pnlIV(sc)
        wsA.Cells(42, 2 + nSpot + 1 + sc).NumberFormat = "#,##0;[Red](#,##0)"
    Next sc

```

Next sc

stage = "done"

Application.StatusBar = "Scenario Analysis OK " & Format(Now, "hh:mm:ss")

Exit Sub

Fail:

Application.StatusBar = False

MsgBox "BuildScenarioAnalysis failed at stage [" & stage & "]: " & vbCrLf & _
Err.Description, vbExclamation, "Fl Scenario Analysis"

End Sub

' --- Helpers -----

Private Sub ScanScenarioOptionsSection(wsR As Worksheet, rStart As Long, rEnd As Long, _
rRate As Double, spotBumps As Variant, ivBumps As Variant, _
ByRef pnlSpot() As Double, ByRef pnlIV() As Double)

Dim stage As String: stage = "opt init"

On Error GoTo Fail

Dim i As Long

For i = rStart To rEnd

stage = "opt row " & i & " read"

Dim contract As String: contract = Trim(CStr(wsR.Cells(i, 1).Value))

If contract = "" Then GoTo NextI

Dim s As Double: s = ToDbld(wsR.Cells(i, 11).Value) ' K UNDL LAST

Dim k As Double: k = ToDbld(wsR.Cells(i, 9).Value) ' I STRIKE

Dim dte As Double: dte = ToDbld(wsR.Cells(i, 5).Value) ' E DTE

Dim iv As Double: iv = ToDbld(wsR.Cells(i, 19).Value) ' S IV

Dim signedContracts As Double: signedContracts = ToDbld(wsR.Cells(i, 12).Value) ' L

Dim mult As Double: mult = ToDbld(wsR.Cells(i, 13).Value) ' M

Dim oType As String: oType = CStr(wsR.Cells(i, 8).Value) ' H

Dim curPrem As Double: curPrem = ToDbld(wsR.Cells(i, 15).Value) ' O LAST PREM

If s <= 0 Or k <= 0 Or dte <= 0 Or iv <= 0 Then GoTo NextI

If signedContracts = 0 Or mult = 0 Then GoTo NextI

Dim t As Double: t = dte / 365#

stage = "opt row " & i & " baseline price"

Dim basePx As Double

basePx = BS_Price(s, k, t, rRate, iv, oType)

' If LAST PREM is filled, use it as baseline (more accurate). Else BS theoretical.

Dim baseline As Double

If curPrem > 0 Then baseline = curPrem Else baseline = basePx

' --- spot scenarios ---

stage = "opt row " & i & " spot scenarios"

Dim sb As Long

For sb = 0 To UBound(spotBumps)

Dim SN As Double: SN = s * (1 + CDbl(spotBumps(sb)))

Dim newPx As Double: newPx = BS_Price(SN, k, t, rRate, iv, oType)

Dim deltaPx As Double: deltaPx = newPx - basePx ' use BS-based delta to avoid baseline noise

pnlSpot(sb) = pnlSpot(sb) + signedContracts * mult * deltaPx

Next sb

' --- iv scenarios ---

stage = "opt row " & i & " iv scenarios"

Dim ib As Long

For ib = 0 To UBound(ivBumps)

Dim ivN As Double: ivN = iv + CDbl(ivBumps(ib))

If ivN < 0.001 Then ivN = 0.001

Dim ivPx As Double: ivPx = BS_Price(s, k, t, rRate, ivN, oType)

pnlIV(ib) = pnlIV(ib) + signedContracts * mult * (ivPx - basePx)

Next ib

NextI:

Next i

Exit Sub

Fail:

Err.Raise Err.Number, "ScanScenarioOptionsSection / " & stage, Err.Description

End Sub


```
Private Sub ScanScenarioWarrantsSection(wsR As Worksheet, rStart As Long, rEnd As Long, _
    rRate As Double, spotBumps As Variant, ivBumps As Variant, _
    ByRef pnlSpot() As Double, ByRef pnlIV() As Double)
```

```
    Dim stage As String: stage = "wrt init"
```

```
    On Error GoTo Fail
```

```
    Dim i As Long
```

```
    For i = rStart To rEnd
```

```
        stage = "wrt row " & i & " read"
```

```
        Dim code As String: code = Trim(CStr(wsR.Cells(i, 1).Value))
```

```
        If code = "" Then GoTo NextI
```

```
        Dim s As Double: s = ToDbld(wsR.Cells(i, 10).Value) ' J UNDL LAST
```

```
        Dim k As Double: k = ToDbld(wsR.Cells(i, 11).Value) ' K STRIKE
```

```
        Dim conv As Double: conv = ToDbld(wsR.Cells(i, 12).Value) ' L CONV RATIO
```

```
        Dim shares As Double: shares = ToDbld(wsR.Cells(i, 13).Value) ' M
```

```
        Dim dte As Double: dte = ToDbld(wsR.Cells(i, 5).Value)
```

```
        Dim iv As Double: iv = ToDbld(wsR.Cells(i, 19).Value)
```

```
        Dim wType As String: wType = CStr(wsR.Cells(i, 7).Value)
```

```
        If s <= 0 Or k <= 0 Or conv <= 0 Or shares = 0 Or dte <= 0 Or iv <= 0 Then GoTo NextI
```

```
        Dim t As Double: t = dte / 365#
```

```
        Dim oTypeW As String
```

```
        If InStr(UCase(wType), "CALL") > 0 Or InStr(UCase(wType), "BULL") > 0 Then
```

```
            oTypeW = "Call"
```

```
        Else
```

```
            oTypeW = "Put"
```

```
        End If
```

```
        stage = "wrt row " & i & " baseline"
```

```
        Dim baseW As Double: baseW = BS_Price(s, k, t, rRate, iv, oTypeW) / conv
```

```
        stage = "wrt row " & i & " spot bumps"
```

```
        Dim sb As Long
```

```
        For sb = 0 To UBound(spotBumps)
```

```
            Dim SN As Double: SN = s * (1 + CDbld(spotBumps(sb)))
```

```
            Dim newW As Double: newW = BS_Price(SN, k, t, rRate, iv, oTypeW) / conv
```

```
            pnlSpot(sb) = pnlSpot(sb) + shares * (newW - baseW)
```

```
        Next sb
```

```
        stage = "wrt row " & i & " iv bumps"
```

```
        Dim ib As Long
```

```
        For ib = 0 To UBound(ivBumps)
```

```
            Dim ivN As Double: ivN = iv + CDbld(ivBumps(ib))
```

```
            If ivN < 0.001 Then ivN = 0.001
```

```
            Dim ivW As Double: ivW = BS_Price(s, k, t, rRate, ivN, oTypeW) / conv
```

```
            pnlIV(ib) = pnlIV(ib) + shares * (ivW - baseW)
```

```
        Next ib
```

```
NextI:
```

```
    Next i
```

```
    Exit Sub
```

```
Fail:
```

```
    Err.Raise Err.Number, "ScanScenarioWarrantsSection / " & stage, Err.Description
```

```
End Sub
```

```
Private Sub ScanScenarioFuturesSection(wsR As Worksheet, rStart As Long, rEnd As Long, _
    spotBumps As Variant, ByRef pnlSpot() As Double)
```

```
    Dim stage As String: stage = "fut init"
```

```
    On Error GoTo Fail
```

```
    Dim i As Long
```

```
    For i = rStart To rEnd
```

```
        stage = "fut row " & i
```

```
        Dim contract As String: contract = Trim(CStr(wsR.Cells(i, 1).Value))
```

```
        If contract = "" Then GoTo NextI
```

```
        Dim s As Double: s = ToDbld(wsR.Cells(i, 12).Value) ' L LAST
```

```
        Dim contracts As Double: contracts = ToDbld(wsR.Cells(i, 9).Value) ' I CONTRACTS
```

```
        Dim mult As Double: mult = ToDbld(wsR.Cells(i, 10).Value) ' J MULTIPLIER
```

```

Dim direction As String: direction = CStr(wsR.Cells(i, 8).Value) ' H DIRECTION

If s <= 0 Or contracts <= 0 Or mult <= 0 Then GoTo NextI
Dim sign As Long: sign = IIf(UCase(direction) = "LONG", 1, -1)

Dim sb As Long
For sb = 0 To UBound(spotBumps)
    Dim deltaS As Double: deltaS = s * CDb1(spotBumps(sb))
    pnlSpot(sb) = pnlSpot(sb) + sign * contracts * mult * deltaS
Next sb

NextI:
Next i
Exit Sub

Fail:
Err.Raise Err.Number, "ScanScenarioFuturesSection / " & stage, Err.Description
End Sub

Private Function ScenarioLabel(v As Double, kind As String) As String
    If kind = "pct" Then
        If v = 0 Then ScenarioLabel = "0" Else ScenarioLabel = Format(v, "+0%;-0%")
    Else
        If v = 0 Then ScenarioLabel = "IV 0" Else ScenarioLabel = "IV" & Format(v * 100, "+0;-0") & "pp"
    End If
End Function

Private Function GetConfigVal(wsC As Worksheet, key As String) As Variant
    Dim i As Long
    For i = 1 To 30
        If wsC.Cells(i, 1).Value = key Then
            GetConfigVal = wsC.Cells(i, 2).Value
            Exit Function
        End If
    Next i
    GetConfigVal = ""
End Function

Private Function ToDb1D(v As Variant) As Double
    If IsNumeric(v) Then ToDb1D = CDb1(v) Else ToDb1D = 0
End Function

' =====
' END SCENARIO ANALYSIS (F1)
' =====

' =====
' F4: EQUITY CURVE + DRAWDOWN CHART
' -----
' Reads HistoryLog (cols A=Date, B=TotalMarketValue), writes two
' chart-helper data tables to Analysis worksheet rows 55-80, then
' creates/updates two ChartObjects: "EquityCurve" + "DrawdownChart".
'
' Drawdown formula: dd_t = (MV_t - peak_t) / peak_t where peak_t =
' running max of MV up to time t. Always <= 0.
'
' Output layout (Analysis):
' Row 55 "EQUITY CURVE DATA"
' Row 56 Date | Total MV | Drawdown%
' Row 57+ time series
'
' Charts anchored to D52 (Equity) and L52 (Drawdown).
' =====

Public Sub BuildEquityDrawdownCharts()
    Dim stage As String: stage = "init"
    On Error GoTo Fail

    stage = "open sheets"
    Dim wsA As Worksheet, wsH As Worksheet
    Set wsA = ThisWorkbook.Sheets("Analysis")
    Set wsH = ThisWorkbook.Sheets("HistoryLog")

    stage = "read HistoryLog"

```

```

Dim lr As Long: lr = wsH.Cells(wsH.Rows.count, "A").End(xlUp).row
If lr < 3 Then
    MsgBox "HistoryLog has fewer than 2 data rows -- cannot draw charts.", _
        vbInformation, "F4 Equity Curve"
    Exit Sub
End If

stage = "clear old data table"
wsA.Range("A55:C200").ClearContents
wsA.Cells(55, 1).Value = "EQUITY CURVE DATA"
wsA.Cells(56, 1).Value = "Date"
wsA.Cells(56, 2).Value = "Total MV"
wsA.Cells(56, 3).Value = "Drawdown%"

stage = "iterate HistoryLog and compute drawdown"
Dim peak As Double: peak = 0
Dim outR As Long: outR = 57
Dim i As Long
For i = 2 To lr
    Dim dt As Variant: dt = wsH.Cells(i, 1).Value
    Dim mv As Variant: mv = wsH.Cells(i, 2).Value
    If Not IsDate(dt) Then GoTo NextI
    If Not IsNumeric(mv) Then GoTo NextI
    If CDBl(mv) <= 0 Then GoTo NextI

    If CDBl(mv) > peak Then peak = CDBl(mv)
    Dim dd As Double
    If peak > 0 Then dd = (CDBl(mv) - peak) / peak Else dd = 0

    wsA.Cells(outR, 1).Value = CDate(dt)
    wsA.Cells(outR, 1).NumberFormat = "yyyy/m/d"
    wsA.Cells(outR, 2).Value = CDBl(mv)
    wsA.Cells(outR, 2).NumberFormat = "#,##0"
    wsA.Cells(outR, 3).Value = dd
    wsA.Cells(outR, 3).NumberFormat = "0.00%"
    outR = outR + 1
NextI:
Next i
Dim lastR As Long: lastR = outR - 1
If lastR < 58 Then
    MsgBox "Not enough valid HistoryLog rows.", vbInformation
    Exit Sub
End If

stage = "build/update Equity Curve chart"
Call EnsureChart(wsA, "EquityCurve", "D52", _
    wsA.Range(wsA.Cells(57, 1), wsA.Cells(lastR, 1)), _
    wsA.Range(wsA.Cells(57, 2), wsA.Cells(lastR, 2)), _
    "Equity Curve (Total MV)", "Date", "Total MV (TWD)", _
    Nothing, Nothing, "")

stage = "build/update Drawdown chart"
Call EnsureChart(wsA, "DrawdownChart", "L52", _
    wsA.Range(wsA.Cells(57, 1), wsA.Cells(lastR, 1)), _
    wsA.Range(wsA.Cells(57, 3), wsA.Cells(lastR, 3)), _
    "Drawdown Underwater", "Date", "Drawdown %", _
    Nothing, Nothing, "")

Application.StatusBar = "Equity + Drawdown charts updated"
Exit Sub

Fail:
Application.StatusBar = False
MsgBox "BuildEquityDrawdownCharts failed at stage [" & stage & "]: " & vbCrLf & _
    Err.Description, vbExclamation, "F4 Equity Curve"
End Sub

' --- Helper: find or create a chart anchored to a cell, with 1 series ---
Private Sub EnsureChart(wsA As Worksheet, chartName As String, anchorCell As String, _
    xRange As Range, yRange As Range, _
    title As String, xLabel As String, yLabel As String, _
    Optional zX As Variant, Optional zY As Variant, _

```

```

Optional extraTitle As String = "")
Dim co As ChartObject, cc As ChartObject
Dim found As Boolean: found = False
For Each cc In wsA.ChartObjects
    If cc.Name = chartName Then
        Set co = cc
        found = True
        Exit For
    End If
Next cc
If Not found Then
    Set co = wsA.ChartObjects.Add( _
        Left:=wsA.Range(anchorCell).Left, _
        Top:=wsA.Range(anchorCell).Top, _
        Width:=460, Height:=260)
    co.Name = chartName
End If

With co.Chart
    .ChartType = xlXYScatterLines
    Do While .SeriesCollection.count > 0
        .SeriesCollection(1).Delete
    Loop
    Dim sr As Series: Set sr = .SeriesCollection.NewSeries
    sr.XValues = xRange
    sr.Values = yRange
    sr.Name = chartName

    .HasTitle = True
    .ChartTitle.Text = title
    .HasLegend = False
    On Error Resume Next
    .Axes(xlCategory).HasTitle = True
    .Axes(xlCategory).AxisTitle.Text = xLabel
    .Axes(xlValue).HasTitle = True
    .Axes(xlValue).AxisTitle.Text = yLabel
    On Error GoTo 0
End With
End Sub
' =====
' END F4
' =====

Private Function DeriveTrend(typStr As String, undlPx As Variant, _
    strike As Variant, delta As Variant) As String
    If Not IsNumeric(undlPx) Or Not IsNumeric(strike) Then
        DeriveTrend = "-"
        Exit Function
    End If
    Dim t As String: t = LCase(typStr)
    Dim isCall As Boolean
    isCall = (InStr(t, "call") > 0 Or InStr(t, "bull") > 0)
    Dim u As Double: u = Cdbl(undlPx)
    Dim k As Double: k = Cdbl(strike)
    Dim m As String

    If isCall Then
        If u > k * 1.02 Then
            m = "ITM"
        ElseIf u < k * 0.98 Then
            m = "OTM"
        Else
            m = "ATM"
        End If
    Else
        If u < k * 0.98 Then
            m = "ITM"
        ElseIf u > k * 1.02 Then
            m = "OTM"
        Else
            m = "ATM"
        End If
    End If
End Function

```

```
End If
End If
```

```
DeriveTrend = m & " " & IIf(isCall, "Bull", "Bear")
```

```
End Function
```

```
Private Function DeriveNote(dte As Variant, delta As Variant, chgPct As Variant) As String
```

```
Dim notes As String: notes = ""
```

```
If IsNumeric(dte) Then
```

```
    If CLng(dte) <= 7 And CLng(dte) > 0 Then
```

```
        notes = notes & "EXP<=7d; "
```

```
    ElseIf CLng(dte) <= 30 Then
```

```
        notes = notes & "<=30d; "
```

```
    End If
```

```
End If
```

```
If IsNumeric(delta) Then
```

```
    Dim d As Double: d = Abs(CDbl(delta))
```

```
    If d > 0.8 Then
```

```
        notes = notes & "Deep ITM; "
```

```
    ElseIf d < 0.15 And d > 0 Then
```

```
        notes = notes & "Far OTM; "
```

```
    End If
```

```
End If
```

```
If IsNumeric(chgPct) Then
```

```
    Dim c As Double: c = CDbl(chgPct)
```

```
    If c > 0.5 Then
```

```
        notes = notes & "+" & Format(c, "0%") & "; "
```

```
    ElseIf c < -0.3 Then
```

```
        notes = notes & Format(c, "0%") & "; "
```

```
    End If
```

```
End If
```

```
If Right(notes, 2) = "; " Then notes = Left(notes, Len(notes) - 2)
```

```
DeriveNote = notes
```

```
End Function
```

```

' =====
' Label writing (idempotent)
' =====

```

```
Private Sub EnsureDashboardLabels(wsR As Worksheet)
```

```
Dim row3Lbl As Variant
```

```
row3Lbl = Array( _
```

```
    Array("A3", "IV RANK"), Array("C3", "IV PCTL"), _
```

```
    Array("E3", "THETA CUM"), Array("G3", "DELTA D7"), _
```

```
    Array("I3", "EXP <=7D"), Array("K3", "AVG DTE"), _
```

```
    Array("M3", "TRADES"), Array("O3", "30D TRD"), _
```

```
    Array("Q3", "WIN%"), Array("S3", "AVG WIN"), _
```

```
    Array("U3", "AVG LOSS"), Array("W3", "PF"))
```

```
Dim row4Lbl As Variant
```

```
row4Lbl = Array( _
```

```
    Array("A4", "MV 7D"), Array("C4", "MV 30D"), _
```

```
    Array("E4", "MV CHG7%"), Array("G4", "MV CHG30%"), _
```

```
    Array("I4", "MDD%"), Array("K4", "MEAN DD%"), _
```

```
    Array("M4", "BEST D%"), Array("O4", "WORST D%"), _
```

```
    Array("Q4", "DAYS LOG"), Array("S4", "IV 30D"))
```

```
Dim i As Long
```

```
For i = LBound(row3Lbl) To UBound(row3Lbl)
```

```
    With wsR.Range(row3Lbl(i)(0))
```

```
        .Value = row3Lbl(i)(1)
```

```
        .Font.Bold = True
```

```
        .Font.Color = RGB(150, 150, 150)
```

```
        .Font.Size = 9
```

```
        .Font.Name = "Consolas"
```

```
    End With
```

```
Next i
```

```
For i = LBound(row4Lbl) To UBound(row4Lbl)
```

```
    With wsR.Range(row4Lbl(i)(0))
```

```
        .Value = row4Lbl(i)(1)
```

```
        .Font.Bold = True
```

```
        .Font.Color = RGB(150, 150, 150)
```

```

        .Font.Size = 9
        .Font.Name = "Consolas"
    End With
Next i
End Sub

' =====
' Realized aggregates: Trades / Win% / Avg Win / Avg Loss / Profit Factor
' Realized cols: A=TICKER B=TYPE C=STRATEGY D=PNL E=SHARES F=AVG COST
'               G=NET AMT H=STRATEGY TAG I=PNL(TWD) J=DATE
' =====

Private Sub WriteRealizedMetrics(wsR As Worksheet)
    Dim ws As Worksheet
    On Error Resume Next
    Set ws = ThisWorkbook.Sheets("Realized")
    On Error GoTo 0
    If ws Is Nothing Then Exit Sub

    Dim lr As Long
    lr = ws.Cells(ws.Rows.count, "A").End(xlUp).row
    If lr < 2 Then GoTo WriteOut

    Dim totalTrades As Long, wins As Long, losses As Long
    Dim sumWin As Double, sumLoss As Double, totalPnL As Double
    Dim trades30D As Long
    Dim cutDate As Date: cutDate = Date - 30

    Dim i As Long
    For i = 2 To lr
        Dim pnl As Variant: pnl = ws.Cells(i, 9).Value    ' I = PNL(TWD)
        Dim dt As Variant: dt = ws.Cells(i, 10).Value    ' J = DATE
        If IsNumeric(pnl) Then
            totalTrades = totalTrades + 1
            totalPnL = totalPnL + CDbl(pnl)
            If CDbl(pnl) > 0 Then
                wins = wins + 1
                sumWin = sumWin + CDbl(pnl)
            ElseIf CDbl(pnl) < 0 Then
                losses = losses + 1
                sumLoss = sumLoss + CDbl(pnl)
            End If
            If IsDate(dt) Then
                If CDate(dt) >= cutDate Then trades30D = trades30D + 1
            End If
        End If
    Next i

WriteOut:
    With wsR
        .Range("N3").Value = totalTrades
        .Range("P3").Value = trades30D
        If totalTrades > 0 Then
            .Range("R3").Value = wins / totalTrades
            .Range("R3").NumberFormat = "0%"
        Else
            .Range("R3").Value = 0
        End If
        If wins > 0 Then
            .Range("T3").Value = sumWin / wins
            .Range("T3").NumberFormat = "#,##0"
        End If
        If losses > 0 Then
            .Range("V3").Value = sumLoss / losses
            .Range("V3").NumberFormat = "#,##0;(#,##0)"
        End If
        If sumLoss <> 0 Then
            .Range("X3").Value = Abs(sumWin / sumLoss)
            .Range("X3").NumberFormat = "0.00"
        End If
    End With
End Sub

```

```

' =====
' IVTracker aggregates: latest IV Rank / Percentile per underlying, avg
'   IVTracker cols: A=Date B=Underlying C=IV Current D=IV HV30
'                   E=IV Percentile F=IV Rank
' =====

Private Sub WriteIVMetrics(wsR As Worksheet)
    Dim ws As Worksheet
    On Error Resume Next
    Set ws = ThisWorkbook.Sheets("IVTracker")
    On Error GoTo 0
    If ws Is Nothing Then Exit Sub

    Dim lr As Long
    lr = ws.Cells(ws.Rows.count, "A").End(xlUp).row
    If lr < 2 Then Exit Sub

    Dim dict As Object
    Set dict = CreateObject("Scripting.Dictionary")

    Dim entry(3) As Variant
    Dim i As Long
    For i = 2 To lr
        Dim dt As Variant: dt = ws.Cells(i, 1).Value
        Dim und As String: und = Trim(CStr(ws.Cells(i, 2).Value))
        If und = "" Or Not IsDate(dt) Then GoTo NextI

        Dim shouldUpdate As Boolean: shouldUpdate = True
        If dict.Exists(und) Then
            Dim existing As Variant: existing = dict(und)
            If CDate(existing(0)) >= CDate(dt) Then shouldUpdate = False
        End If

        If shouldUpdate Then
            entry(0) = dt
            entry(1) = ws.Cells(i, 3).Value    ' IV Current
            entry(2) = ws.Cells(i, 5).Value    ' IV Percentile
            entry(3) = ws.Cells(i, 6).Value    ' IV Rank
            dict(und) = entry
        End If
    NextI:
    Next i

    Dim sumPctl As Double, sumRank As Double, cnt As Long
    Dim key As Variant
    For Each key In dict.keys
        Dim ent As Variant: ent = dict(key)
        If IsNumeric(ent(2)) Then sumPctl = sumPctl + Cdbl(ent(2))
        If IsNumeric(ent(3)) Then sumRank = sumRank + Cdbl(ent(3))
        cnt = cnt + 1
    Next key

    With wsR
        If cnt > 0 Then
            .Range("B3").Value = sumRank / cnt
            .Range("B3").NumberFormat = "0.0"
            .Range("D3").Value = sumPctl / cnt
            .Range("D3").NumberFormat = "0.0"
        End If
    End With
End Sub

' =====
' HistoryLog metrics: MV time-series, MDD, best/worst day
'   HistoryLog cols: A=Date B=TotalMV C=CumPnL D=G=NetGreeks
'                   H=Px(TX) I=Ret%(TX) J=Px(SPY) K=Ret%(SPY)
'                   L=RealizedPnL M=PortRet% N=PortRetChg% O=IV_Avg
' =====

Private Sub WriteHistoryLogMetrics(wsR As Worksheet)
    Dim ws As Worksheet
    On Error Resume Next

```

```
Set wS = ThisWorkbook.Sheets("HistoryLog")
```

```
On Error GoTo 0
```

```
If wS Is Nothing Then Exit Sub
```

```
Dim lr As Long
```

```
lr = wS.Cells(wS.Rows.count, "A").End(xlUp).row
```

```
If lr < 2 Then Exit Sub
```

```
Dim today As Date: today = Date
```

```
Dim cut7 As Date: cut7 = today - 7
```

```
Dim cut30 As Date: cut30 = today - 30
```

```
Dim curMV As Double, mv7d As Double, mv30d As Double
```

```
Dim foundLatest As Boolean, found7 As Boolean, found30 As Boolean
```

```
Dim i As Long
```

```
For i = 1r To 2 Step -1
```

```
Dim dt As Variant: dt = wS.Cells(i, 1).Value
```

```
Dim mv As Variant: mv = wS.Cells(i, 2).Value
```

```
If Not IsDate(dt) Then GoTo NextRev
```

```
If Not IsNumeric(mv) Then GoTo NextRev
```

```
If Not foundLatest Then
```

```
curMV = CDBl(mv): foundLatest = True
```

```
End If
```

```
If Not found7 And CDate(dt) <= cut7 Then
```

```
mv7d = CDBl(mv): found7 = True
```

```
End If
```

```
If Not found30 And CDate(dt) <= cut30 Then
```

```
mv30d = CDBl(mv): found30 = True
```

```
End If
```

```
If foundLatest And found7 And found30 Then Exit For
```

```
NextRev:
```

```
Next i
```

```
' MDD / mean DD / best/worst day from PortRetChg% (col N)
```

```
Dim peak As Double, dd As Double, maxDD As Double
```

```
Dim sumDD As Double, ddCnt As Long
```

```
Dim bestDay As Double, worstDay As Double
```

```
Dim daysCnt As Long
```

```
Dim ivSum As Double, ivCnt As Long
```

```
bestDay = -1: worstDay = 1: peak = 0
```

```
For i = 2 To 1r
```

```
Dim dti As Variant: dti = wS.Cells(i, 1).Value
```

```
Dim mvi As Variant: mvi = wS.Cells(i, 2).Value
```

```
Dim chg As Variant: chg = wS.Cells(i, 14).Value
```

```
Dim ivi As Variant: ivi = wS.Cells(i, 15).Value
```

```
If IsDate(dti) Then daysCnt = daysCnt + 1
```

```
If IsNumeric(mvi) Then
```

```
If CDBl(mvi) > peak Then peak = CDBl(mvi)
```

```
If peak > 0 Then
```

```
dd = (CDBl(mvi) - peak) / peak
```

```
If dd < maxDD Then maxDD = dd
```

```
If dd < 0 Then
```

```
sumDD = sumDD + dd
```

```
ddCnt = ddCnt + 1
```

```
End If
```

```
End If
```

```
End If
```

```
If IsNumeric(chg) Then
```

```
If CDBl(chg) > bestDay Then bestDay = CDBl(chg)
```

```
If CDBl(chg) < worstDay Then worstDay = CDBl(chg)
```

```
End If
```

```
If IsDate(dti) And IsNumeric(ivi) Then
```

```
If CDate(dti) >= cut30 Then
```

```
ivSum = ivSum + CDBl(ivi)
```

```
ivCnt = ivCnt + 1
```



```

        End If
    End If
Next i

With wsR
    If mv7d <> 0 Then
        .Range("B4").Value = mv7d
        .Range("B4").NumberFormat = "#,##0"
        If mv7d > 0 And foundLatest Then
            .Range("F4").Value = (curMV - mv7d) / mv7d
            .Range("F4").NumberFormat = "0.00%"
        End If
    End If
    If mv30d <> 0 Then
        .Range("D4").Value = mv30d
        .Range("D4").NumberFormat = "#,##0"
        If mv30d > 0 And foundLatest Then
            .Range("H4").Value = (curMV - mv30d) / mv30d
            .Range("H4").NumberFormat = "0.00%"
        End If
    End If
    .Range("J4").Value = maxDD
    .Range("J4").NumberFormat = "0.00%"
    If ddCnt > 0 Then
        .Range("L4").Value = sumDD / ddCnt
        .Range("L4").NumberFormat = "0.00%"
    End If
    If bestDay > -1 Then
        .Range("N4").Value = bestDay
        .Range("N4").NumberFormat = "0.00%"
    End If
    If worstDay < 1 Then
        .Range("P4").Value = worstDay
        .Range("P4").NumberFormat = "0.00%"
    End If
    .Range("R4").Value = daysCnt
    If ivCnt > 0 Then
        .Range("T4").Value = ivSum / ivCnt
        .Range("T4").NumberFormat = "0.0%"
    End If
End With
End Sub

' =====
' Greeks_History aggregates: cumulative Theta_PnL_Est, 7D Delta drift
' Greeks_History cols: A=Date B=Spot_TX C=Port_Delta D=Port_Gamma
'                      E=Port_Theta F=Port_Vega G=IV_Avg
'                      H=Delta_PnL_Est I=Gamma_PnL_Est J=Theta_PnL_Est
'                      K=Vega_PnL_Est L=Residual_PnL
' =====

Private Sub WriteGreeksHistoryMetrics(wsR As Worksheet)
    Dim ws As Worksheet
    On Error Resume Next
    Set ws = ThisWorkbook.Sheets("Greeks_History")
    On Error GoTo 0
    If ws Is Nothing Then Exit Sub

    Dim lr As Long
    lr = ws.Cells(ws.Rows.count, "A").End(xlUp).row
    If lr < 2 Then Exit Sub

    Dim thetaCum As Double
    Dim curDelta As Double, delta7d As Double
    Dim foundLatest As Boolean, found7 As Boolean
    Dim cut7 As Date: cut7 = Date - 7

    Dim i As Long
    For i = 2 To lr
        Dim t As Variant: t = ws.Cells(i, 10).Value ' J = Theta_PnL_Est
        If IsNumeric(t) Then thetaCum = thetaCum + CDbl(t)
    Next i

```

```

For i = 1r To 2 Step -1
    Dim dt As Variant: dt = ws.Cells(i, 1).Value
    Dim d As Variant: d = ws.Cells(i, 3).Value
    If IsDate(dt) And IsNumeric(d) Then
        If Not foundLatest Then
            curDelta = CDbl(d): foundLatest = True
        End If
        If Not found7 And CDate(dt) <= cut7 Then
            delta7d = CDbl(d): found7 = True
        End If
        If foundLatest And found7 Then Exit For
    End If
Next i

With wsR
    .Range("F3").Value = thetaCum
    .Range("F3").NumberFormat = "#,##0;(#,##0)"
    If foundLatest And found7 Then
        .Range("H3").Value = curDelta - delta7d
        .Range("H3").NumberFormat = "+0.00;-0.00;0.00"
    End If
End With
End Sub

' =====
' Position metrics from RR5 itself: expiring count, avg DTE
' =====

Private Sub WritePositionMetrics(wsR As Worksheet)
    Dim cntExp As Long, sumDTE As Double, cntDTE As Long

    ' Compact layout: Futures 7-14 / Options 19-26 / Warrants 31-38
    Dim secs As Variant
    secs = Array(Array(7, 14), Array(19, 26), Array(31, 38))

    Dim r As Variant
    For Each r In secs
        Dim i As Long
        For i = CLng(r(0)) To CLng(r(1))
            Dim dte As Variant: dte = wsR.Cells(i, 5).Value
            If IsNumeric(dte) Then
                If CLng(dte) > 0 Then
                    sumDTE = sumDTE + CLng(dte)
                    cntDTE = cntDTE + 1
                    If CLng(dte) <= 7 Then cntExp = cntExp + 1
                End If
            End If
        Next i
    Next r

    With wsR
        .Range("J3").Value = cntExp
        If cntDTE > 0 Then
            .Range("L3").Value = sumDTE / cntDTE
            .Range("L3").NumberFormat = "0"
        End If
    End With
End Sub

```

Option Explicit

```

=====
' STRATEGY BUILDER -- Multi-Leg Payoff Calculator (max 6 legs)
'-----
' Worksheet: StrategyBuilder
'
' Layout (cols A..K):
'   Row 1   Title
'   Row 3   A "Underlying Symbol:" | B (input symbol e.g. TWII)
'           C "Underlying Price (ref):" | D (input spot)
'           E "Risk-Free Rate:" | F (input, e.g. 0.02)
'           G "Multiplier (default):" | H (input, e.g. 50)
'   Row 4   A "DTE (days):" | B (input, e.g. 30)
'           C "IV (default):" | D (input, e.g. 0.25)
'   Row 5   Note
'   Row 7   Header: Leg# / TYPE / STRIKE / CONTRACTS / MULTIPLIER /
'             DIRECTION / PREMIUM / DELTA / GAMMA / THETA / VEGA
'   Row 8-13 Leg rows (6 legs)
'   Row 15  "STRATEGY AGGREGATES"
'   Row 16  Net Delta
'   Row 17  Net Gamma
'   Row 18  Net Theta/Day
'   Row 19  Net Vega
'   Row 20  Net Premium Paid      (positive = debit, negative = credit)
'   Row 21  Max Profit (est., at expiry)
'   Row 22  Max Loss (est., at expiry)
'   Row 23  Breakeven (est., at expiry)
'
' Direction sign:
'   LONG  -> +1 (paid premium, you bought the leg)
'   SHORT -> -1 (received premium, you sold the leg)
'
' Aggregation formula:
'   net_Greek = sum( dirSign * Contracts * Multiplier * leg_Greek )
'   net_Premium = sum( dirSign * Contracts * Multiplier * Premium )
'               positive = net debit (you paid)
'               negative = net credit (you received)
'
' Payoff at expiry (per leg):
'   intrinsic = max(0, S - K) for Call, max(0, K - S) for Put
'   leg_PnL   = dirSign * Contracts * Multiplier * (intrinsic - Premium)
'
' Max Profit / Max Loss / Breakeven detected by scanning S over
' [minStrike * 0.7, maxStrike * 1.3] in 400 steps.
' Unlimited cases flagged when payoff trends past scan boundary.
'=====
'-----
' One-shot layout setup. Run after first install or whenever you want
' to reset labels / defaults. Doesn't touch user inputs that already exist.
'-----
Public Sub Setup_StrategyBuilder()
    On Error GoTo Fail
    Dim wsS As Worksheet
    Set wsS = ThisWorkbook.Sheets("StrategyBuilder")

    ' --- Row 3 header labels ---
    wsS.Range("A3").Value = "Underlying Symbol:"
    wsS.Range("C3").Value = "Underlying Price (ref):"
    wsS.Range("E3").Value = "Risk-Free Rate:"
    If Not IsNumeric(wsS.Range("F3").Value) Then wsS.Range("F3").Value = 0.02
    wsS.Range("G3").Value = "Multiplier (default):"
    If Not IsNumeric(wsS.Range("H3").Value) Then wsS.Range("H3").Value = 50

    ' --- Row 4 header labels (NEW) ---
    wsS.Range("A4").Value = "DTE (days):"
    wsS.Range("C4").Value = "IV (default):"

    ' --- Row 5 note ---
    wsS.Range("A5").Value = _

```

"NOTE: Greeks (H/I/J/K) auto-filled by RefreshStrategyBuilder. " & _
 "Direction = Long or Short. Premium per single share/index point."

' --- Row 7 leg table header ---

Dim hdr As Variant

hdr = Array("Leg #", "TYPE", "STRIKE", "CONTRACTS", "MULTIPLIER", _
 "DIRECTION", "PREMIUM", "DELTA", "GAMMA", "THETA", "VEGA")

Dim c As Long

For c = 0 To UBound(hdr)

wsS.Cells(7, c + 1).Value = hdr(c)

Next c

' --- Pre-fill leg #s and default multiplier ---

Dim row As Long

For row = 8 To 13

wsS.Cells(row, 1).Value = row - 7

If Not IsNumeric(wsS.Cells(row, 5).Value) Or wsS.Cells(row, 5).Value = 0 Then

wsS.Cells(row, 5).Value = wsS.Range("H3").Value

End If

Next row

' --- Row 15-23 aggregate labels ---

wsS.Range("A15").Value = "STRATEGY AGGREGATES"

wsS.Range("A16").Value = "Net Delta"

wsS.Range("A17").Value = "Net Gamma"

wsS.Range("A18").Value = "Net Theta/Day"

wsS.Range("A19").Value = "Net Vega"

wsS.Range("A20").Value = "Net Premium Paid"

wsS.Range("A21").Value = "Max Profit (est.)"

wsS.Range("A22").Value = "Max Loss (est.)"

wsS.Range("A23").Value = "Breakeven (est.)"

' Clear stale results

wsS.Range("B16:B23").ClearContents

MsgBox "StrategyBuilder layout ready." & vbCrLf & vbCrLf & _

"Fill in inputs:" & vbCrLf & _

" D3 = Spot price F3 = Risk-free rate H3 = Default multiplier" & vbCrLf & _

" B4 = DTE (days) D4 = IV (e.g. 0.25)" & vbCrLf & _

" Rows 8-13: TYPE (Call/Put), STRIKE, CONTRACTS, DIRECTION (Long/Short), PREMIUM" & vbCrLf & vbCrLf & _

"Then Alt+F8 -> RefreshStrategyBuilder.", _

vbInformation, "StrategyBuilder Setup"

Exit Sub

Fail:

MsgBox "Setup_StrategyBuilder failed: " & Err.Description, vbCritical

End Sub

' -----
 ' Main recalculation routine.
 ' Reads inputs, fills per-leg Greeks (H/I/J/K), writes aggregates (B16-B23).
 ' -----

Public Sub RefreshStrategyBuilder()

On Error GoTo Fail

Dim wsS As Worksheet

Set wsS = ThisWorkbook.Sheets("StrategyBuilder")

' --- Header inputs ---

Dim s As Double, rRate As Double, multDef As Double

Dim dteDays As Double, ivDef As Double, t As Double

s = ToDbl(wsS.Range("D3").Value)

rRate = ToDbl(wsS.Range("F3").Value)

multDef = ToDbl(wsS.Range("H3").Value)

dteDays = ToDbl(wsS.Range("B4").Value)

ivDef = ToDbl(wsS.Range("D4").Value)

If s <= 0 Or dteDays <= 0 Or ivDef <= 0 Then

MsgBox "Missing required inputs:" & vbCrLf & _

" D3 Spot = " & s & vbCrLf & _

" B4 DTE = " & dteDays & vbCrLf & _

" D4 IV = " & ivDef, _

```

        vbExclamation, "StrategyBuilder"
    Exit Sub
End If

t = dteDays / 365#
If multDef = 0 Then multDef = 50

' --- Walk 6 legs ---
Dim netDelta As Double, netGamma As Double, netTheta As Double
Dim netVega As Double, netPrem As Double
Dim n As Long: n = 0

' Parallel arrays for payoff scan
Dim lgType(1 To 6) As String
Dim lgStrike(1 To 6) As Double
Dim lgQty(1 To 6) As Double      ' signed weight = dirSign * contracts * mult
Dim lgPrem(1 To 6) As Double
Dim lgMinK As Double: lgMinK = 1E+15
Dim lgMaxK As Double: lgMaxK = 0

' Clear stale Greeks
wsS.Range("H8:K13").ClearContents

Dim row As Long
For row = 8 To 13
    Dim tStr As String, kStr As Double, qty As Double, mult As Double
    Dim dirStr As String, prem As Double, dirSign As Long

    tStr = UCase(Trim(CStr(wsS.Cells(row, 2).Value)))
    If tStr = "" Then GoTo NextLeg
    If tStr <> "CALL" And tStr <> "PUT" Then GoTo NextLeg

    kStr = ToDbL(wsS.Cells(row, 3).Value)
    qty = ToDbL(wsS.Cells(row, 4).Value)
    mult = ToDbL(wsS.Cells(row, 5).Value)
    If mult = 0 Then mult = multDef
    dirStr = UCase(Trim(CStr(wsS.Cells(row, 6).Value)))
    prem = ToDbL(wsS.Cells(row, 7).Value)

    If kStr <= 0 Or qty <= 0 Then GoTo NextLeg

    dirSign = 0
    If dirStr = "LONG" Then dirSign = 1
    If dirStr = "SHORT" Then dirSign = -1
    If dirSign = 0 Then GoTo NextLeg      ' direction required

    Dim oType As String
    If tStr = "CALL" Then oType = "Call" Else oType = "Put"

    ' --- BS Greeks per single contract ---
    Dim legDelta As Double, legGamma As Double, legTheta As Double, legVega As Double
    legDelta = BS_Delta(s, kStr, t, rRate, ivDef, oType)
    legGamma = BS_Gamma(s, kStr, t, rRate, ivDef)
    legTheta = BS_Theta(s, kStr, t, rRate, ivDef, oType)
    legVega = BS_Vega(s, kStr, t, rRate, ivDef)

    wsS.Cells(row, 8).Value = legDelta
    wsS.Cells(row, 9).Value = legGamma
    wsS.Cells(row, 10).Value = legTheta
    wsS.Cells(row, 11).Value = legVega
    wsS.Cells(row, 8).NumberFormat = "0.0000"
    wsS.Cells(row, 9).NumberFormat = "0.000000"
    wsS.Cells(row, 10).NumberFormat = "0.0000"
    wsS.Cells(row, 11).NumberFormat = "0.0000"

    ' --- Aggregate (signed ? qty ? multiplier ? per-share Greek) ---
    Dim w As Double: w = dirSign * qty * mult
    netDelta = netDelta + w * legDelta
    netGamma = netGamma + w * legGamma
    netTheta = netTheta + w * legTheta
    netVega = netVega + w * legVega

```

```

netPrem = netPrem + w * prem

' --- Cache for payoff scan ---
n = n + 1
lgType(n) = oType
lgStrike(n) = kStr
lgQty(n) = w
lgPrem(n) = prem
If kStr < lgMinK Then lgMinK = kStr
If kStr > lgMaxK Then lgMaxK = kStr
NextLeg:
Next row

' --- Write aggregates ---
wsS.Range("B16").Value = netDelta: wsS.Range("B16").NumberFormat = "#,##0.0000"
wsS.Range("B17").Value = netGamma: wsS.Range("B17").NumberFormat = "#,##0.000000"
wsS.Range("B18").Value = netTheta: wsS.Range("B18").NumberFormat = "#,##0.00"
wsS.Range("B19").Value = netVega: wsS.Range("B19").NumberFormat = "#,##0.00"
wsS.Range("B20").Value = netPrem: wsS.Range("B20").NumberFormat = "#,##0;(#,##0)"

' --- Payoff scan ---
If n = 0 Then
    wsS.Range("B21:B23").ClearContents
    Application.StatusBar = "StrategyBuilder: no valid legs"
    Exit Sub
End If

Dim scanLo As Double, scanHi As Double
scanLo = lgMinK * 0.7
scanHi = lgMaxK * 1.3
If scanLo < 0 Then scanLo = 0
Dim stepSize As Double: stepSize = (scanHi - scanLo) / 400
If stepSize <= 0 Then stepSize = 1

Dim maxP As Double: maxP = -1E+15
Dim minP As Double: minP = 1E+15
Dim maxAtBoundary As Boolean, minAtBoundary As Boolean
Dim breakevenList As String: breakevenList = ""

Dim prevPayoff As Double, prevPx As Double
Dim firstPx As Boolean: firstPx = True

Dim px As Double: px = scanLo
Do While px <= scanHi
    Dim payoff As Double
    payoff = PayoffAtExpiry(px, lgType, lgStrike, lgQty, lgPrem, n)

    If payoff > maxP Then
        maxP = payoff
        maxAtBoundary = (px <= scanLo + stepSize) Or (px >= scanHi - stepSize)
    End If
    If payoff < minP Then
        minP = payoff
        minAtBoundary = (px <= scanLo + stepSize) Or (px >= scanHi - stepSize)
    End If

    If Not firstPx Then
        If (prevPayoff < 0 And payoff >= 0) Or (prevPayoff > 0 And payoff <= 0) Then
            If payoff - prevPayoff <> 0 Then
                Dim be As Double
                be = prevPx - prevPayoff * (px - prevPx) / (payoff - prevPayoff)
                If breakevenList = "" Then
                    breakevenList = Format(be, "#,##0.00")
                Else
                    breakevenList = breakevenList & " / " & Format(be, "#,##0.00")
                End If
            End If
        End If
    End If

    prevPayoff = payoff
    prevPx = px
    px = px + stepSize
Loop

```

```

    prevPx = px
    firstPx = False
    px = px + stepSize
Loop

' --- Unlimited detection (gradient at boundaries) ---
Dim payLo As Double, payLoPlus As Double
Dim payHi As Double, payHiMinus As Double
payLo = PayoffAtExpiry(scanLo, lgType, lgStrike, lgQty, lgPrem, n)
payLoPlus = PayoffAtExpiry(scanLo + stepSize, lgType, lgStrike, lgQty, lgPrem, n)
payHiMinus = PayoffAtExpiry(scanHi - stepSize, lgType, lgStrike, lgQty, lgPrem, n)
payHi = PayoffAtExpiry(scanHi, lgType, lgStrike, lgQty, lgPrem, n)

Dim gradHi As Double: gradHi = payHi - payHiMinus
Dim gradLo As Double: gradLo = payLoPlus - payLo

Dim maxStr As String, minStr As String
If maxAtBoundary And (gradHi > 0.01 Or gradLo < -0.01) Then
    maxStr = "UNLIMITED (scan caps ~" & Format(maxP, "#,##0") & ")"
Else
    maxStr = Format(maxP, "#,##0")
End If

If minAtBoundary And (gradHi < -0.01 Or gradLo > 0.01) Then
    minStr = "UNLIMITED LOSS (scan caps ~" & Format(minP, "#,##0") & ")"
Else
    minStr = Format(minP, "#,##0")
End If

wsS.Range("B21").Value = maxStr
wsS.Range("B22").Value = minStr
wsS.Range("B23").Value = IIf(breakevenList = "", "None in scan range", breakevenList)

' F2: redraw payoff chart with the same legs
Call BuildPayoffChart

Application.StatusBar = "StrategyBuilder refreshed " & Format(Now, "hh:mm:ss") & _
    " | Legs=" & n & " | Scan " & Format(scanLo, "0") & "~" & Format(scanHi, "0")
Exit Sub

Fail:
Application.StatusBar = False
MsgBox "RefreshStrategyBuilder failed: " & Err.Description, vbExclamation
End Sub

' =====
' F2: PAYOFF CHART
' -----
' Builds a dynamic P&L-at-expiry curve in the StrategyBuilder sheet.
' Writes data table at A30:B82, then creates/updates an XY scatter chart
' anchored at cell M2 (~col M, top row).
'
' Stage-by-stage error reporting -- on failure MsgBox shows which stage.
' =====

Public Sub BuildPayoffChart()
    Dim stage As String: stage = "init"
    On Error GoTo Fail

    stage = "open sheet"
    Dim wsS As Worksheet: Set wsS = ThisWorkbook.Sheets("StrategyBuilder")

    stage = "read defaults"
    Dim multDef As Double
    multDef = ToDb1(wsS.Range("H3").Value)
    If multDef = 0 Then multDef = 50

    stage = "collect legs"
    Dim nL As Long: nL = 0
    Dim lgType(1 To 6) As String
    Dim lgStrike(1 To 6) As Double
    Dim lgQty(1 To 6) As Double
    Dim lgPrem(1 To 6) As Double

```

```
Dim minK As Double: minK = 1E+15
```

```
Dim maxK As Double: maxK = 0
```

```
Dim r As Long
```

```
For r = 8 To 13
```

```
    stage = "leg row " & r
```

```
    Dim tStr As String: tStr = UCase(Trim(CStr(wsS.Cells(r, 2).Value)))
```

```
    If tStr <> "CALL" And tStr <> "PUT" Then GoTo NextLeg
```

```
    Dim k As Double: k = ToDbl(wsS.Cells(r, 3).Value)
```

```
    Dim qq As Double: qq = ToDbl(wsS.Cells(r, 4).Value)
```

```
    Dim mm As Double: mm = ToDbl(wsS.Cells(r, 5).Value)
```

```
    If mm = 0 Then mm = multDef
```

```
    Dim d As String: d = UCase(Trim(CStr(wsS.Cells(r, 6).Value)))
```

```
    Dim pr As Double: pr = ToDbl(wsS.Cells(r, 7).Value)
```

```
    If k <= 0 Or qq <= 0 Then GoTo NextLeg
```

```
    ' dirSign instead of sgn to avoid clash with VBA built-in Sgn()
```

```
    Dim dirSign As Long
```

```
    If d = "LONG" Then
```

```
        dirSign = 1
```

```
    ElseIf d = "SHORT" Then
```

```
        dirSign = -1
```

```
    Else
```

```
        GoTo NextLeg
```

```
    End If
```

```
    nL = nL + 1
```

```
    If tStr = "CALL" Then lgType(nL) = "Call" Else lgType(nL) = "Put"
```

```
    lgStrike(nL) = k
```

```
    lgQty(nL) = dirSign * qq * mm
```

```
    lgPrem(nL) = pr
```

```
    If k < minK Then minK = k
```

```
    If k > maxK Then maxK = k
```

```
NextLeg:
```

```
Next r
```

```
stage = "validate legs"
```

```
wsS.Range("A30:B82").ClearContents
```

```
wsS.Cells(30, 1).Value = "PAYOFF DATA"
```

```
If nL = 0 Then
```

```
    ' Remove existing chart if no legs
```

```
    On Error Resume Next
```

```
    wsS.ChartObjects("PayoffChart").Delete
```

```
    On Error GoTo Fail
```

```
    Exit Sub
```

```
End If
```

```
stage = "compute scan range"
```

```
Dim loSpot As Double, hiSpot As Double, stepSz As Double
```

```
loSpot = minK * 0.7
```

```
hiSpot = maxK * 1.3
```

```
If loSpot < 0 Then loSpot = 0
```

```
If hiSpot <= loSpot Then hiSpot = loSpot + 100
```

```
stepSz = (hiSpot - loSpot) / 50
```

```
If stepSz <= 0 Then stepSz = 1
```

```
stage = "write payoff data"
```

```
wsS.Cells(31, 1).Value = "Spot"
```

```
wsS.Cells(31, 2).Value = "PnL"
```

```
Dim sp As Double: sp = loSpot
```

```
Dim outR As Long: outR = 32
```

```
Do While sp <= hiSpot + 0.0001
```

```
    Dim payoff As Double: payoff = 0
```

```
    Dim i As Long
```

```
    For i = 1 To nL
```

```
        Dim intr As Double
```

```
        If lgType(i) = "Call" Then
```

```
            intr = sp - lgStrike(i)
```

```
            If intr < 0 Then intr = 0
```

```
        Else
```

```
            intr = lgStrike(i) - sp
```

```
            If intr < 0 Then intr = 0
```

```
        End If
```



```

        payoff = payoff + lgQty(i) * (intr - lgPrem(i))
    Next i
    wsS.Cells(outR, 1).Value = sp
    wsS.Cells(outR, 2).Value = payoff
    outR = outR + 1
    sp = sp + stepSz
Loop
Dim lastR As Long: lastR = outR - 1
If lastR < 33 Then Exit Sub

stage = "find/create chart"
Dim co As ChartObject
Dim foundChart As Boolean: foundChart = False
Dim cc As ChartObject
For Each cc In wsS.ChartObjects
    If cc.Name = "PayoffChart" Then
        Set co = cc
        foundChart = True
        Exit For
    End If
Next cc
If Not foundChart Then
    Dim anchorL As Double, anchorT As Double
    anchorL = wsS.Range("M2").Left
    anchorT = wsS.Range("M2").Top
    Set co = wsS.ChartObjects.Add(Left:=anchorL, Top:=anchorT, _
        Width:=460, Height:=300)
    co.Name = "PayoffChart"
End If

stage = "configure chart"
With co.Chart
    .ChartType = xlXYScatterLines
    ' Clear existing series
    Do While .SeriesCollection.count > 0
        .SeriesCollection(1).Delete
    Loop
    ' Add PnL series
    Dim sr As Series: Set sr = .SeriesCollection.NewSeries
    sr.XValues = wsS.Range(wsS.Cells(32, 1), wsS.Cells(lastR, 1))
    sr.Values = wsS.Range(wsS.Cells(32, 2), wsS.Cells(lastR, 2))
    sr.Name = "P&L at Expiry"
    ' Zero line
    Dim sr0 As Series: Set sr0 = .SeriesCollection.NewSeries
    sr0.XValues = Array(loSpot, hiSpot)
    sr0.Values = Array(0, 0)
    sr0.Name = "Zero"
    On Error Resume Next
    sr0.Format.Line.ForeColor.RGB = RGB(150, 150, 150)
    sr0.Format.Line.DashStyle = msoLineDash
    On Error GoTo Fail

    .HasTitle = True
    .ChartTitle.Text = "Payoff at Expiry (Spot " & _
        Format(loSpot, "0") & " ~ " & Format(hiSpot, "0") & ")"
    .HasLegend = False
    On Error Resume Next
    .Axes(xlCategory).HasTitle = True
    .Axes(xlCategory).AxisTitle.Text = "Spot Price"
    .Axes(xlValue).HasTitle = True
    .Axes(xlValue).AxisTitle.Text = "P&L"
    On Error GoTo Fail
End With

Exit Sub
Fail:
MsgBox "BuildPayoffChart failed at stage [" & stage & "]: " & vbCrLf & _
    Err.Description, vbExclamation, "F2 Payoff Chart"
End Sub

```

```

' F3: STRATEGY TEMPLATES -- one-click leg pre-fill
' -----
' Each template reads D3 (Spot) + H3 (Default Multiplier) + an optional
' "width" parameter (distance between strikes) and writes legs 1-N
' into rows 8-13. Calls RefreshStrategyBuilder at the end.
'
' Strike rounding:
'   TWII / TXO -> nearest 100 (typical TXO strike grid)
'   2330 / Equity -> nearest 50 or 10 depending on spot
'   Otherwise -> nearest 1
'
' All templates default to qty = 1 contract, premium = 0 (user fills
' actual premiums after pre-fill).
' =====
'
' --- Helper: round strike to product grid ---
Private Function RoundStrike(spot As Double) As Double
    Dim grid As Double
    Select Case True
        Case spot >= 10000: grid = 100 ' TXO / TWII
        Case spot >= 1000: grid = 50
        Case spot >= 100: grid = 10
        Case Else: grid = 1
    End Select
    RoundStrike = Application.WorksheetFunction.Round(spot / grid, 0) * grid
End Function

Private Function StrikeGrid(spot As Double) As Double
    Select Case True
        Case spot >= 10000: StrikeGrid = 100
        Case spot >= 1000: StrikeGrid = 50
        Case spot >= 100: StrikeGrid = 10
        Case Else: StrikeGrid = 1
    End Select
End Function

' --- Helper: clear all leg rows ---
Private Sub ClearLegRows(wsS As Worksheet)
    Dim r As Long
    For r = 8 To 13
        wsS.Range(wsS.Cells(r, 2), wsS.Cells(r, 11)).ClearContents
        wsS.Cells(r, 1).Value = r - 7 ' restore leg #
    Next r
End Sub

' --- Helper: write one leg ---
Private Sub WriteLeg(wsS As Worksheet, legRow As Long, _
    legType As String, strike As Double, _
    qty As Long, direction As String, premium As Double, _
    Optional mult As Variant)
    wsS.Cells(legRow, 2).Value = legType
    wsS.Cells(legRow, 3).Value = strike
    wsS.Cells(legRow, 4).Value = qty
    If Not IsMissing(mult) Then
        wsS.Cells(legRow, 5).Value = mult
    Else
        wsS.Cells(legRow, 5).Value = ToDb1(wsS.Range("H3").Value)
    End If
    wsS.Cells(legRow, 6).Value = direction
    wsS.Cells(legRow, 7).Value = premium
End Sub

' --- Template helper: validate spot ---
Private Function GetSpotOrFail(wsS As Worksheet, label As String) As Double
    GetSpotOrFail = ToDb1(wsS.Range("D3").Value)
    If GetSpotOrFail <= 0 Then
        MsgBox "Set D3 (Underlying Price) before using template [" & label & "].", _
            vbExclamation, "Strategy Template"
    End If
End Function

```

```

' =====
' TEMPLATE 1: Bull Call Spread
'   Long  Call @ ATM
'   Short Call @ ATM + 1 grid
' Directional bullish, limited risk + limited reward
' =====

```

```

Public Sub Tmpl_BullCallSpread()
    Dim stage As String: stage = "init"
    On Error GoTo Fail
    stage = "open sheet"
    Dim wsS As Worksheet: Set wsS = ThisWorkbook.Sheets("StrategyBuilder")
    stage = "get spot"
    Dim s As Double: s = GetSpotOrFail(wsS, "Bull Call Spread")
    If s = 0 Then Exit Sub
    stage = "compute strikes"
    Dim atm As Double: atm = RoundStrike(s)
    Dim g As Double: g = StrikeGrid(s)
    stage = "write legs"
    Call ClearLegRows(wsS)
    Call WriteLeg(wsS, 8, "Call", atm, 1, "Long", 0)
    Call WriteLeg(wsS, 9, "Call", atm + g, 1, "Short", 0)
    stage = "refresh"
    Call RefreshStrategyBuilder
    Exit Sub
Fail:
    MsgBox "Tmpl_BullCallSpread failed at [" & stage & "]: " & Err.Description, vbExclamation
End Sub

```

```

' =====
' TEMPLATE 2: Bear Put Spread
'   Long  Put @ ATM
'   Short Put @ ATM - 1 grid
' =====

```

```

Public Sub Tmpl_BearPutSpread()
    Dim stage As String: stage = "init"
    On Error GoTo Fail
    stage = "open sheet"
    Dim wsS As Worksheet: Set wsS = ThisWorkbook.Sheets("StrategyBuilder")
    stage = "get spot"
    Dim s As Double: s = GetSpotOrFail(wsS, "Bear Put Spread")
    If s = 0 Then Exit Sub
    stage = "compute strikes"
    Dim atm As Double: atm = RoundStrike(s)
    Dim g As Double: g = StrikeGrid(s)
    stage = "write legs"
    Call ClearLegRows(wsS)
    Call WriteLeg(wsS, 8, "Put", atm, 1, "Long", 0)
    Call WriteLeg(wsS, 9, "Put", atm - g, 1, "Short", 0)
    stage = "refresh"
    Call RefreshStrategyBuilder
    Exit Sub
Fail:
    MsgBox "Tmpl_BearPutSpread failed at [" & stage & "]: " & Err.Description, vbExclamation
End Sub

```

```

' =====
' TEMPLATE 3: Iron Condor (1 grid wings, 1 grid inner)
'   Short Put @ ATM - 1g
'   Long  Put @ ATM - 2g
'   Short Call @ ATM + 1g
'   Long  Call @ ATM + 2g
' Range-bound, sell premium, defined risk
' =====

```

```

Public Sub Tmpl_IronCondor()
    Dim stage As String: stage = "init"
    On Error GoTo Fail
    stage = "open sheet"
    Dim wsS As Worksheet: Set wsS = ThisWorkbook.Sheets("StrategyBuilder")
    stage = "get spot"
    Dim s As Double: s = GetSpotOrFail(wsS, "Iron Condor")
    If s = 0 Then Exit Sub

```

```

    stage = "compute strikes"
    Dim atm As Double: atm = RoundStrike(s)
    Dim g As Double: g = StrikeGrid(s)
    stage = "write legs"
    Call ClearLegRows(wsS)
    Call WriteLeg(wsS, 8, "Put", atm - g, 1, "Short", 0)
    Call WriteLeg(wsS, 9, "Put", atm - 2 * g, 1, "Long", 0)
    Call WriteLeg(wsS, 10, "Call", atm + g, 1, "Short", 0)
    Call WriteLeg(wsS, 11, "Call", atm + 2 * g, 1, "Long", 0)
    stage = "refresh"
    Call RefreshStrategyBuilder
    Exit Sub
Fail:
    MsgBox "Tmpl_IronCondor failed at [" & stage & "]: " & Err.Description, vbExclamation
End Sub

```

```

' TEMPLATE 4: Long Straddle
'   Long Call @ ATM
'   Long Put  @ ATM
' Volatility play, unlimited upside, defined downside (= premium paid)

```

```

Public Sub Tmpl_LongStraddle()
    Dim stage As String: stage = "init"
    On Error GoTo Fail
    stage = "open sheet"
    Dim wsS As Worksheet: Set wsS = ThisWorkbook.Sheets("StrategyBuilder")
    stage = "get spot"
    Dim s As Double: s = GetSpotOrFail(wsS, "Long Straddle")
    If s = 0 Then Exit Sub
    stage = "compute strike"
    Dim atm As Double: atm = RoundStrike(s)
    stage = "write legs"
    Call ClearLegRows(wsS)
    Call WriteLeg(wsS, 8, "Call", atm, 1, "Long", 0)
    Call WriteLeg(wsS, 9, "Put", atm, 1, "Long", 0)
    stage = "refresh"
    Call RefreshStrategyBuilder
    Exit Sub
Fail:
    MsgBox "Tmpl_LongStraddle failed at [" & stage & "]: " & Err.Description, vbExclamation
End Sub

```

```

' TEMPLATE 5: Short Strangle
'   Short Call @ ATM + 1g
'   Short Put  @ ATM - 1g
' Range-bound, sell premium, UNLIMITED risk (be careful)

```

```

Public Sub Tmpl_ShortStrangle()
    Dim stage As String: stage = "init"
    On Error GoTo Fail
    stage = "open sheet"
    Dim wsS As Worksheet: Set wsS = ThisWorkbook.Sheets("StrategyBuilder")
    stage = "get spot"
    Dim s As Double: s = GetSpotOrFail(wsS, "Short Strangle")
    If s = 0 Then Exit Sub
    stage = "compute strikes"
    Dim atm As Double: atm = RoundStrike(s)
    Dim g As Double: g = StrikeGrid(s)
    stage = "write legs"
    Call ClearLegRows(wsS)
    Call WriteLeg(wsS, 8, "Call", atm + g, 1, "Short", 0)
    Call WriteLeg(wsS, 9, "Put", atm - g, 1, "Short", 0)
    stage = "refresh"
    Call RefreshStrategyBuilder
    Exit Sub
Fail:
    MsgBox "Tmpl_ShortStrangle failed at [" & stage & "]: " & Err.Description, vbExclamation
End Sub

```

```

' =====
' TEMPLATE 6: Calendar Spread (same strike, different DTE)
'   Short Call @ ATM (near-term -- user adjusts DTE manually)
'   Long  Call @ ATM (longer-term)
' NOTE: This builder uses ONE shared DTE in B4. Calendar spreads
' require per-leg DTE which isn't modeled yet. The output is informational.
' =====

```

```

Public Sub Tmpl_CalendarSpread()
    Dim stage As String: stage = "init"
    On Error GoTo Fail
    stage = "open sheet"
    Dim wsS As Worksheet: Set wsS = ThisWorkbook.Sheets("StrategyBuilder")
    stage = "get spot"
    Dim s As Double: s = GetSpotOrFail(wsS, "Calendar Spread")
    If s = 0 Then Exit Sub
    stage = "compute strikes"
    Dim atm As Double: atm = RoundStrike(s)
    stage = "write legs"
    Call ClearLegRows(wsS)
    Call WriteLeg(wsS, 8, "Call", atm, 1, "Short", 0)
    Call WriteLeg(wsS, 9, "Call", atm, 1, "Long", 0)
    stage = "refresh"
    Call RefreshStrategyBuilder
    MsgBox "Calendar Spread pre-filled (same strike, both Call)." & vbCrLf & _
        "NOTE: This builder uses ONE shared DTE/IV. Calendar spread" & vbCrLf & _
        "Greeks are approximate -- both legs share the SAME expiry here.", _
        vbInformation, "Calendar Spread"
    Exit Sub
Fail:
    MsgBox "Tmpl_CalendarSpread failed at [" & stage & "]: " & Err.Description, vbExclamation
End Sub

```

```

' -----
' Compute strategy payoff at a single underlying price (at expiry).
' Inputs are pre-cached arrays from RefreshStrategyBuilder.
' -----

```

```

Private Function PayoffAtExpiry(px As Double, _
    lgType() As String, lgStrike() As Double, _
    lgQty() As Double, lgPrem() As Double, n As Long) As Double
    Dim total As Double: total = 0
    Dim i As Long
    For i = 1 To n
        Dim intr As Double
        If lgType(i) = "Call" Then
            intr = px - lgStrike(i)
            If intr < 0 Then intr = 0
        Else
            intr = lgStrike(i) - px
            If intr < 0 Then intr = 0
        End If
        ' lgQty already signed: long=+, short=-
        total = total + lgQty(i) * (intr - lgPrem(i))
    Next i
    PayoffAtExpiry = total
End Function

```

```

' -----
' Safe numeric conversion: blank / text -> 0
' -----

```

```

Private Function ToDbl(v As Variant) As Double
    If IsNumeric(v) Then
        ToDbl = CDbl(v)
    Else
        ToDbl = 0
    End If
End Function

```

test - 1

```
Sub TestSaveFlow()  
    Dim msg As String  
    msg = "=== RR5 Save 診斷 ===" & vbCrLf  
  
    ' 1. Transactions 工作表  
    On Error Resume Next  
    Dim wsT As Worksheet  
    Set wsT = ThisWorkbook.Sheets("Transactions")  
    On Error GoTo 0  
    msg = msg & "Transactions sheet: " & IIf(wsT Is Nothing, "MISSING", "OK") & vbCrLf  
  
    ' 2. _Config 工作表  
    On Error Resume Next  
    Dim wsC As Worksheet  
    Set wsC = ThisWorkbook.Sheets("_Config")  
    On Error GoTo 0  
    msg = msg & "_Config sheet: " & IIf(wsC Is Nothing, "MISSING (GetConfigValue 會炸)", "OK") & vbCrLf  
  
    ' 3. DebugLog 工作表  
    On Error Resume Next  
    Dim wsD As Worksheet  
    Set wsD = ThisWorkbook.Sheets("DebugLog")  
    On Error GoTo 0  
    msg = msg & "DebugLog sheet: " & IIf(wsD Is Nothing, "MISSING (AppendTransaction 會炸)", "OK") & vbCrLf  
  
    ' 4. GenerateTxID  
    On Error Resume Next  
    Dim txID As String  
    txID = RR5_Core.GenerateTxID()  
    msg = msg & "GenerateTxID: " & IIf(Err.Number <> 0, "ERROR " & Err.Description, txID) & vbCrLf  
    Err.Clear  
    On Error GoTo 0  
  
    ' 5. GetConfigValue  
    On Error Resume Next  
    Dim cfgVal As Variant  
    cfgVal = RR5_Core.GetConfigValue("BS_RISK_FREE_RATE")  
    msg = msg & "GetConfigValue: " & IIf(Err.Number <> 0, "ERROR " & Err.Description, CStr(cfgVal)) & vbCrLf  
    Err.Clear  
    On Error GoTo 0  
  
    MsgBox msg, vbInformation, "診斷結果"  
End Sub
```